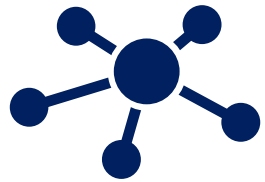
Computer Networking



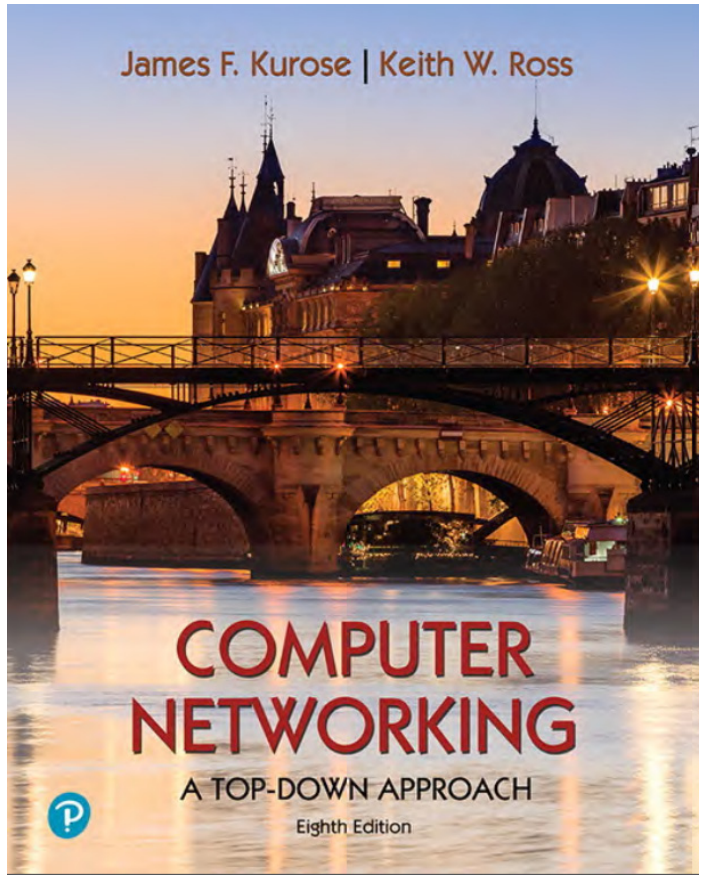
谢 逸

中山大学 • 计算机学院

2024. Fall



Chapter 3
Transport Layer



Computer Networking: A Top-Down Approach
8th edition
Jim Kurose, Keith Ross
Pearson, 2020

A note on the use of these Powerpoint slides:

We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

All material copyright 1996-2016
J.F Kurose and K.W. Ross, All Rights Reserved

©

Assignments :

- Ch3 (ver8, CN): 3、4、7、14、15、20、22、27、32、36、40、43、48、52、56、58

- Keywords: multiplexing, demultiplexing, reliable data transfer, flow control, congestion control, UDP, TCP, Go back n, Selective repeat, TCP window, AIMD

Chapter 3: Transport Layer

our goals:

- understand principles behind transport layer services:
  - multiplexing, demultiplexing
  - reliable data transfer
  - flow control
  - congestion control
- learn about Internet transport layer protocols:
  - UDP: connectionless transport
  - TCP: connection-oriented reliable transport
  - TCP congestion control

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

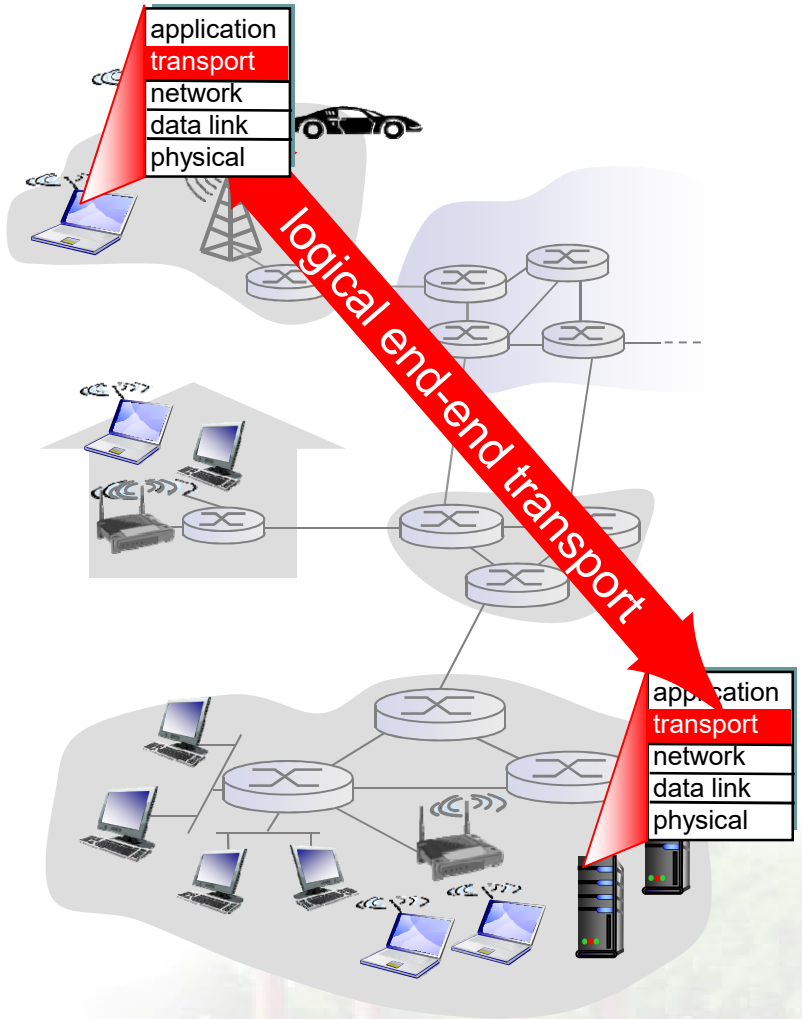
- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

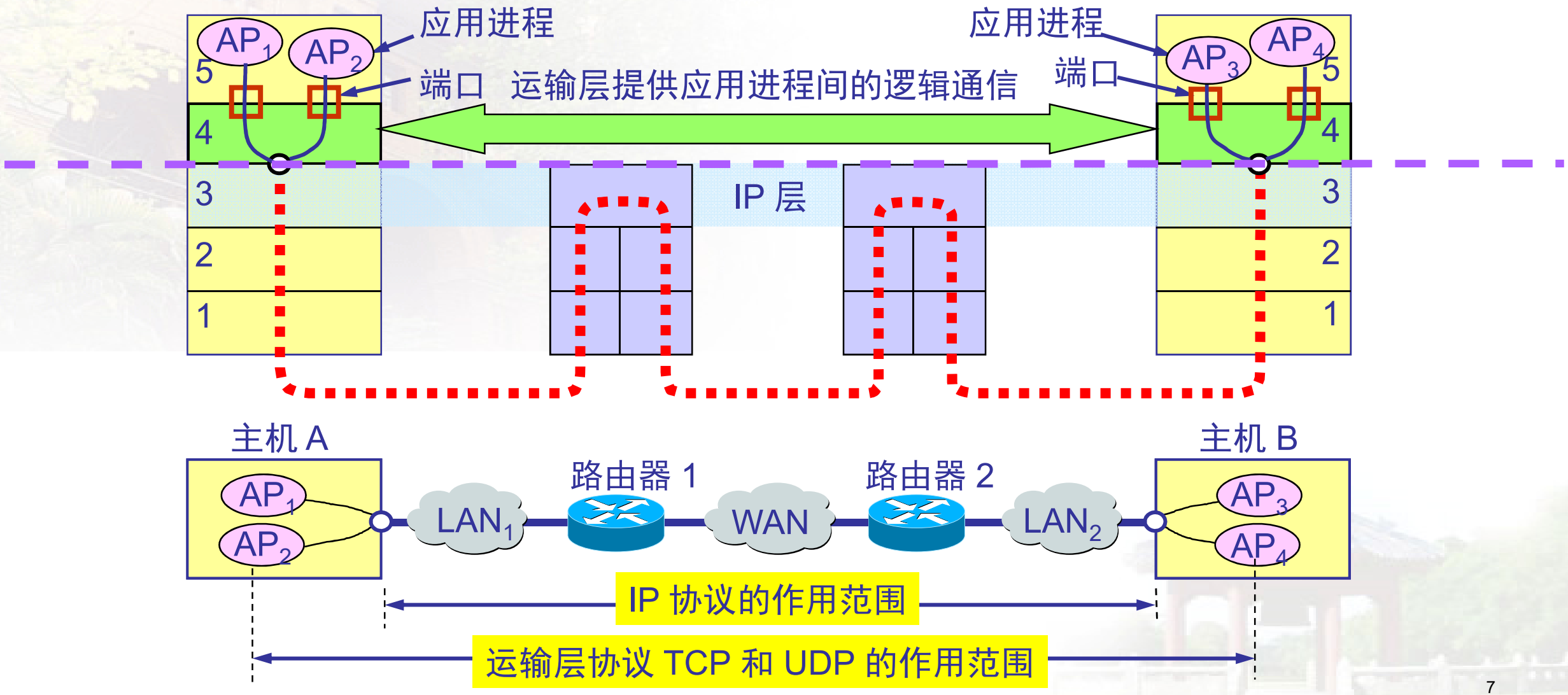
Transport services and protocols

- provide *logical communication* between app processes running on different hosts
- transport protocols run in end systems
  - send side: breaks app messages into *segments*, passes to network layer
  - rcv side: reassembles segments into messages, passes to app layer
- more than one transport protocol available to apps
  - Internet: TCP and UDP



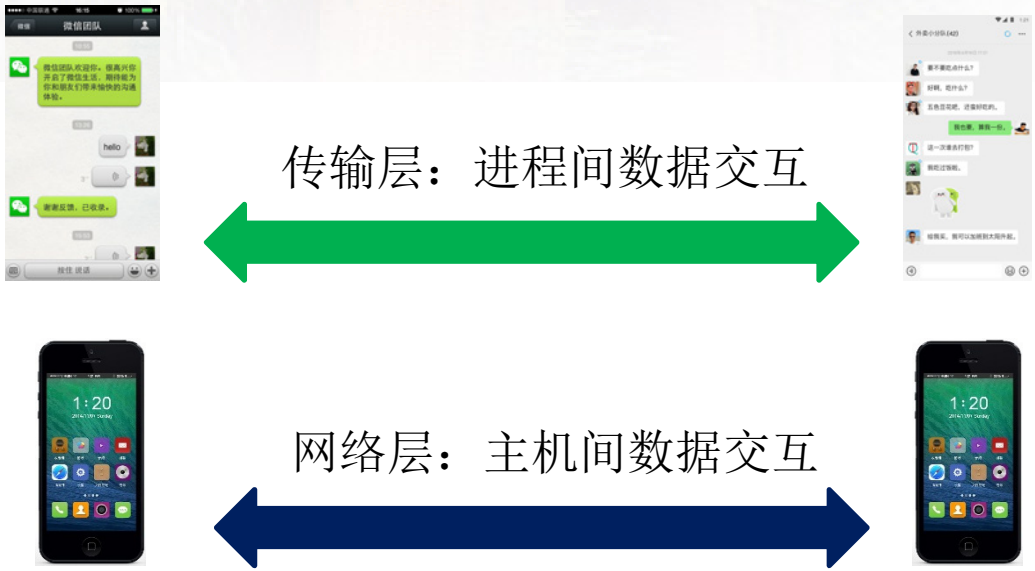
20241012

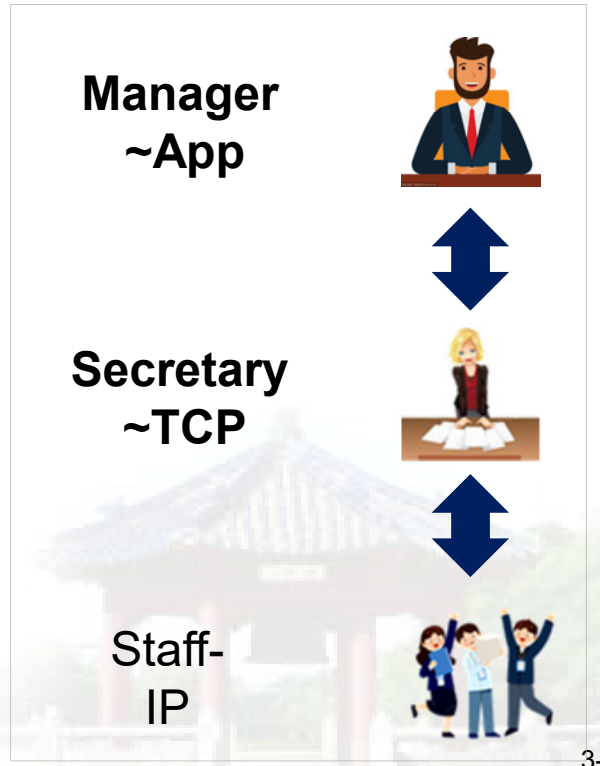
Transport vs. network layer



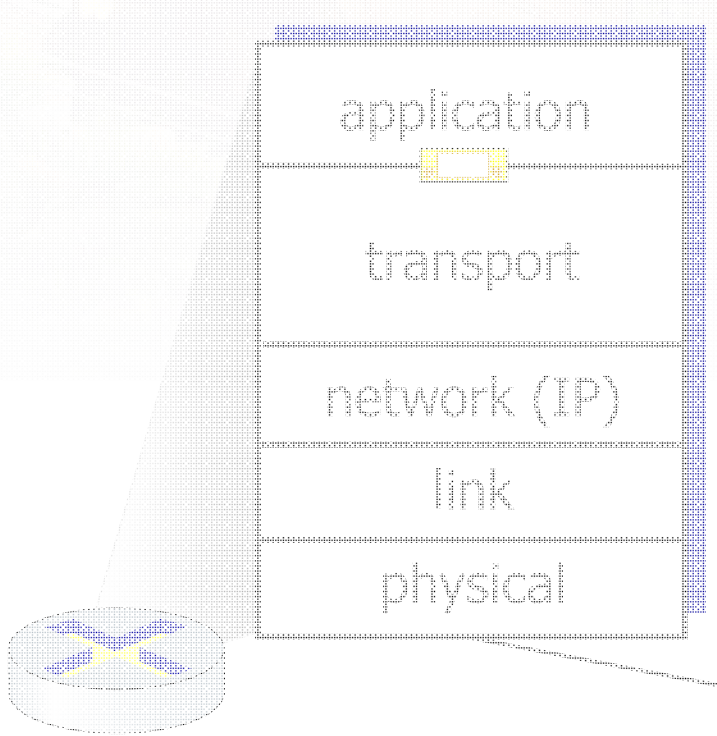
Transport vs. network layer

- *network layer:* logical communication between hosts
- *transport layer:* logical communication between processes
  - relies on, enhances, network layer services



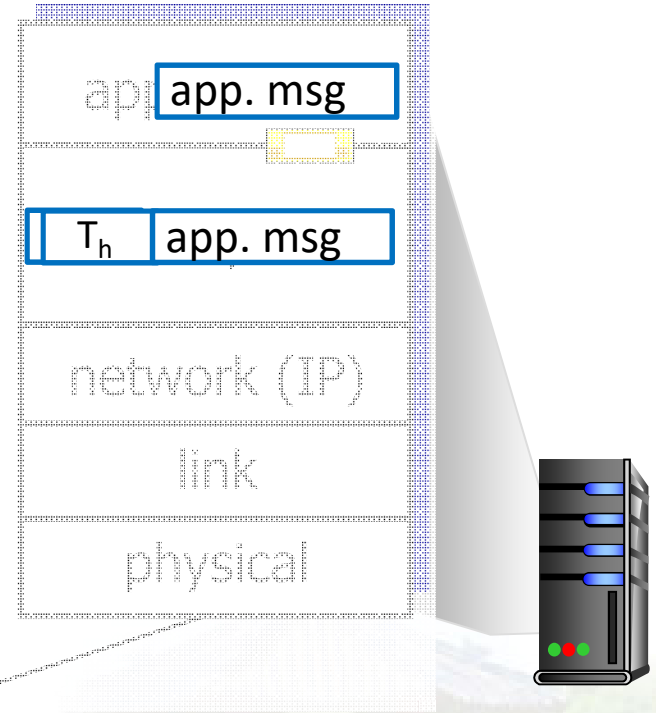


Transport Layer Actions

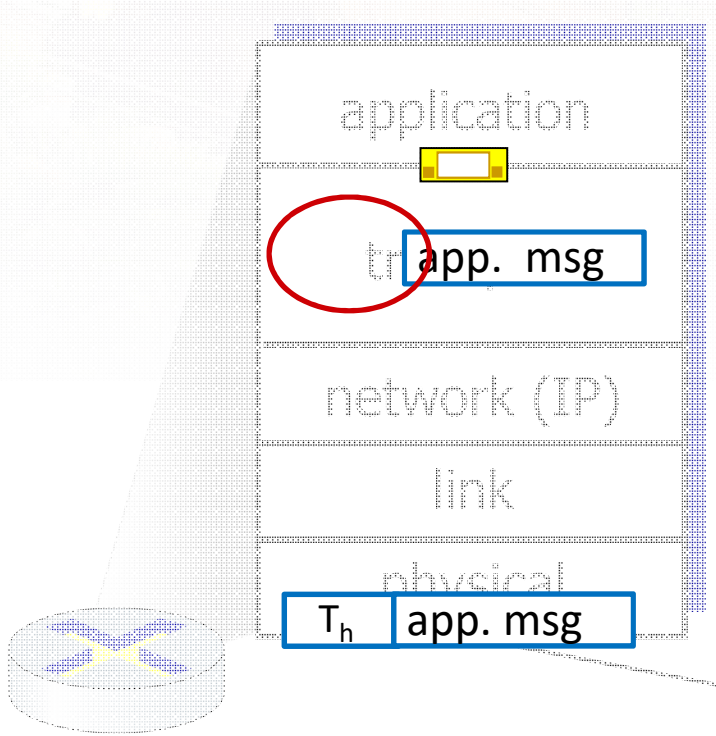


Sender:

- is passed an application-layer message
- determines segment header fields values
- creates segment
- passes segment to IP

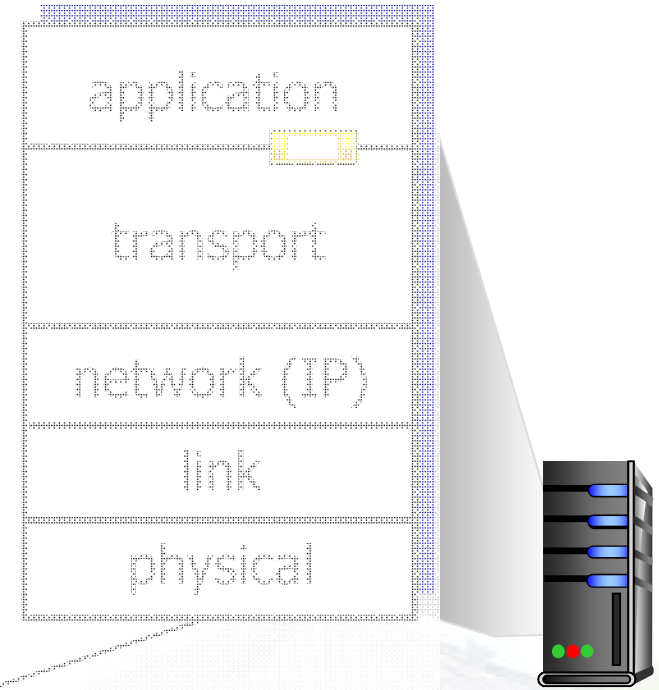


Transport Layer Actions



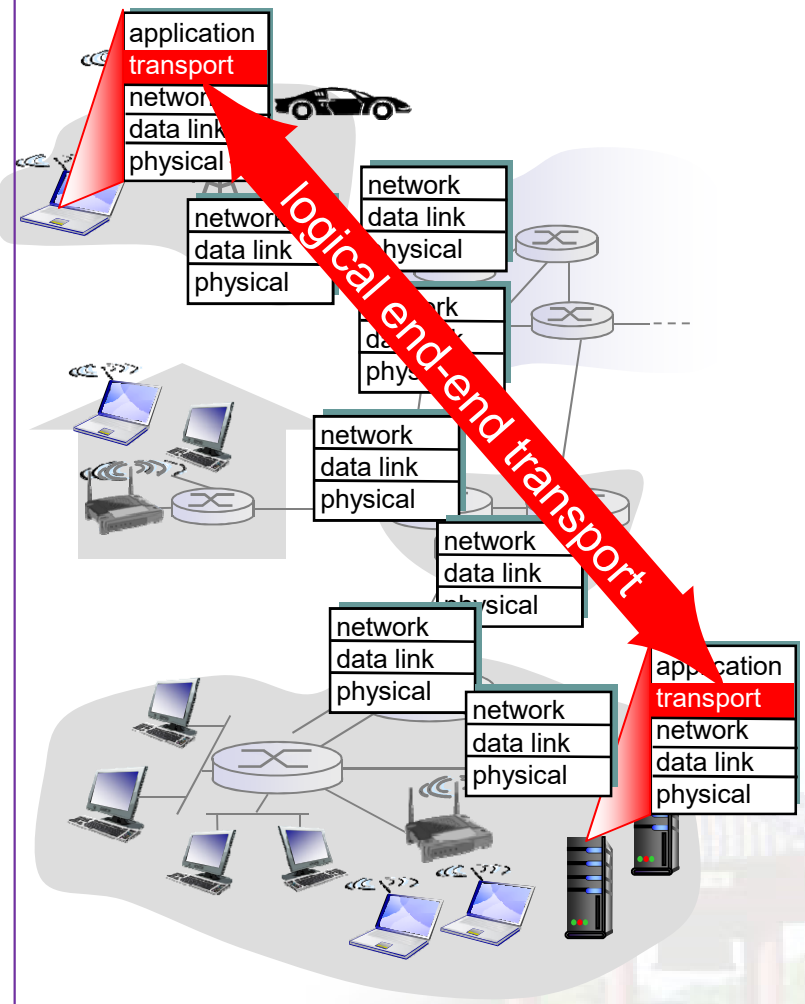
Receiver:

- receives segment from IP
- checks header values
- extracts application-layer message
- demultiplexes message up to application via socket



Internet transport-layer protocols

- reliable, in-order delivery (TCP)
  - congestion control
  - flow control
  - connection setup
- unreliable, unordered delivery: UDP
  - no-frills extension of "best-effort" IP
- services not available:
  - delay guarantees
  - bandwidth guarantees



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

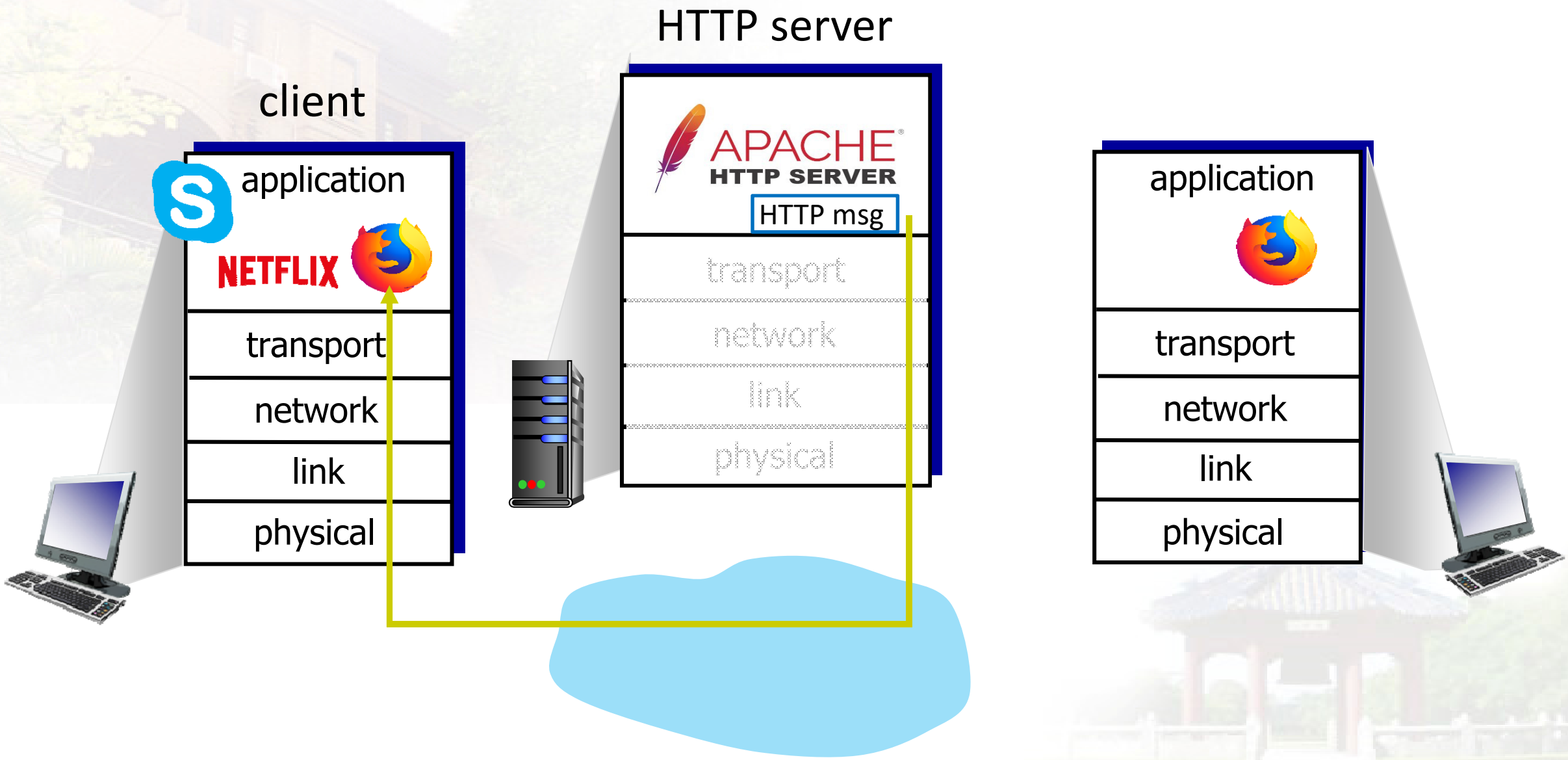
3.4 principles of reliable data transfer

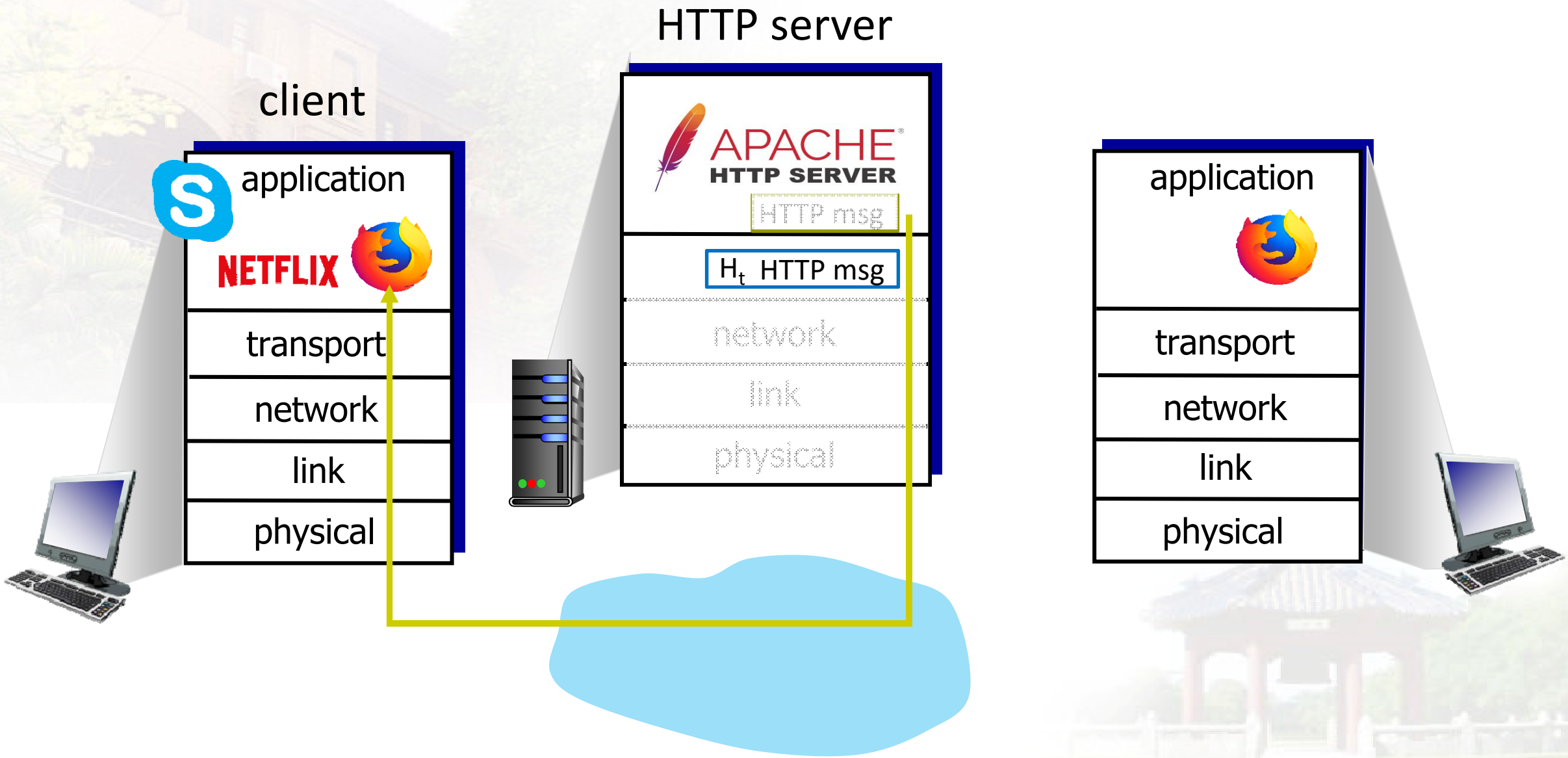
3.5 connection-oriented transport: TCP

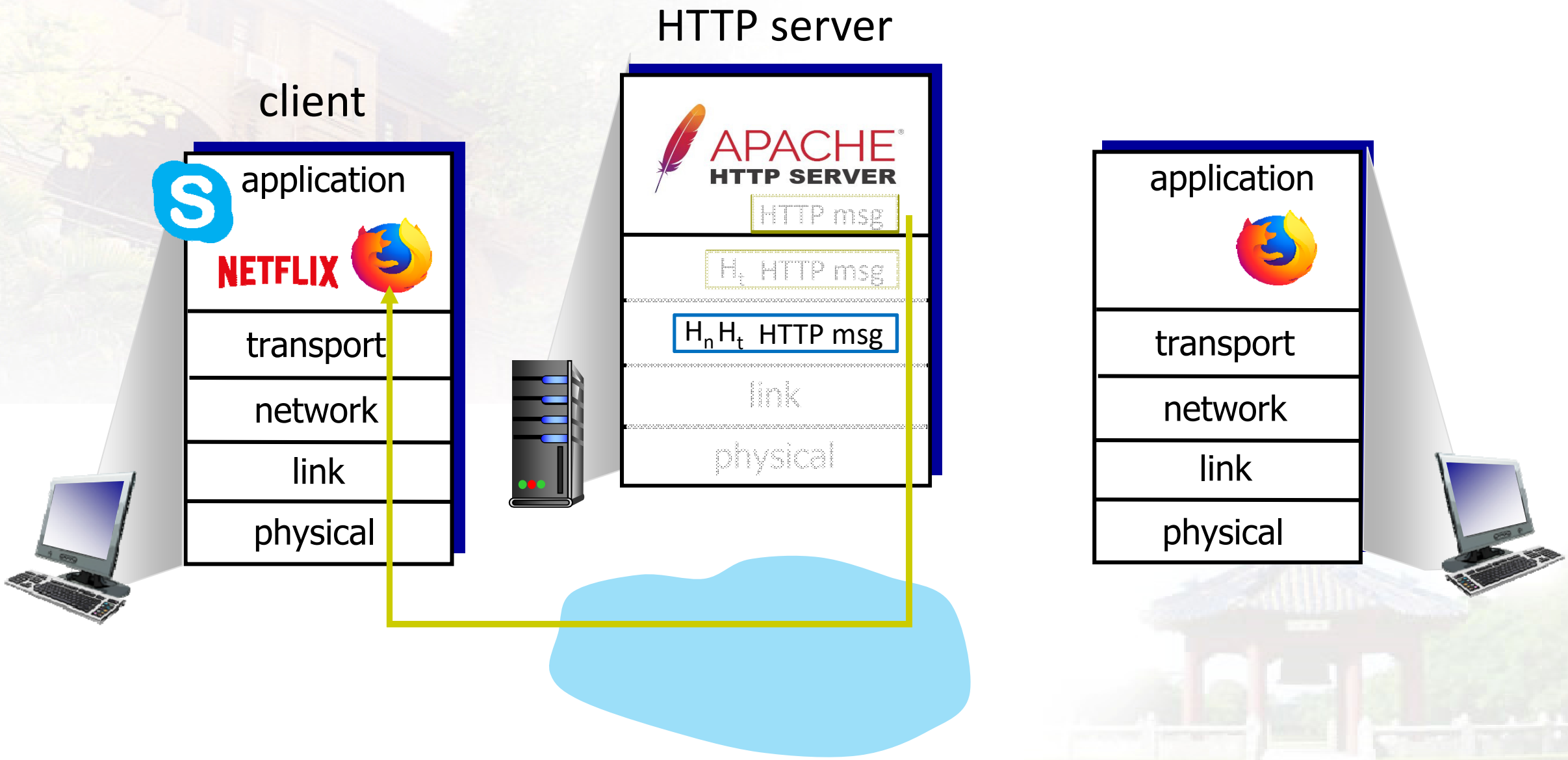
- segment structure
- reliable data transfer
- flow control
- connection management

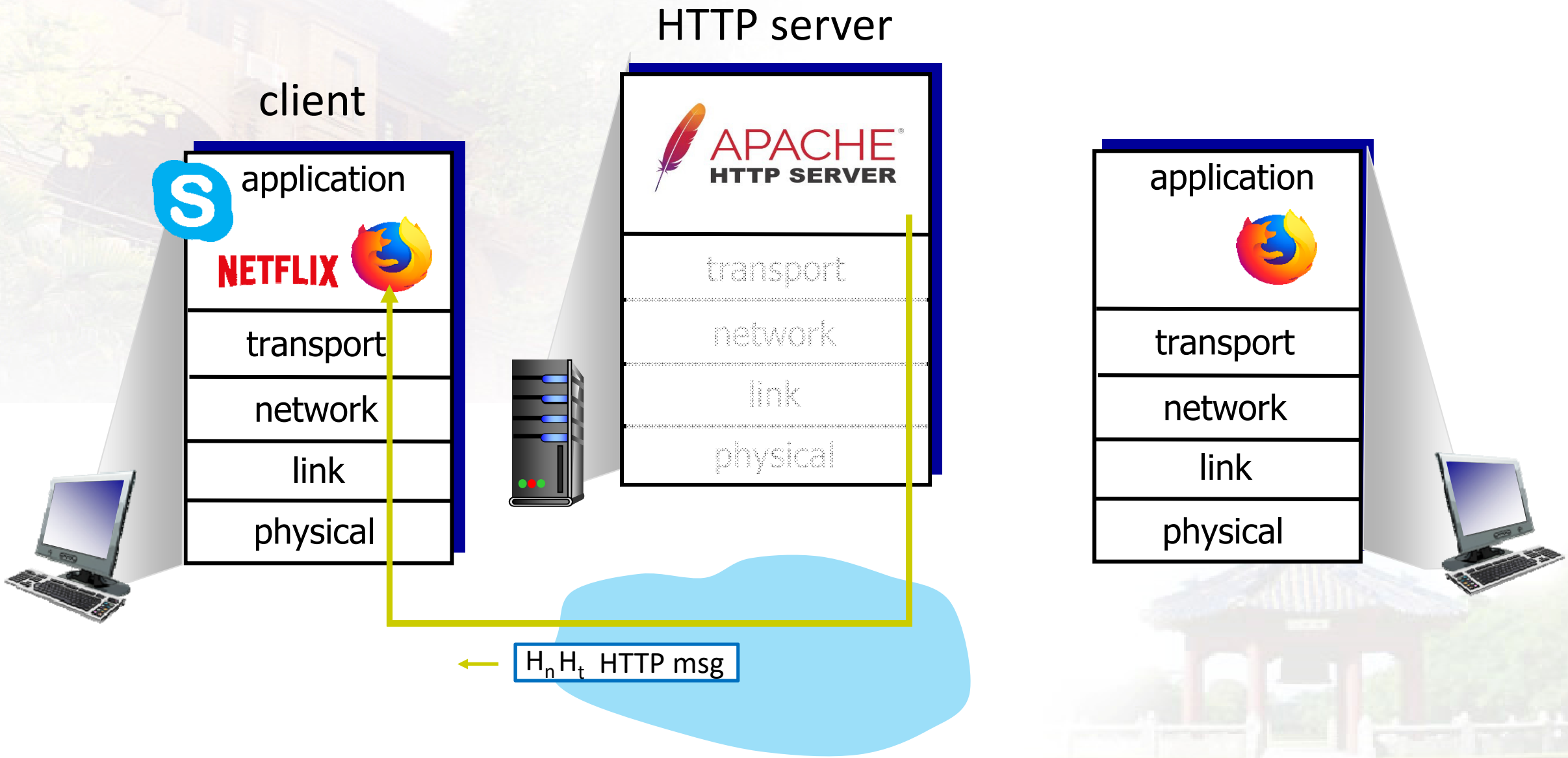
3.6 principles of congestion control

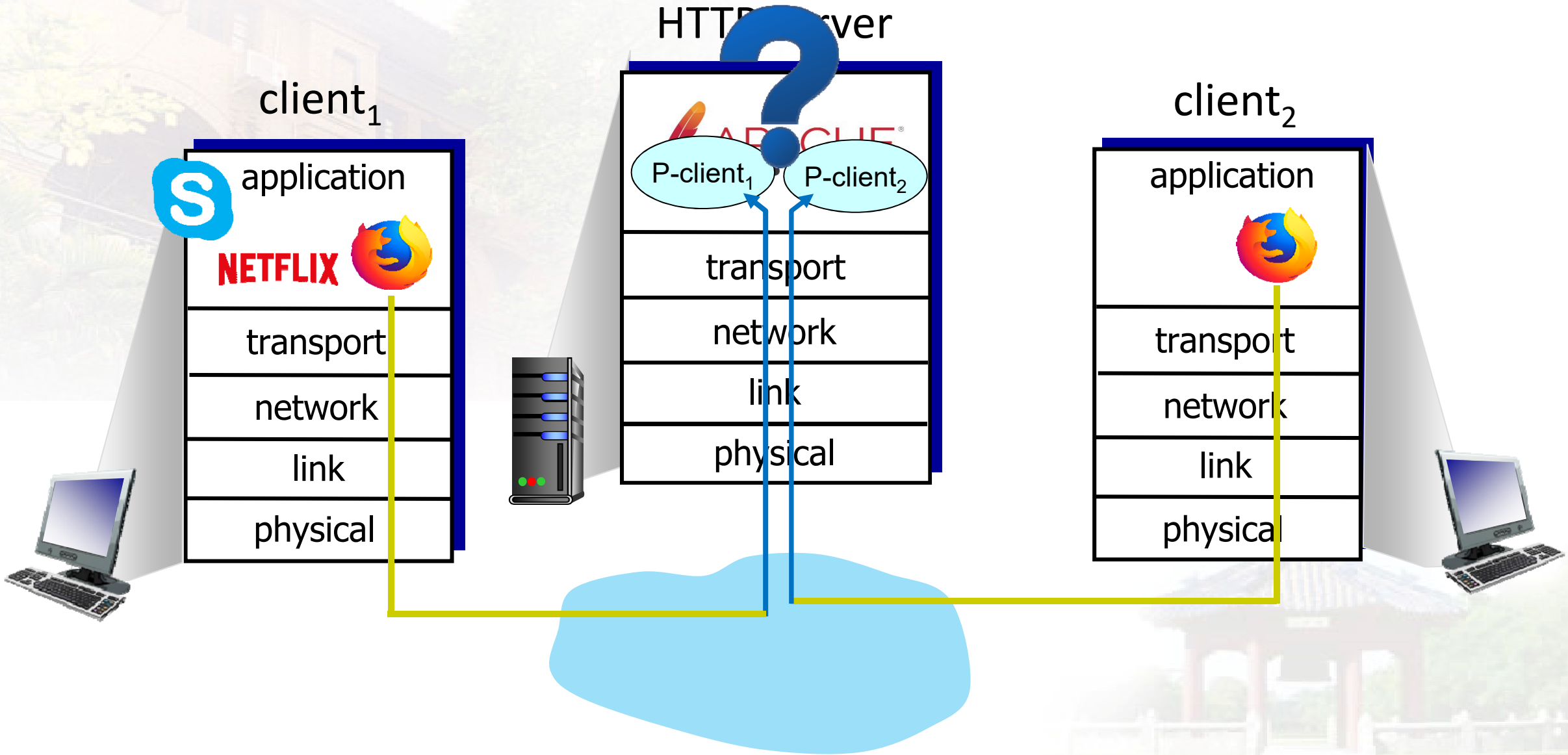
3.7 TCP congestion control


client
application
NETFLIX
transport
network
link
physical
HTTP server
APACHE
HTTP SERVER
HTTP msg
transport
network
link
physical
application
transport
network
link
physical


HTTP server
client
application
NETFLIX
transport
network
link
physical
APACHE
HTTP SERVER
HTTP msg
H_t HTTP msg
network
link
physical
application
transport
network
link
physical


HTTP server
client
application
NETFLIX
transport
network
link
physical
APACHE
HTTP SERVER
HTTP msg
H_t HTTP msg
$H_n H_t$ HTTP msg
link
physical
application
transport
network
link
physical


HTTP server
client
application
NETFLIX
transport
network
link
physical
APACHE
HTTP SERVER
transport
network
link
physical
application
transport
network
link
physical
$H_n H_t$ HTTP msg


client1
application
NETFLIX
transport
network
link
physical
HTTP server
P-client1
P-client2
transport
network
link
physical
client2
application
transport
network
link
physical

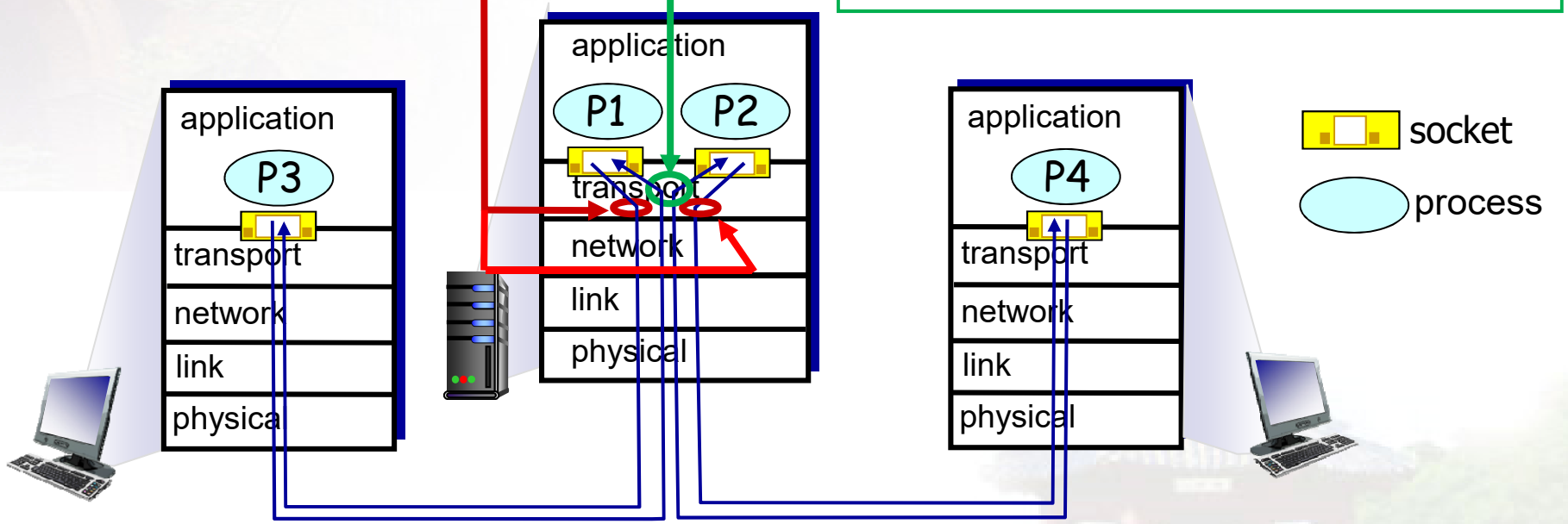
Multiplexing/demultiplexing

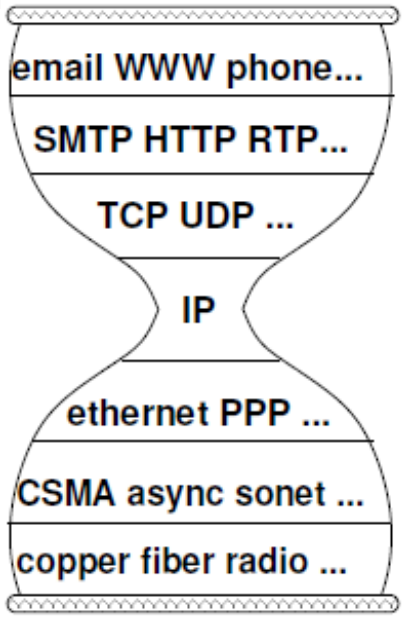
multiplexing at sender:

handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing at receiver:

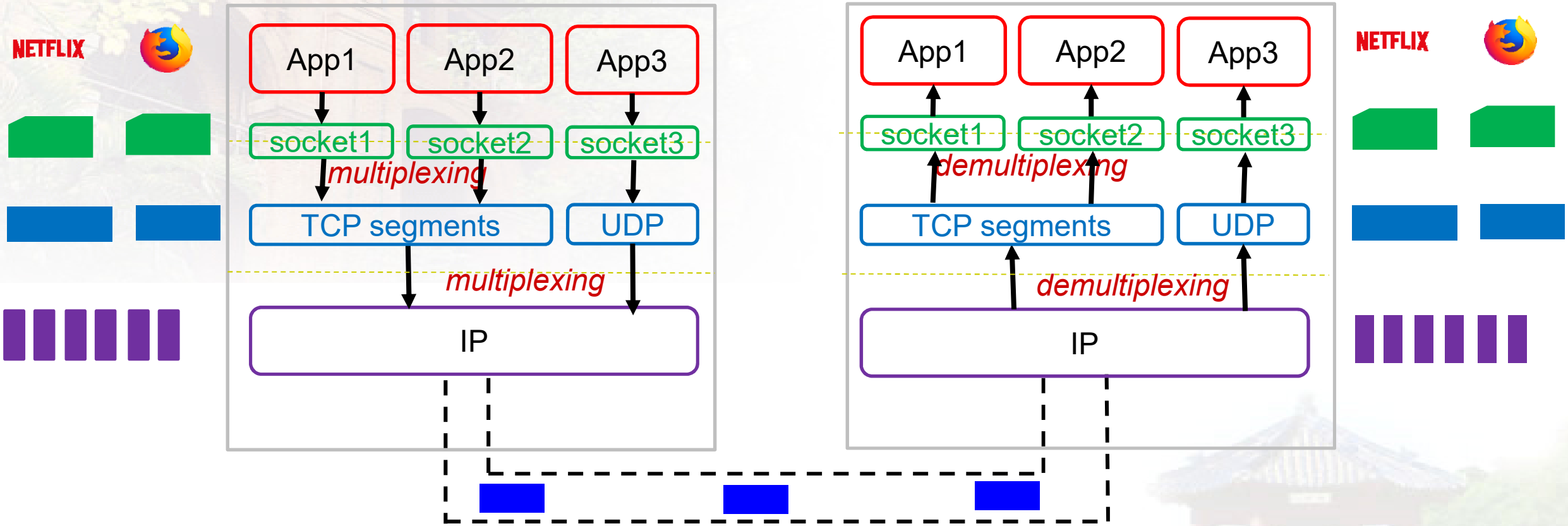
use header info to deliver received segments to correct socket





Multiplexing: Aggregate data from different applications to the TCP and IP layers
Demultiplexing: Distribute network layer data to different applications

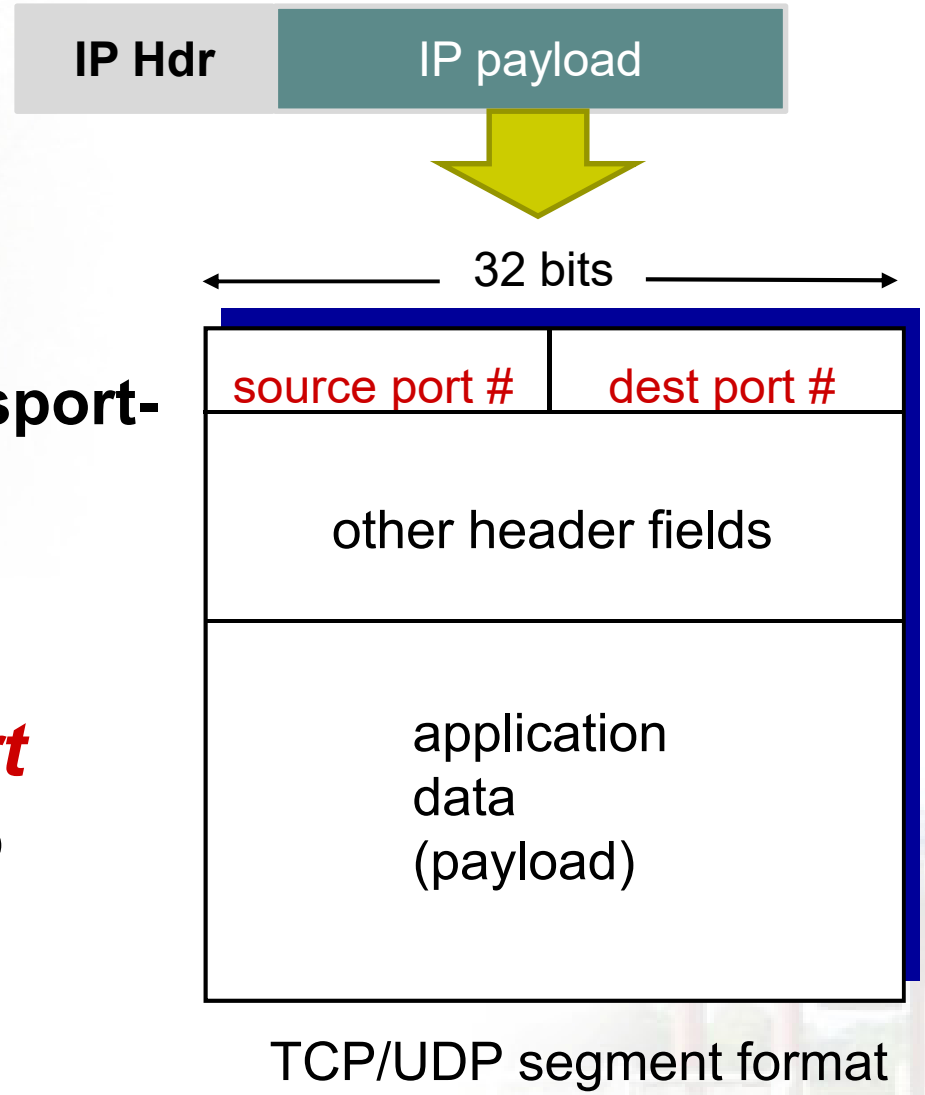
Multiplexing/demultiplexing



How demultiplexing works

- host receives IP datagrams
  - each datagram has source IP address, destination IP address
  - each datagram carries one transport-layer segment
  - each segment has source, destination port number
- host uses *IP addresses & port numbers* to direct segment to appropriate socket

srcIP+dstIP+srcPort+dstPort+protocol



TCP/UDP segment format

Connectionless demultiplexing

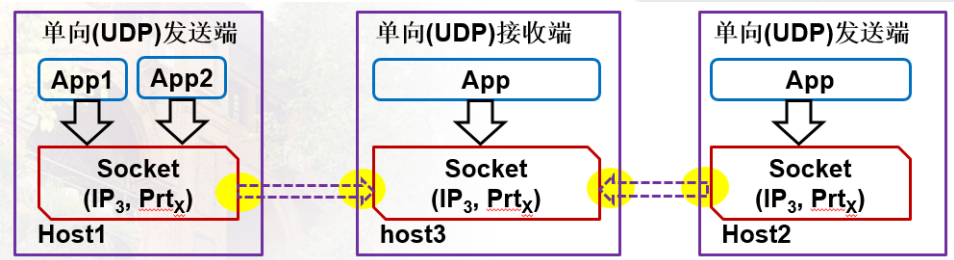
- ***recall:*** **created socket has host-local port #:**

```
DatagramSocket mySocket1
= new DatagramSocket(12534);
```

- *recall:* when creating datagram to send into UDP socket, must specify
  - destination IP address
  - destination port #

- **when host receives UDP segment:**
  - **checks destination port # in segment**
  - **directs UDP segment to socket with that port #**

IP datagrams with *same dest. port #,* but different source IP addresses and/or source port numbers will be directed to *same socket* at dest



Connectionless demux: example

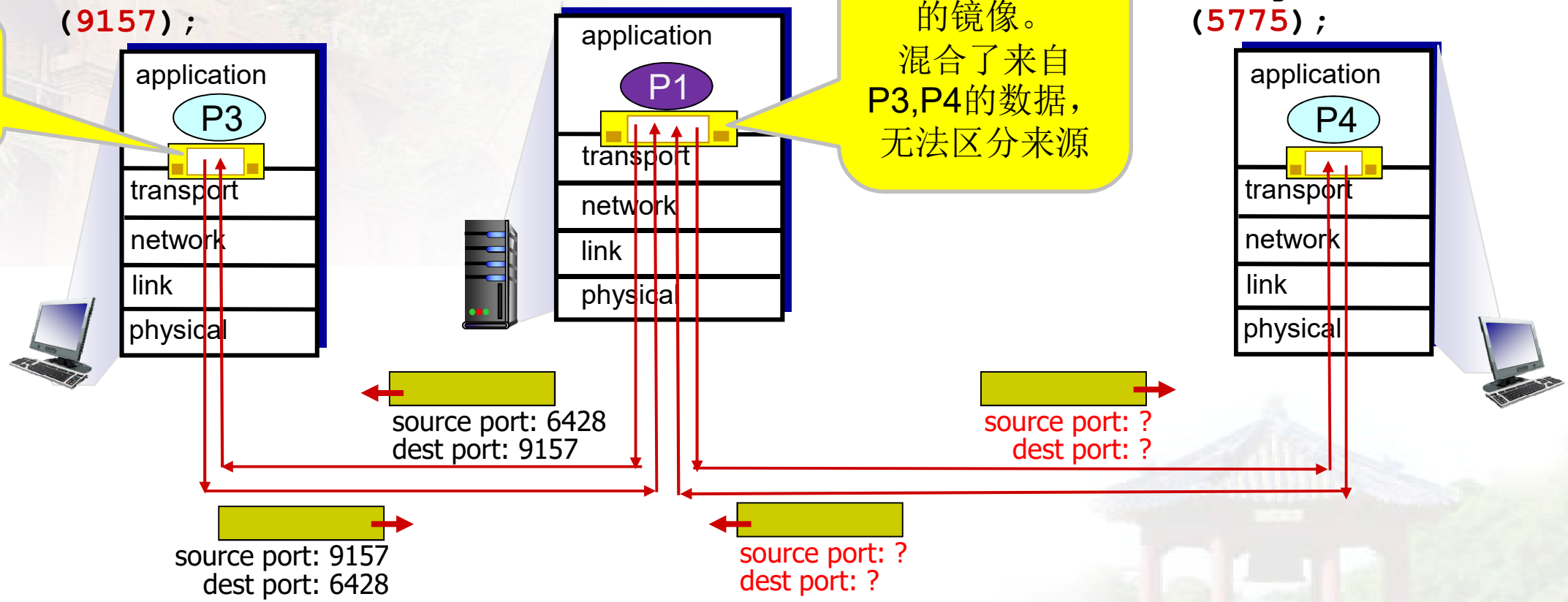
```
DatagramSocket
 serverSocket = new
 DatagramSocket
 (6428);
```

```
DatagramSocket
 mySocket2 = new
 DatagramSocket
 (9157);
```

```
DatagramSocket
 mySocket1 = new
 DatagramSocket
 (5775);
```

接收端程序在本地的一个镜像

发送端在本地的镜像。混合了来自P3,P4的数据，无法区分来源

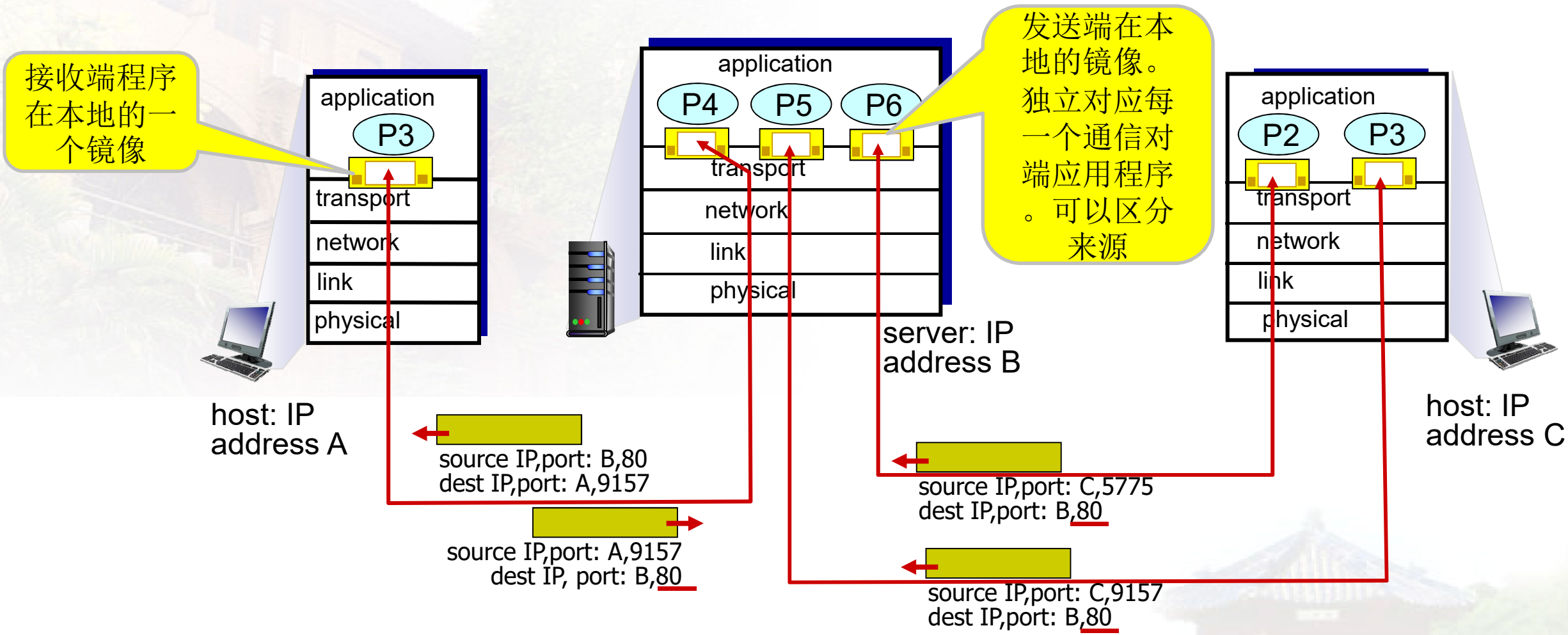


接收端程序（**P1**）无需区分所收到数据是来自哪一个发送端

Connection-oriented demux

- TCP socket identified by 4-tuple:
  - source IP address
  - source port number
  - dest IP address
  - dest port number
- demux: receiver uses all four values to direct segment to appropriate socket
- server host may support many simultaneous TCP sockets:
  - each socket identified by its own 4-tuple
- web servers have different sockets for each connecting client
  - non-persistent HTTP will have different socket for each request

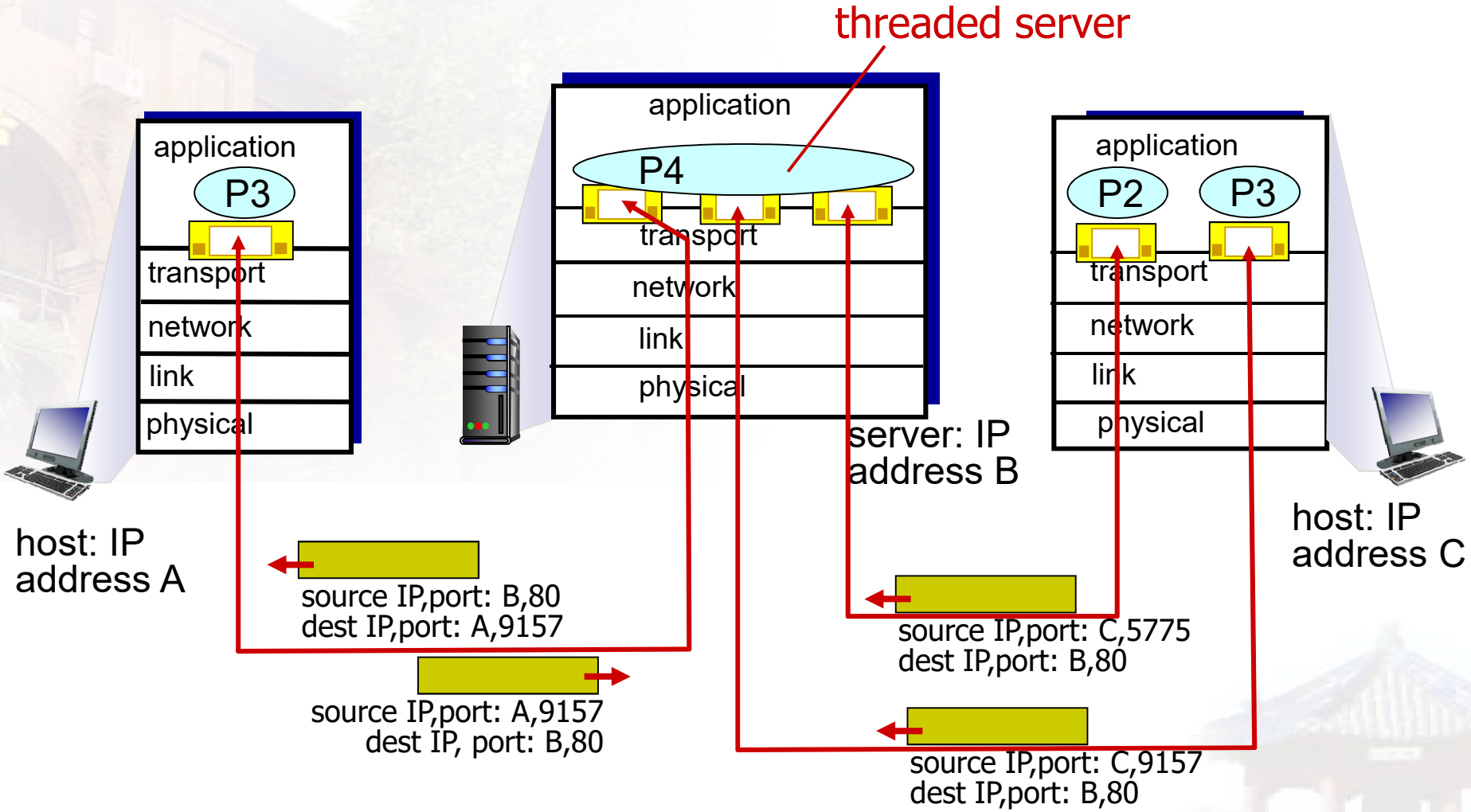
Connection-oriented demux: example



three segments, all destined to IP address: B, dest port: 80 are demultiplexed to *different* sockets

Different Apps (Instances)

Connection-oriented demux: example



Summary

- Multiplexing, demultiplexing: based on segment, datagram header field values
- **UDP:** demultiplexing using destination port number (only)
- **TCP:** demultiplexing using 4-tuple: source and destination IP addresses, and port numbers
- Multiplexing/demultiplexing happen at *all* layers

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

UDP: User Datagram Protocol [RFC 768]

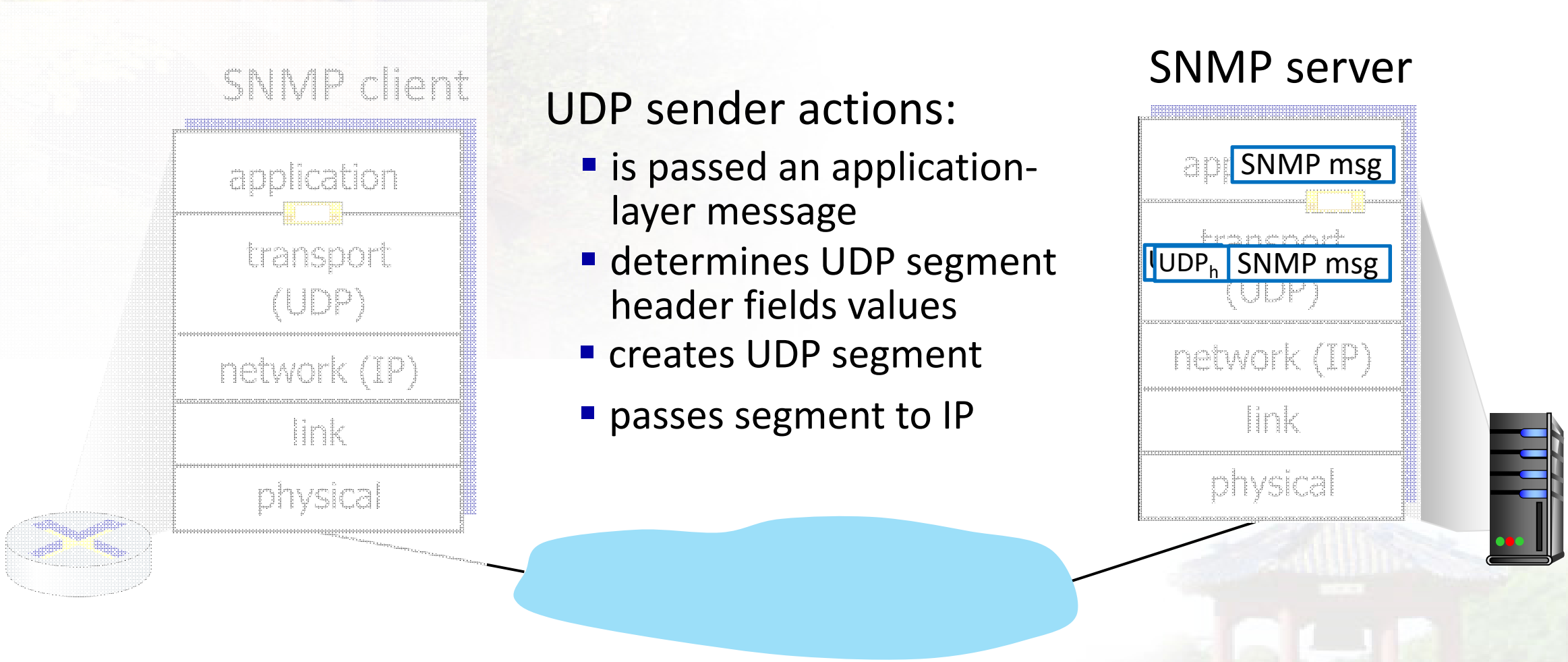
- **"no frills," "bare bones" Internet transport protocol**
- **"best effort" service, UDP segments may be:**
  - **lost**
  - **delivered out-of-order to app**
- ***connectionless:***
  - **no handshaking between UDP sender, receiver**
  - **each UDP segment handled independently of others**

- UDP use:
  - streaming multimedia apps (loss tolerant, rate sensitive)
  - DNS
  - SNMP
- reliable transfer over UDP:
  - add reliability at application layer
  - application-specific error recovery!

UDP: Transport Layer Actions



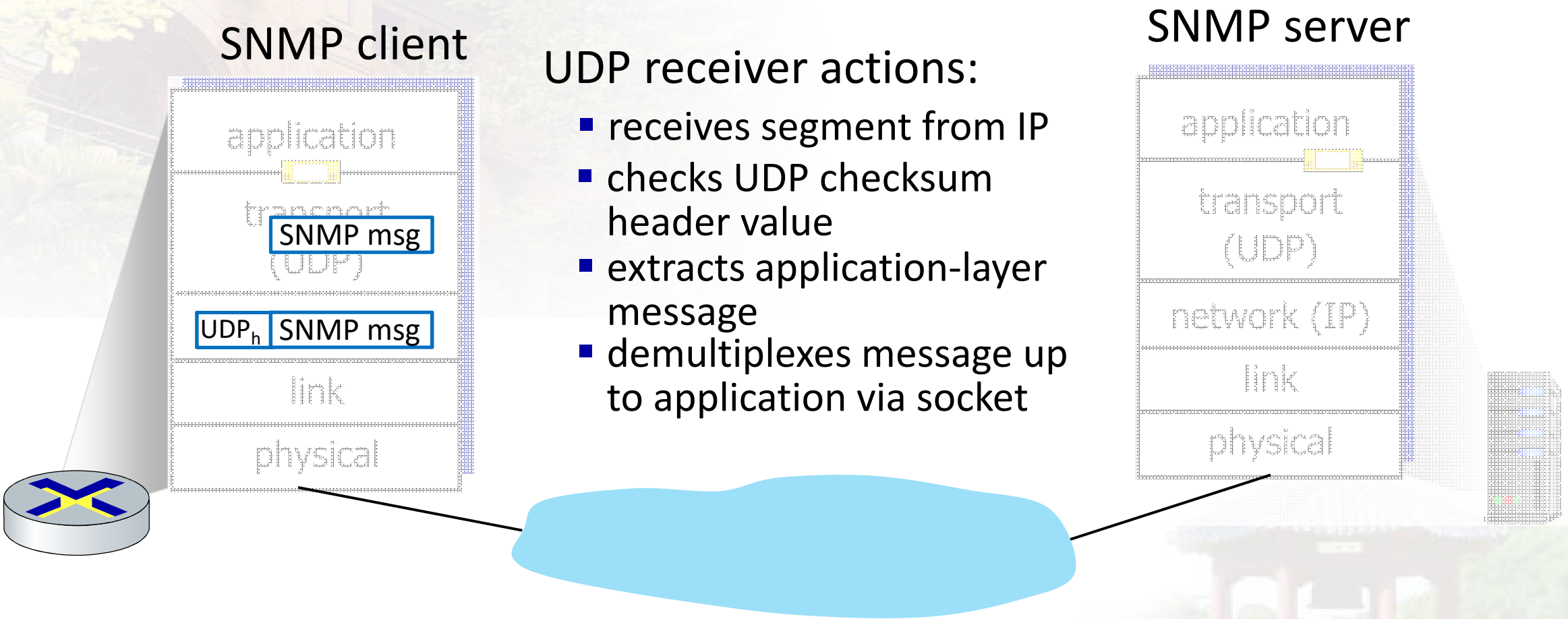
UDP: Transport Layer Actions



UDP sender actions:

- is passed an application-layer message
- determines UDP segment header fields values
- creates UDP segment
- passes segment to IP

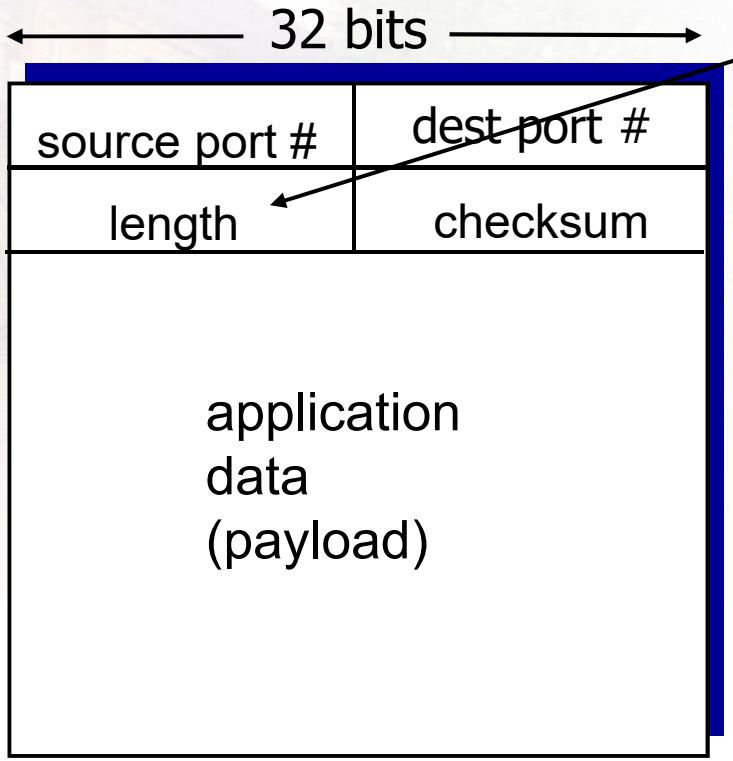
UDP: Transport Layer Actions



UDP receiver actions:

- receives segment from IP
- checks UDP checksum header value
- extracts application-layer message
- demultiplexes message up to application via socket

UDP: segment header



UDP segment format

length, in bytes of UDP segment, including header

why is there a UDP?

- **no connection establishment (which can add delay)**
- **simple: no connection state at sender, receiver**
- **small header size**
- **no congestion control: UDP can blast away as fast as desired**

UDP checksum

Goal: detect "errors" (e.g., flipped bits) in transmitted segment

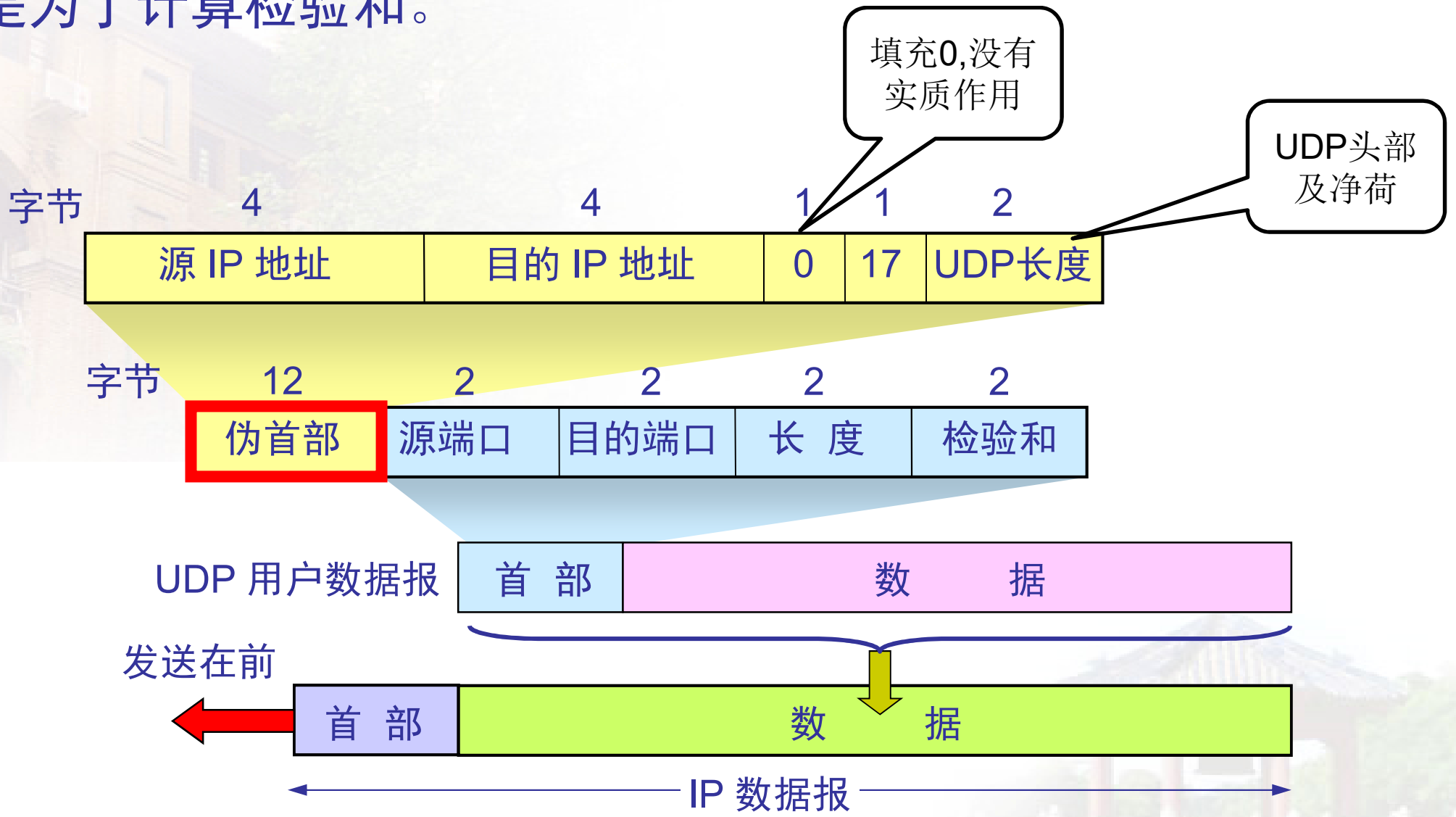
sender:

- treat segment contents, including header fields, as sequence of 16-bit integers
- checksum: addition (one's complement sum) of segment contents
- sender puts checksum value into UDP checksum field

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - NO - error detected
  - YES - no error detected. *But maybe errors nonetheless?* More later ….

在计算检验和时，临时把“伪首部”和 UDP 用户数据报连接在一起。伪首部仅仅是为了计算检验和。



Internet checksum: example

example: add two 16-bit integers

	1	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
sum		1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum		0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

Summary: UDP

- "no frills" protocol:
  - segments may be lost, delivered out of order
  - best effort service: "send and hope for the best"
- UDP has its plusses:
  - no setup/handshaking needed (no RTT incurred)
  - can function when network service is compromised
  - helps with reliability (checksum)
- build additional functionality on top of UDP in application layer (e.g., HTTP/3)

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

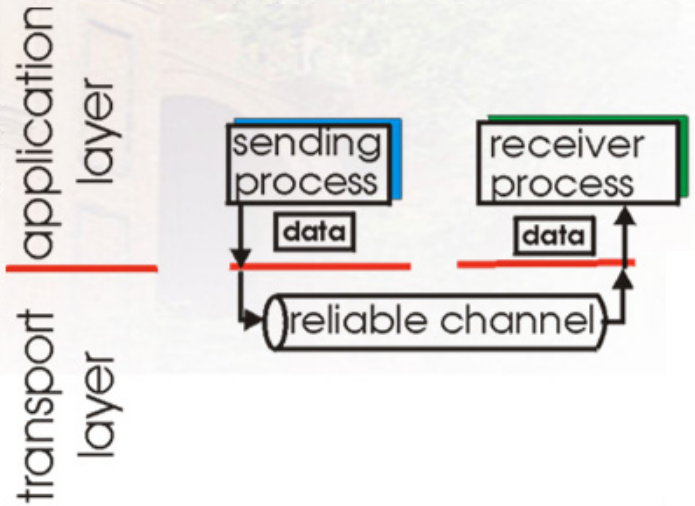
- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

Principles of reliable data transfer

- important in application, transport, link layers
  - top-10 list of important networking topics!

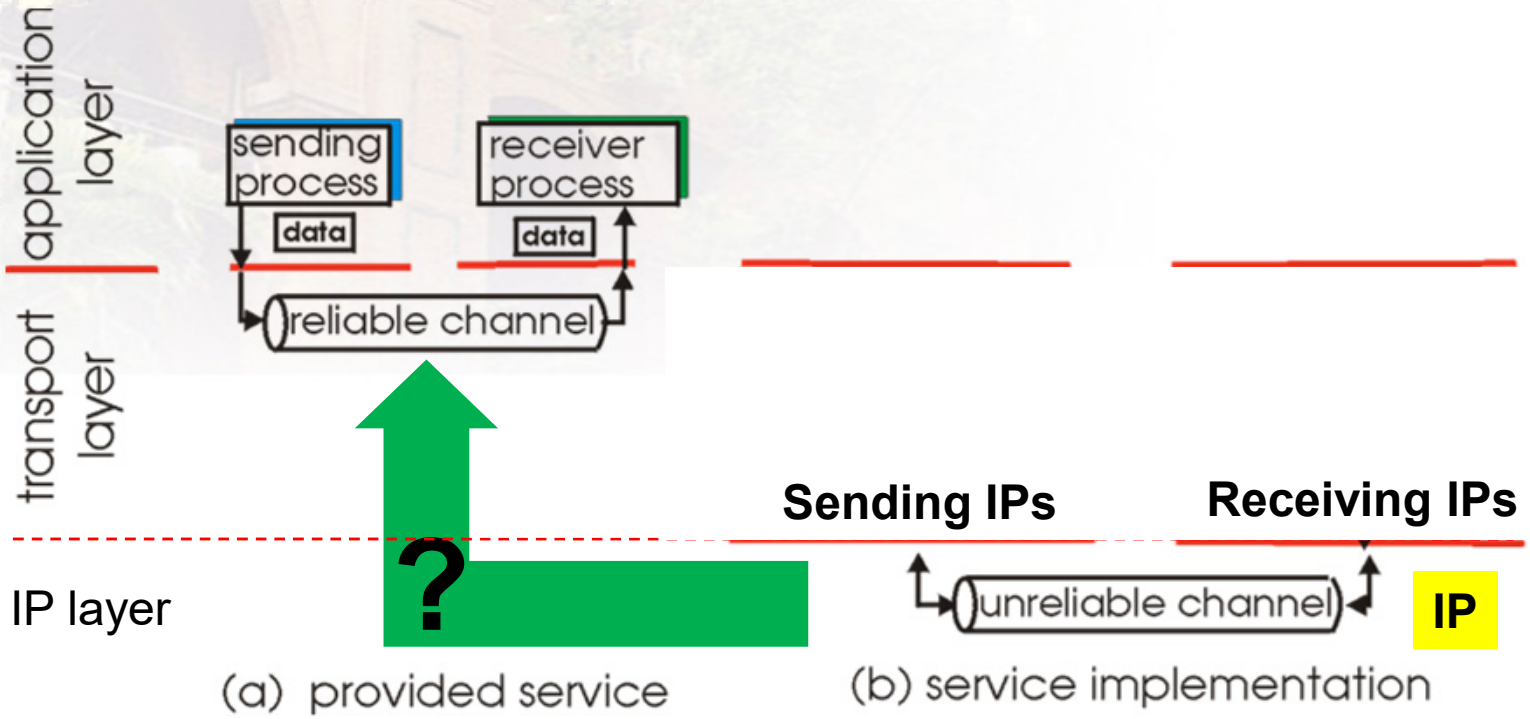


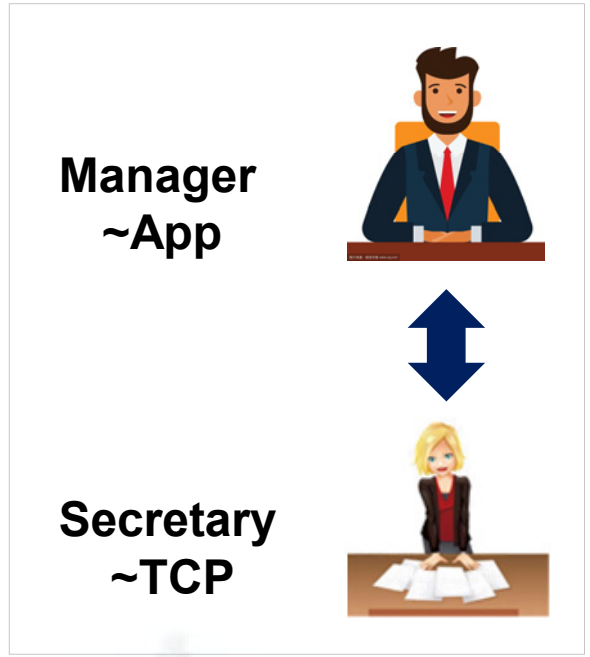
(a) provided service

- characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Principles of reliable data transfer

- important in application, transport, link layers
  - top-10 list of important networking topics!

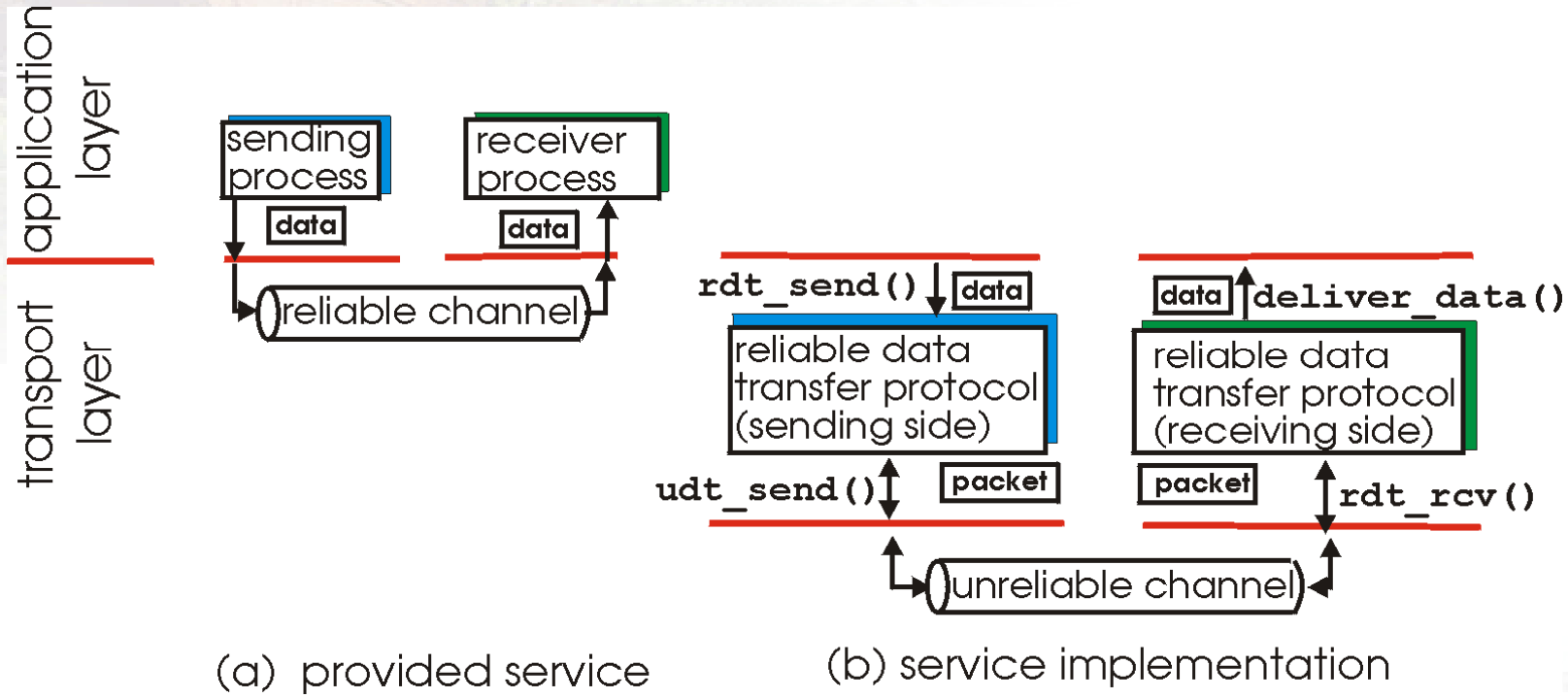




- characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Principles of reliable data transfer

- important in application, transport, link layers
  - top-10 list of important networking topics!

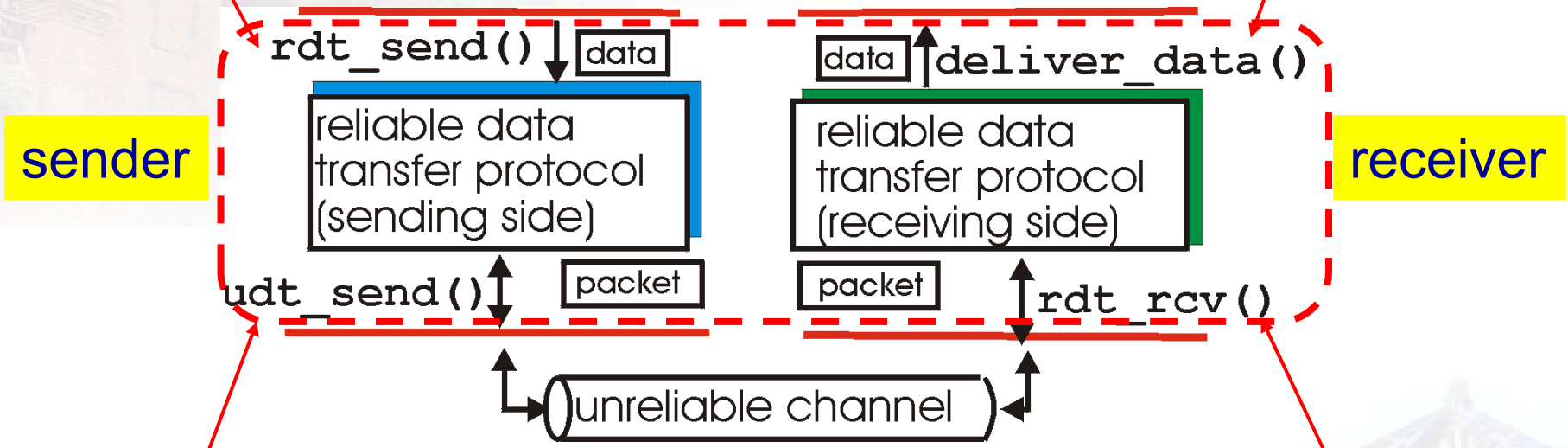


- characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Reliable data transfer: getting started

`rdt_send()`: called from above, (e.g., by app.). Passed data to deliver to receiver upper layer

`deliver_data()`: called by **rdt** to deliver data to upper



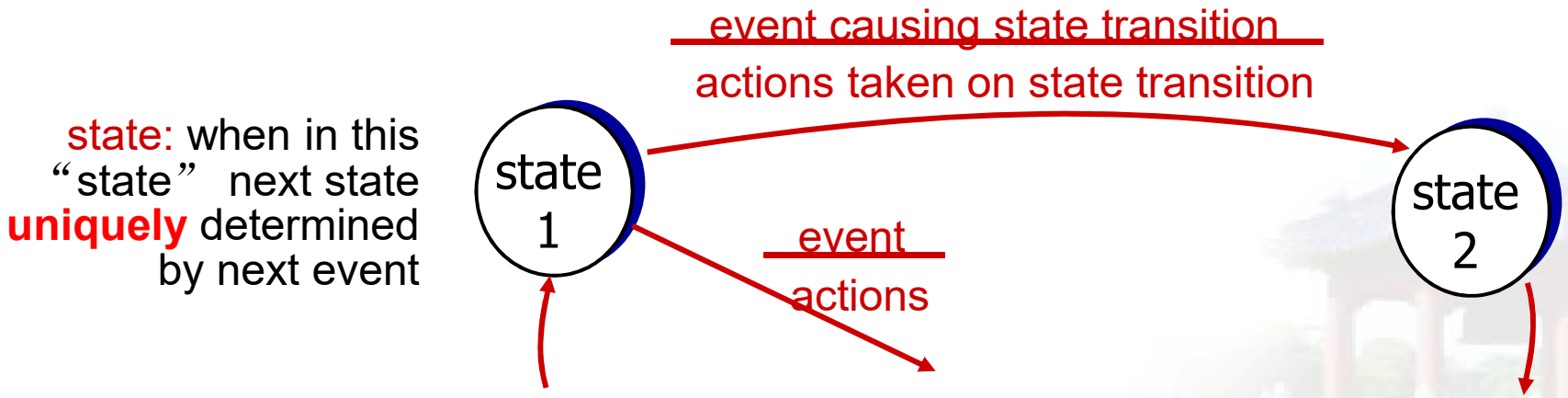
`udt_send()`: called by rdt, to transfer packet over unreliable channel to receiver

`rdt_rcv()`: called when packet arrives on rcv-side of channel

Reliable data transfer: getting started

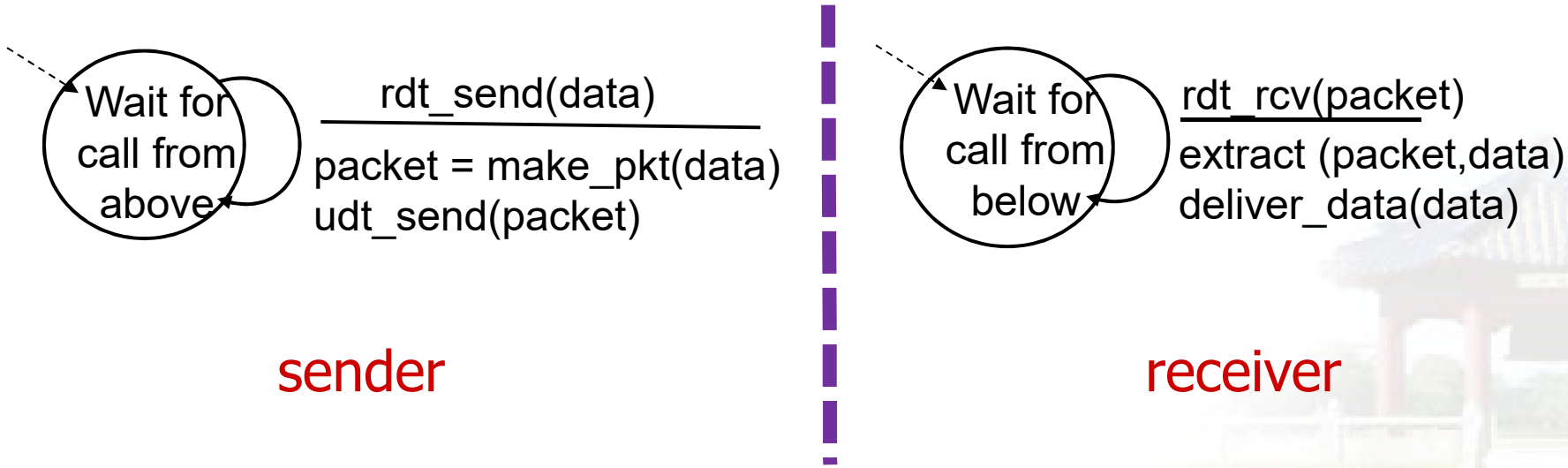
We'll:

- incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)
- consider only unidirectional data transfer
  - but control info will flow on both directions!
- use finite state machines (FSM) to specify sender, receiver



rdt1.0: reliable transfer over a reliable channel

- underlying channel (i.e., IP) perfectly reliable
  - no bit errors
  - no loss of packets
- separate FSMs for sender, receiver:
  - sender sends data into underlying channel
  - receiver reads data from underlying channel



rdt2.0: channel with bit errors

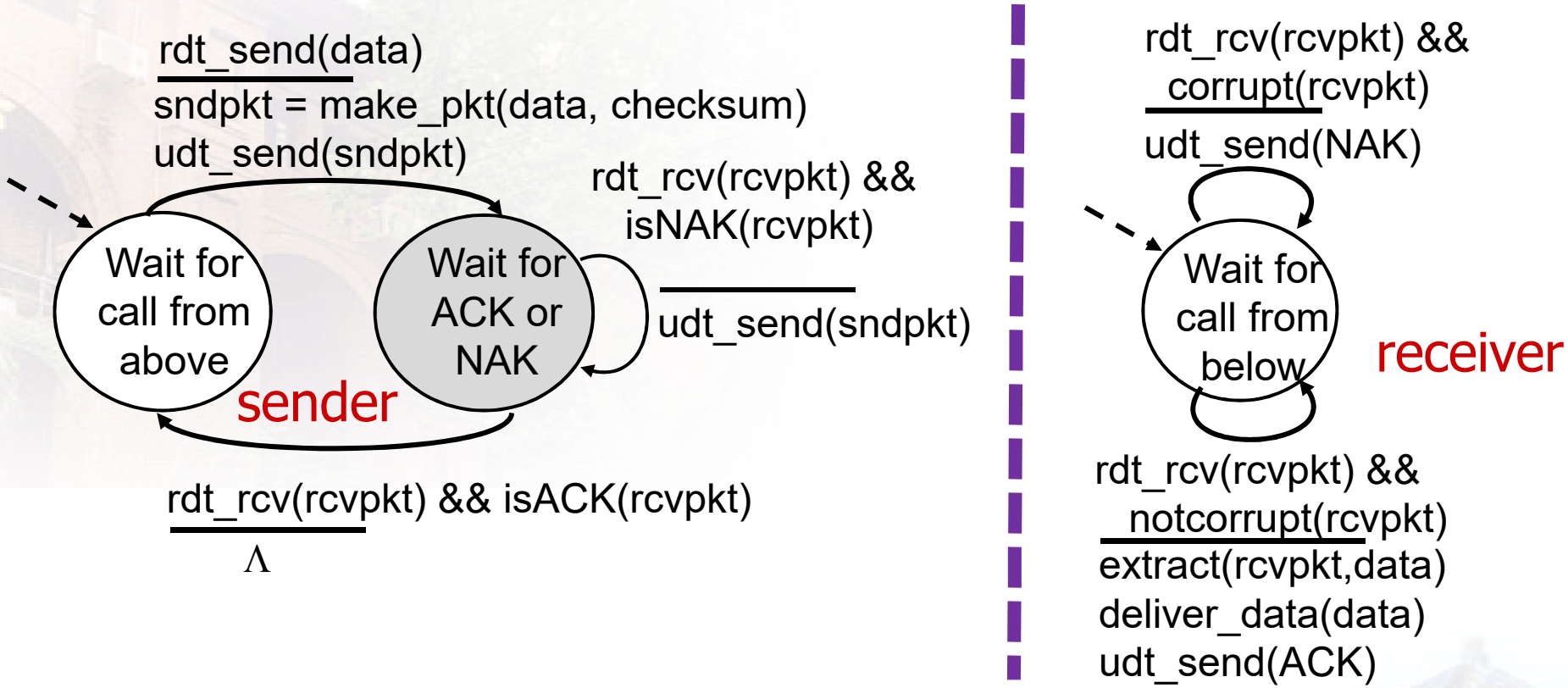
- underlying channel may flip bits in packet
  - checksum to detect bit errors
- *the* question: how to recover from errors:

How do humans recover from "errors" during conversation?

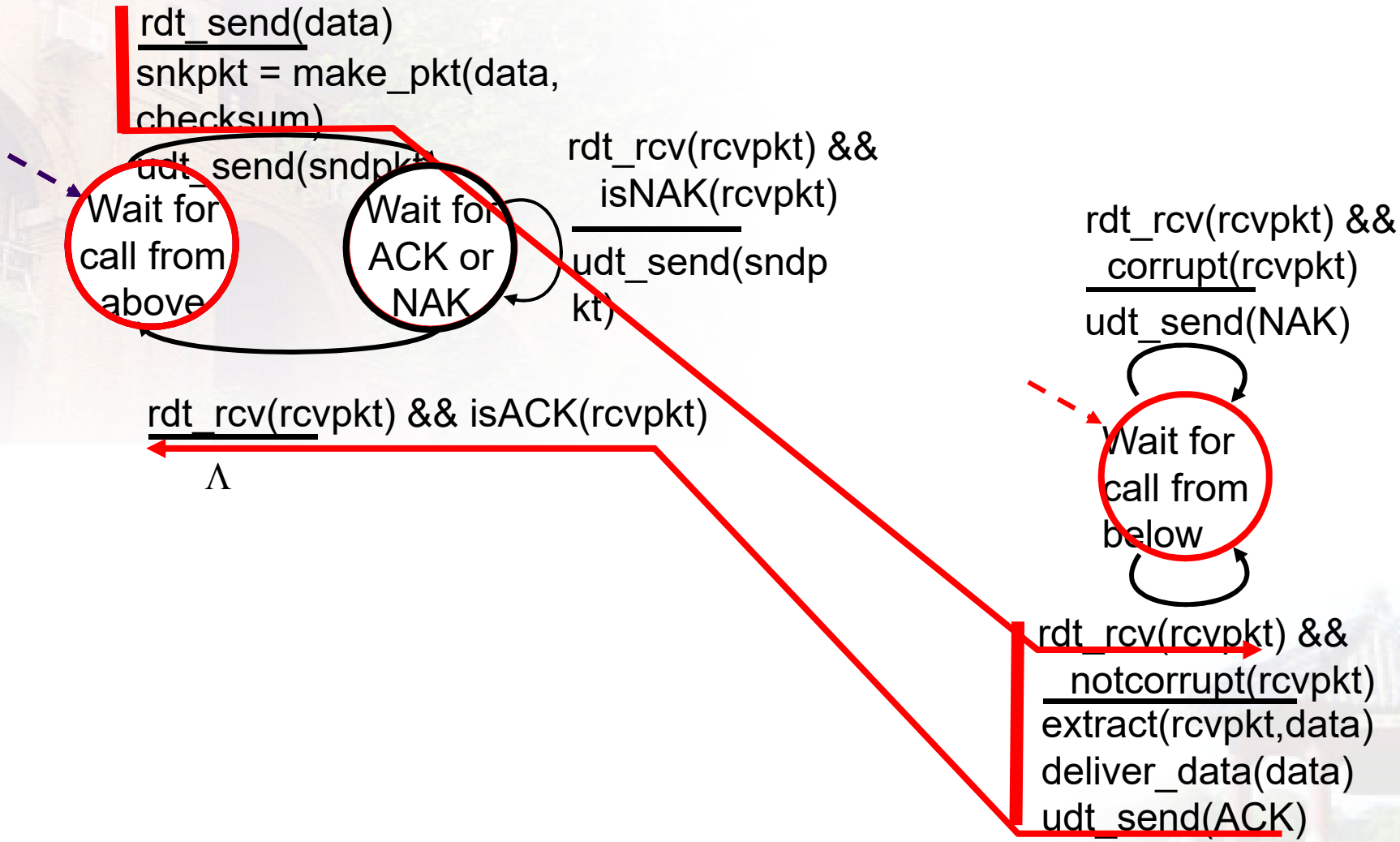
rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
  - checksum to detect bit errors
- *the* question: how to recover from errors:
  - *acknowledgements (ACKs):* receiver explicitly tells sender that pkt received OK
  - *negative acknowledgements (NAKs):* receiver explicitly tells sender that pkt had errors
  - sender retransmits pkt on receipt of NAK
- new mechanisms in `rdt2.0` (beyond `rdt1.0`):
  - error detection
  - feedback: control msgs (ACK,NAK) from receiver to sender

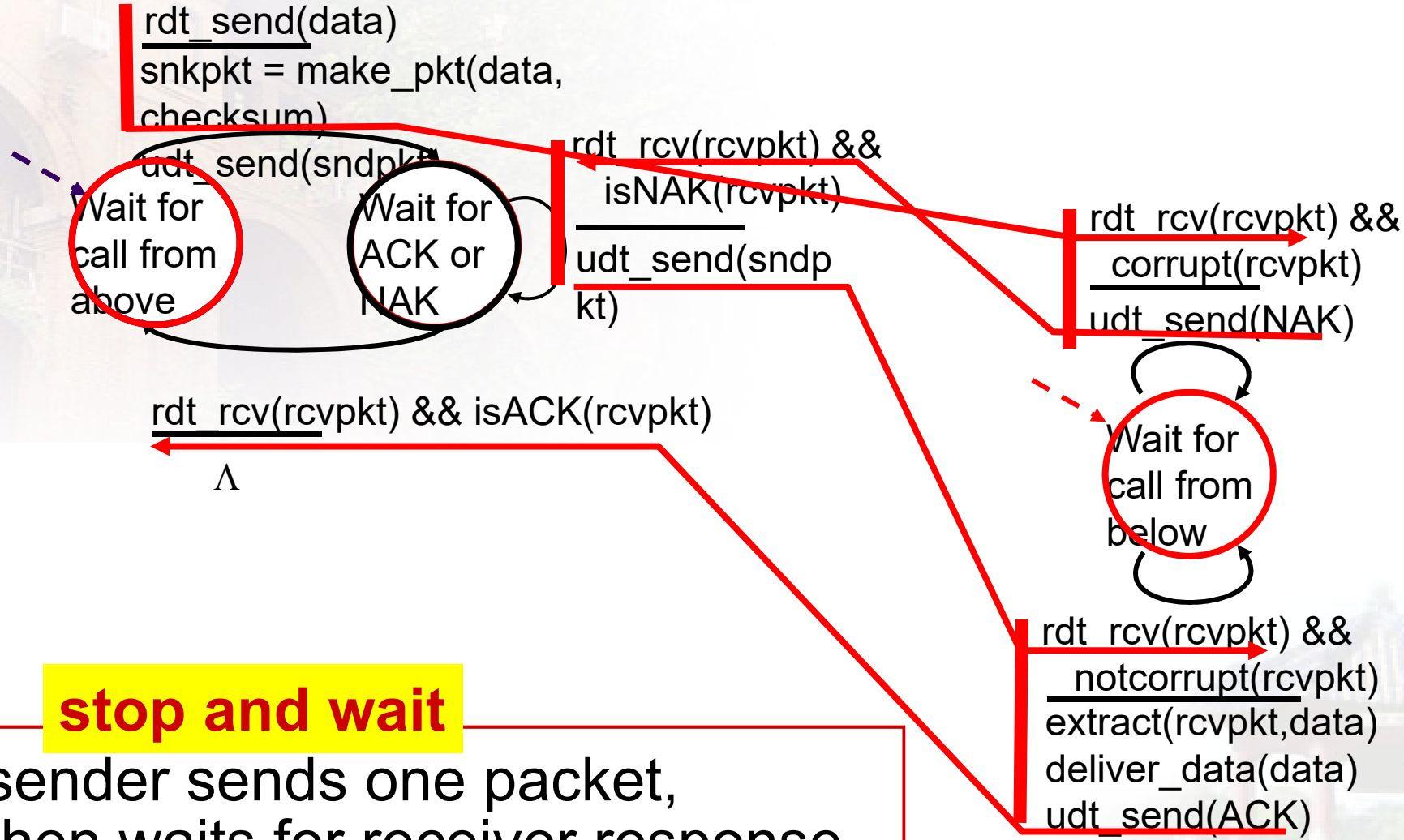
rdt2.0: FSM specification



rdt2.0: operation with no errors



rdt2.0: error scenario



stop and wait

sender sends one packet,
then waits for receiver response

rdt2.0 has a fatal flaw!

what happens if ACK/NAK corrupted?

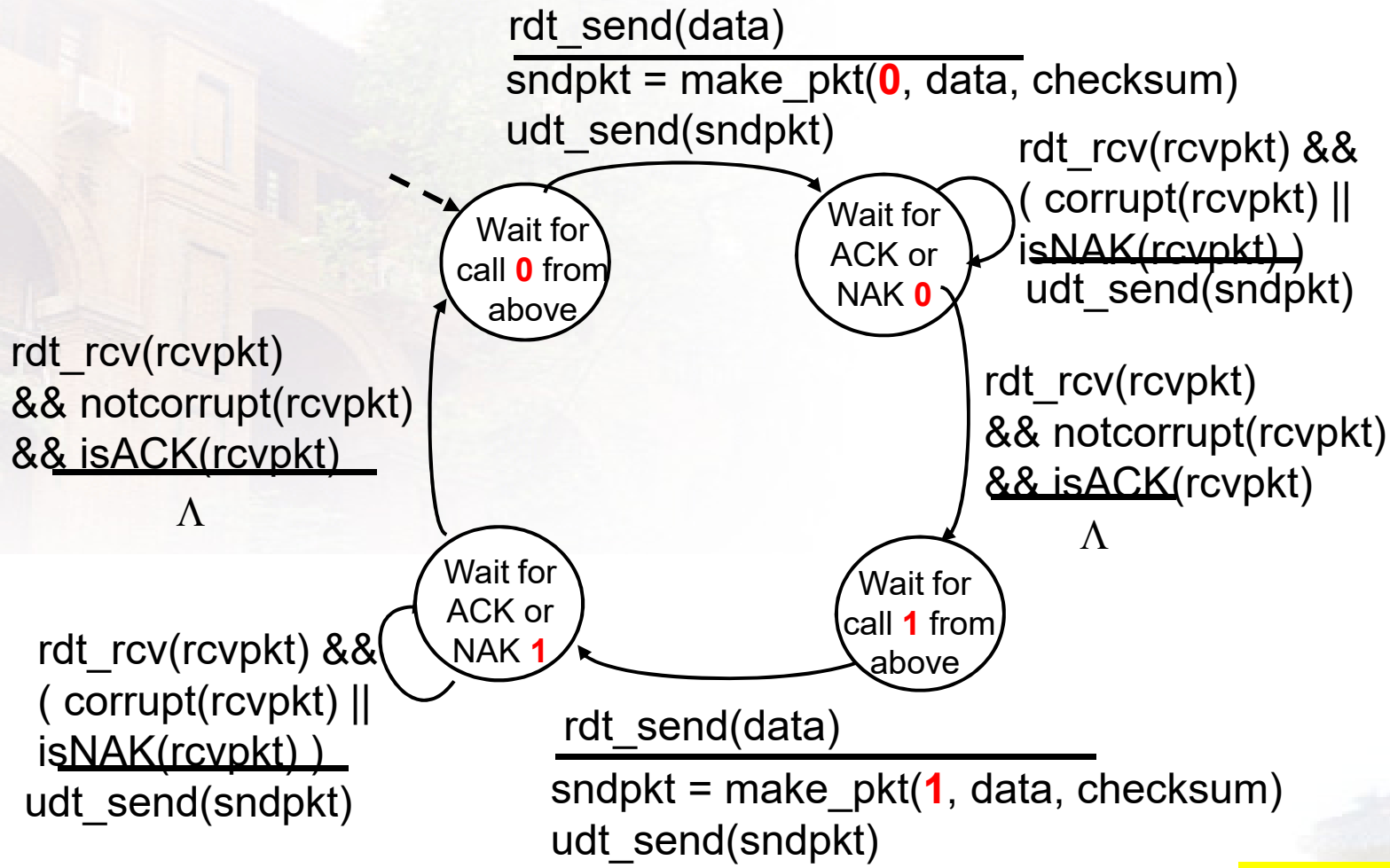
- sender doesn't know what happened at receiver!
- Can't just retransmit after timeout: possible duplicate

KEY QUESTION: duplicates

handling duplicates:

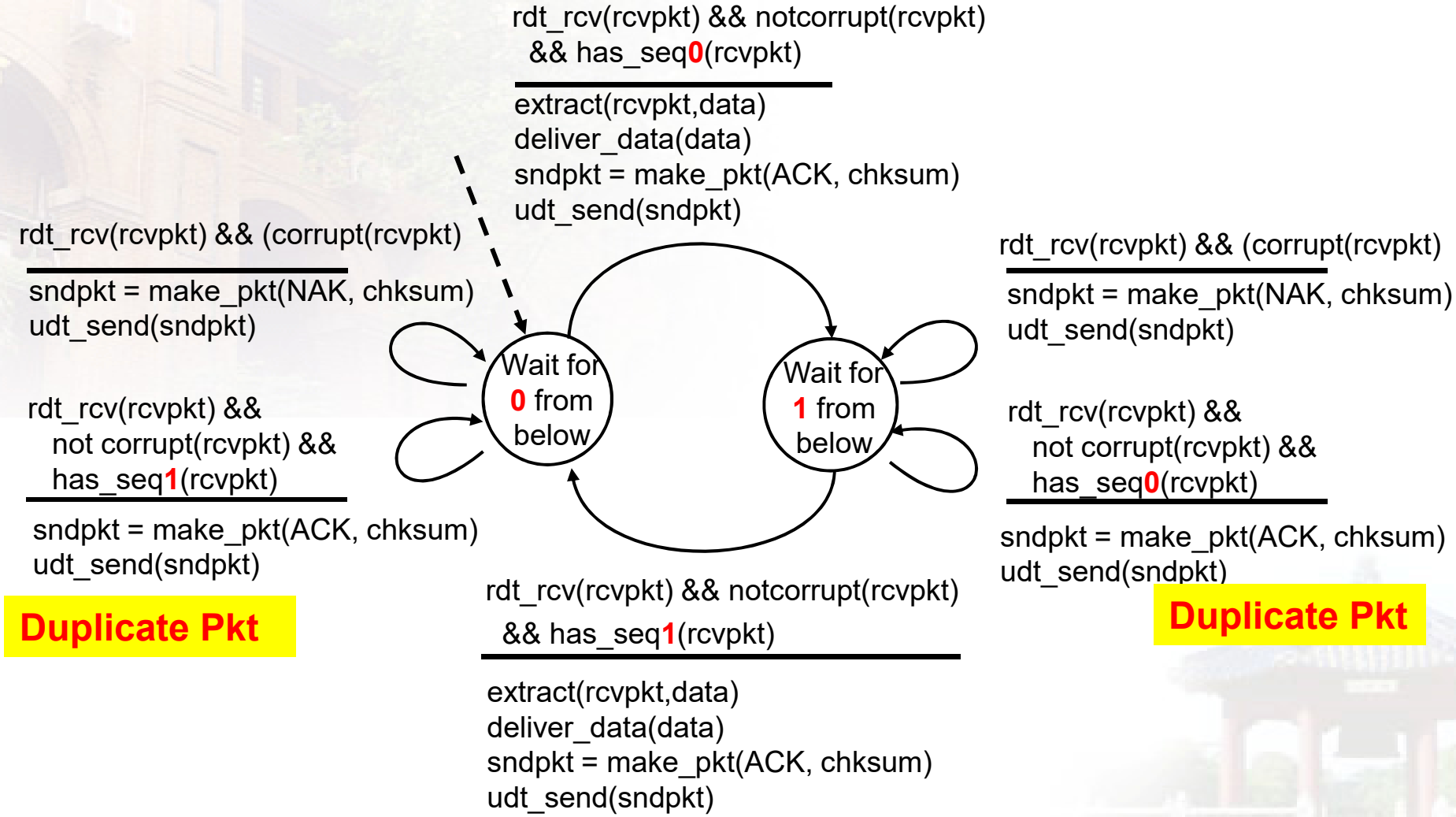
- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

rdt2.1: sender, handles garbled ACK/NAKs



No sequence number in ACK

rdt2.1: receiver, handles garbled ACK/NAKs



rdt2.1: discussion

sender:

- seq # added to pkt
- two seq. #'s (0,1) will suffice. Why?
- must check if received ACK/NAK corrupted
- State # is twice
  - state must "remember" whether "expected" pkt should have seq # of 0 or 1

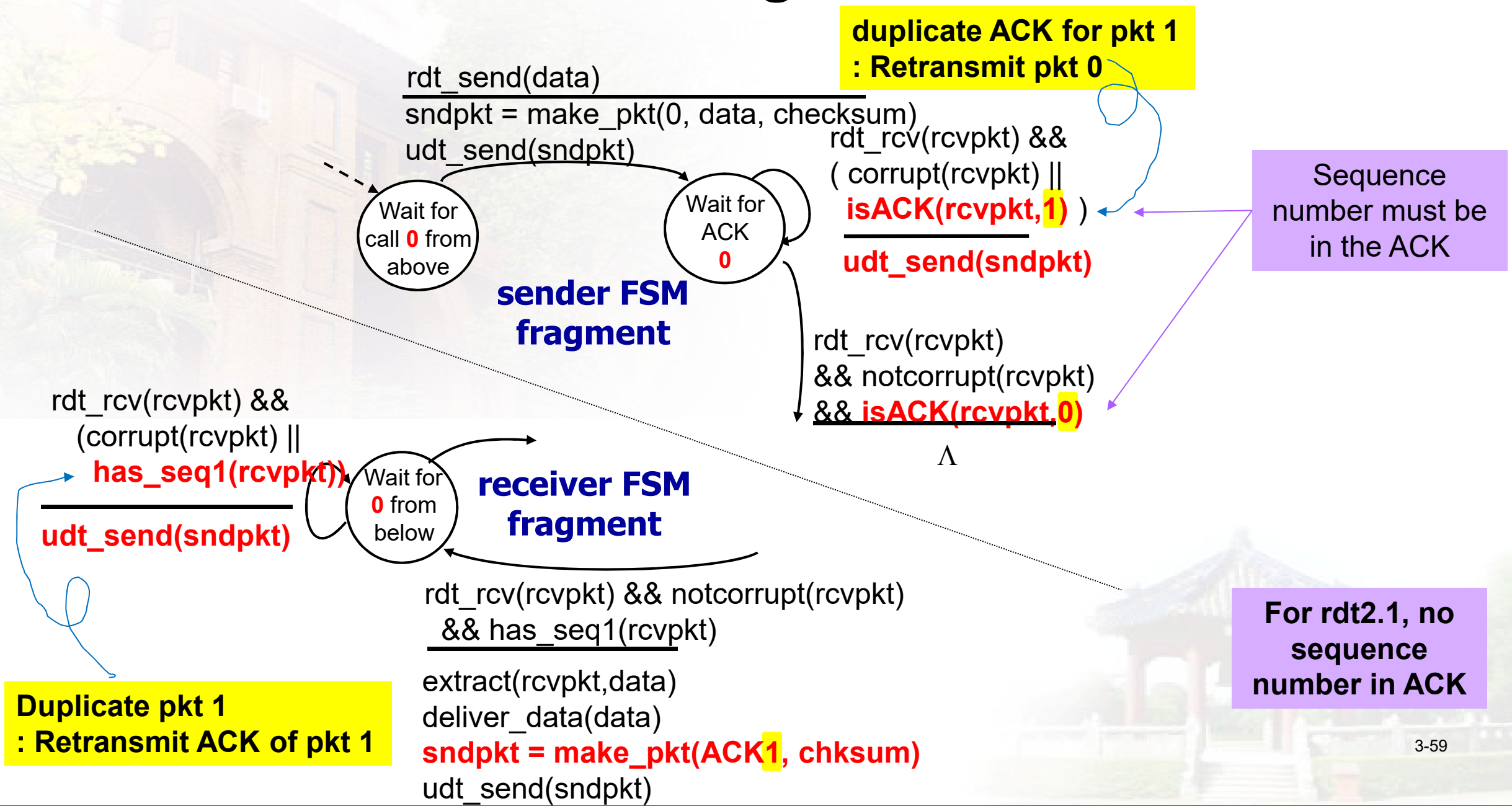
receiver:

- must check if received packet is duplicate
  - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

rdt2.2: a NAK-free protocol

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

rdt2.2: sender, receiver fragments



rdt3.0: channels with *errors* and *loss*

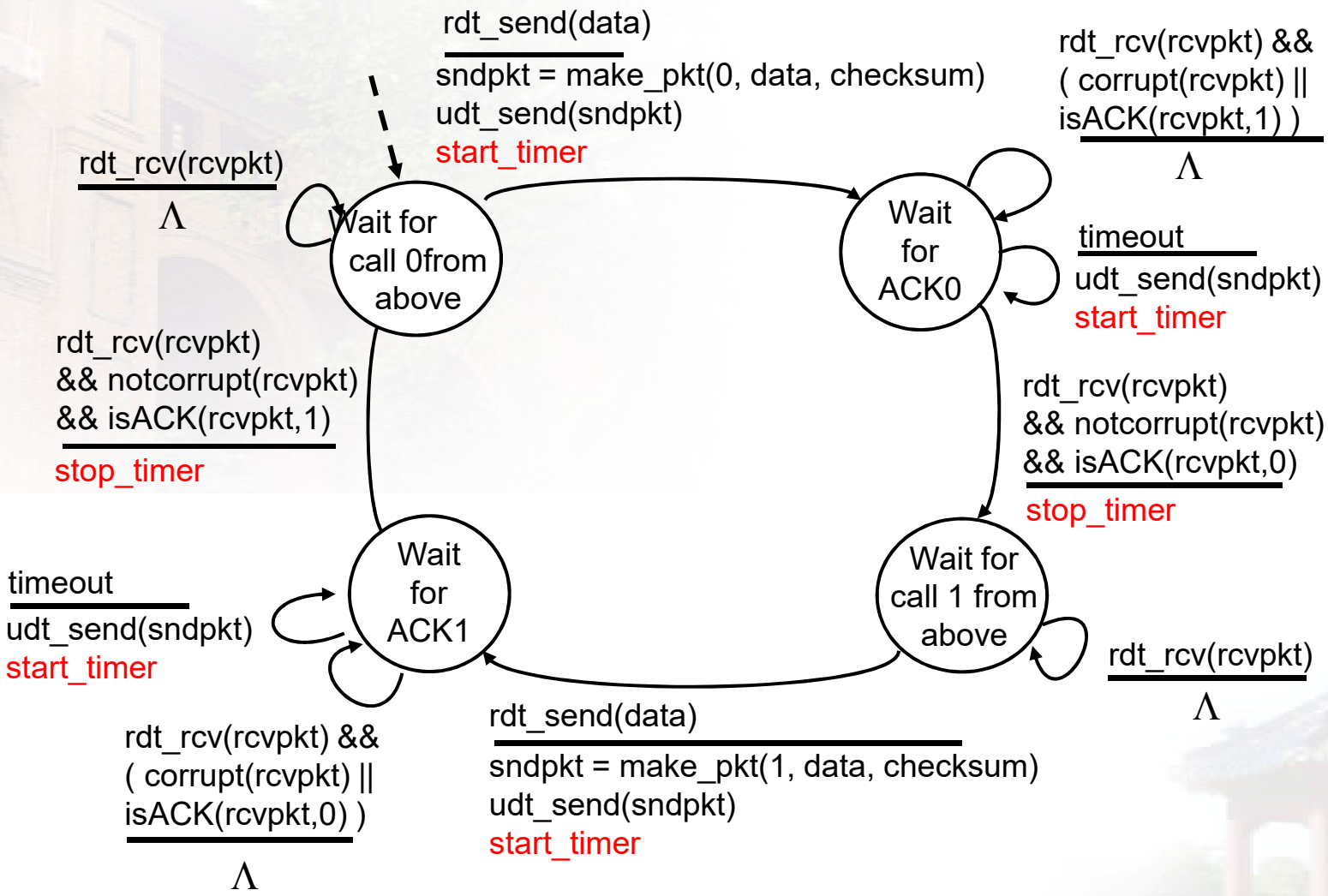
new assumption: underlying channel can also **lose** packets (data, ACKs)

- checksum, seq. #, ACKs, retransmissions will be of help … but **not enough**

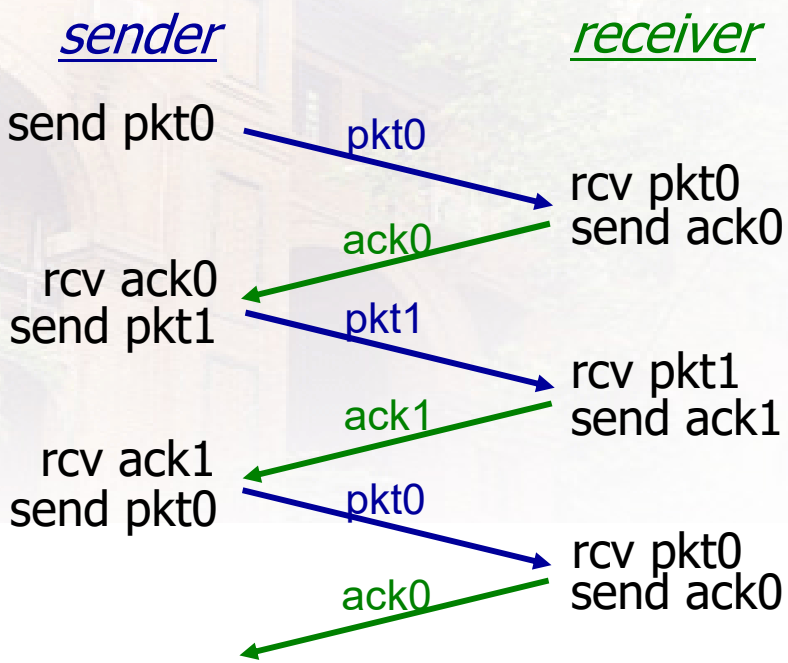
approach: sender waits "reasonable" amount of **time** for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
  - retransmission will be duplicate, but seq. # already handles this
  - receiver must specify seq # of pkt being ACKed
- requires countdown timer

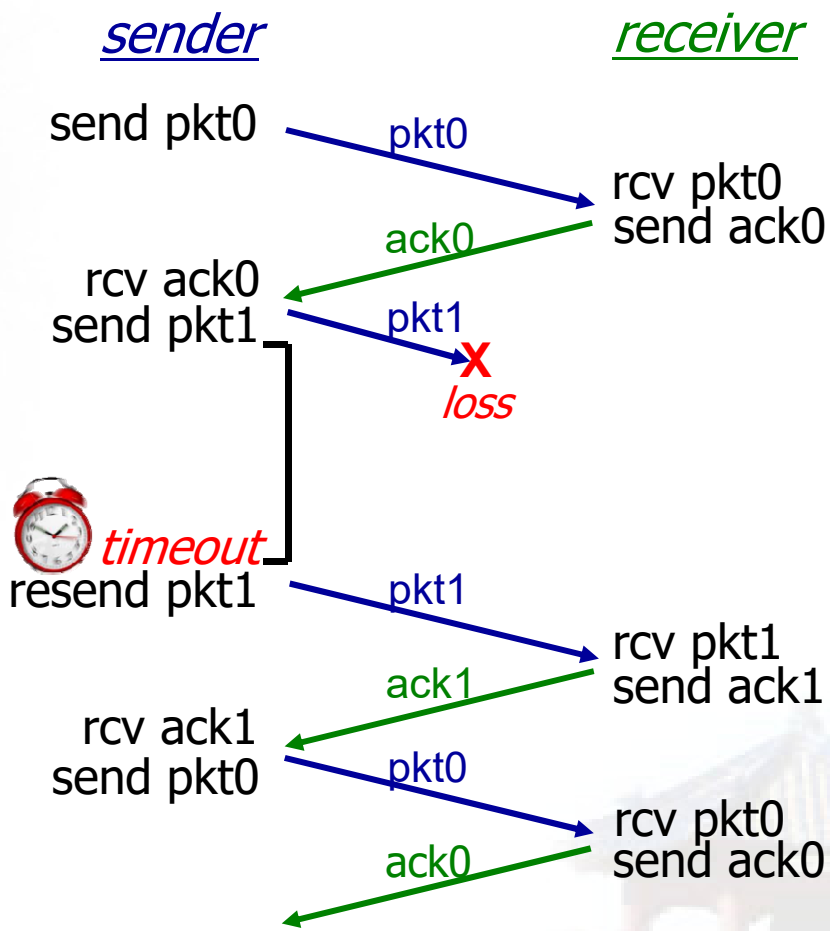
rdt3.0 sender



rdt3.0 in action

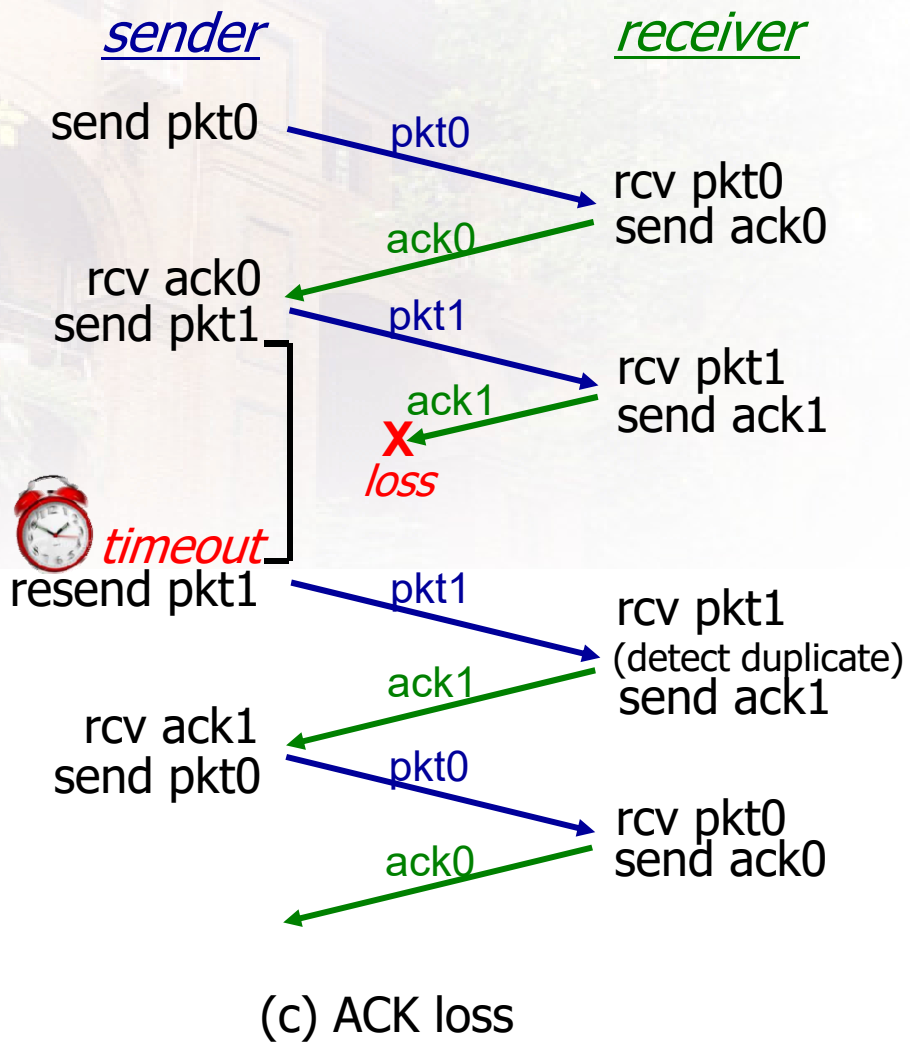


(a) no loss

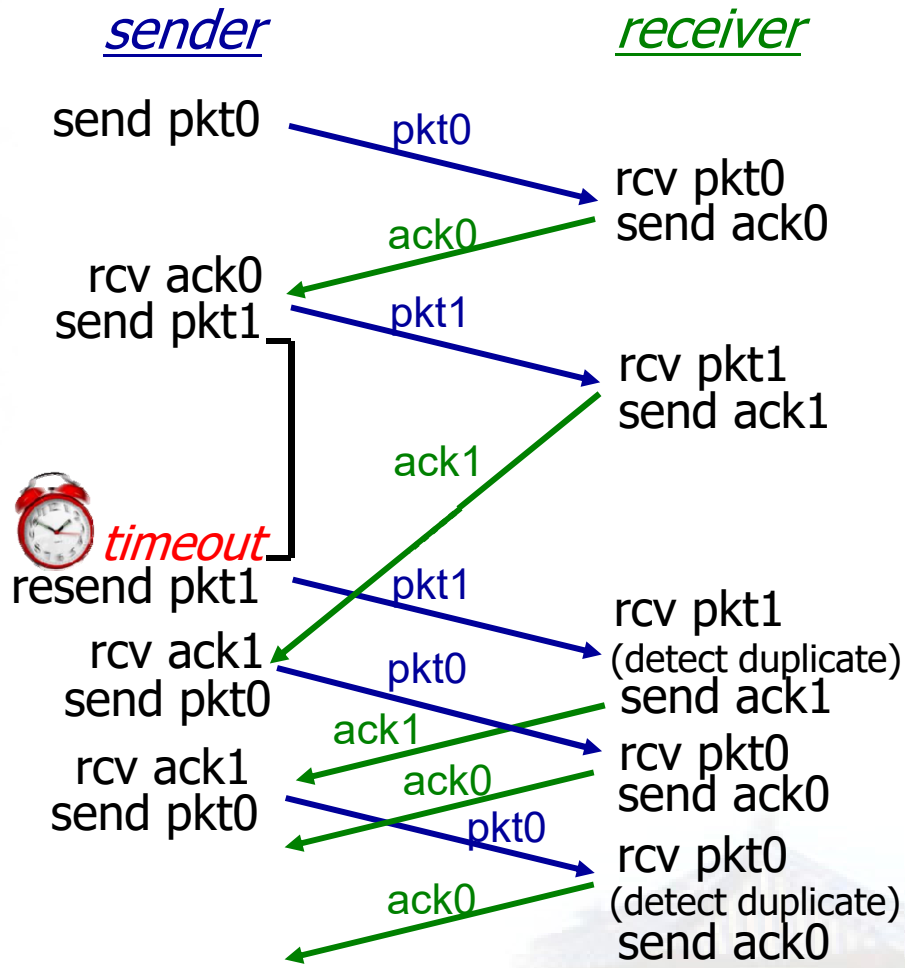


(b) packet loss

rdt3.0 in action



(c) ACK loss



(d) premature timeout/ delayed ACK

Would you please design other possible error scenarios?

Performance of rdt3.0

- **rdt3.0 is correct, but performance stinks**
- **e.g.: 1 Gbps link, 15 ms prop. delay, 8000 bit packet:**

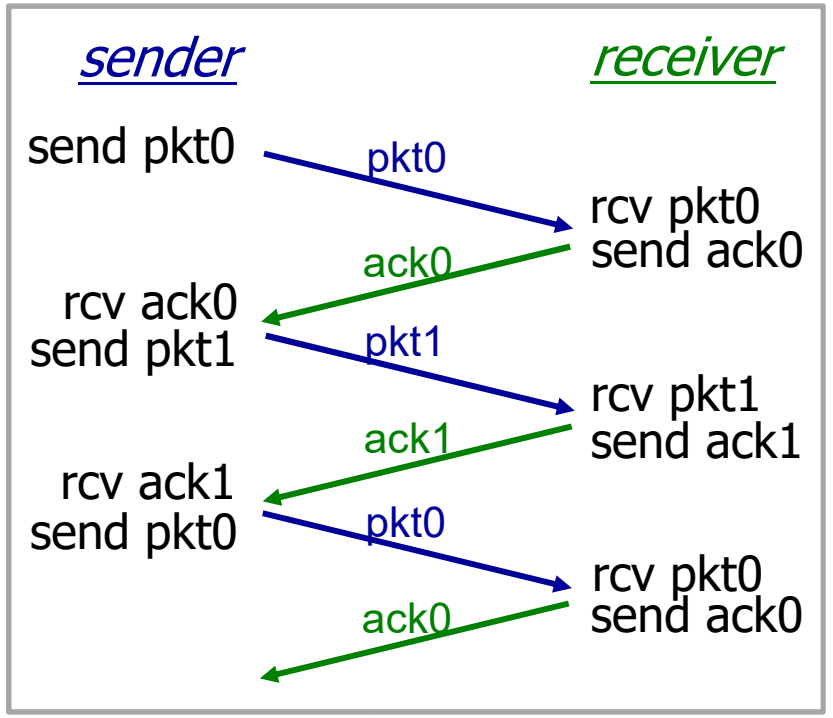
$$D_{trans} = \frac{L}{R} = \frac{8000\ bits}{10^9\ bits/sec} = 8\ microsecs$$

  - U $_{sender}$: *utilization* – fraction of time sender busy sending

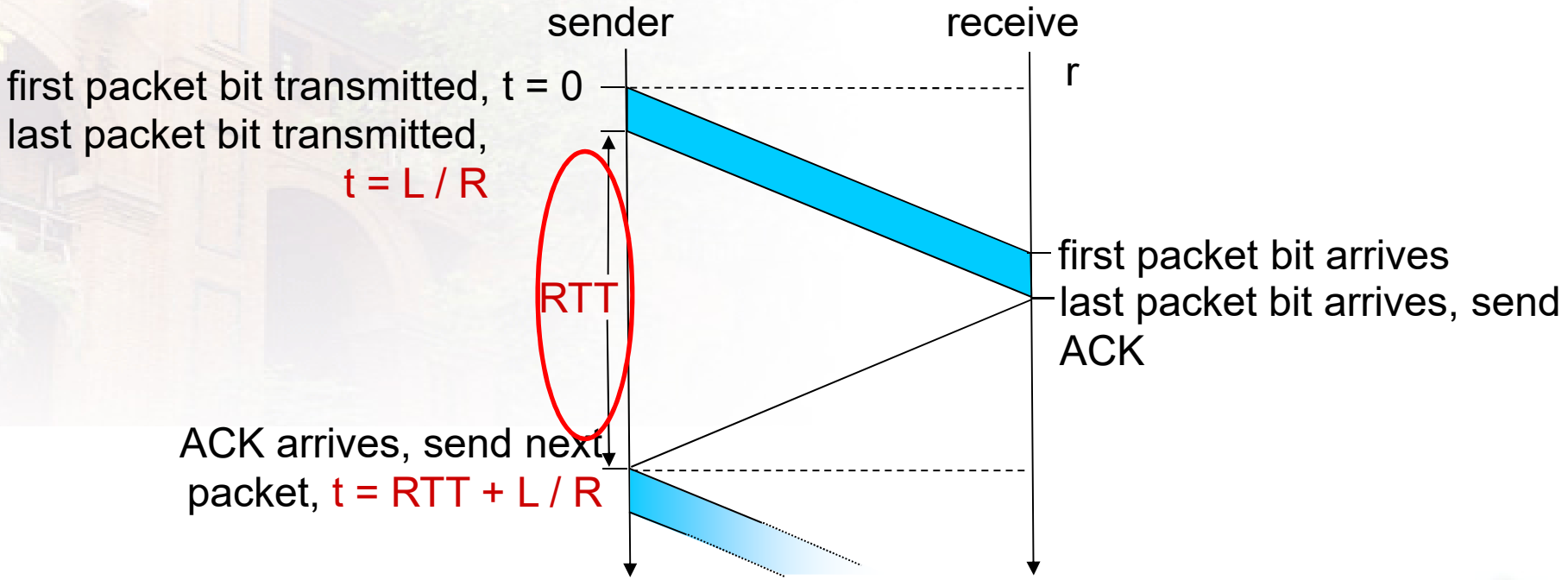
$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

  - if **RTT**=30 msec, 1KB pkt every 30 msec: 33kB/sec thruput over 1 Gbps link

❖ network protocol limits use of physical resources!



rdt3.0: stop-and-wait operation

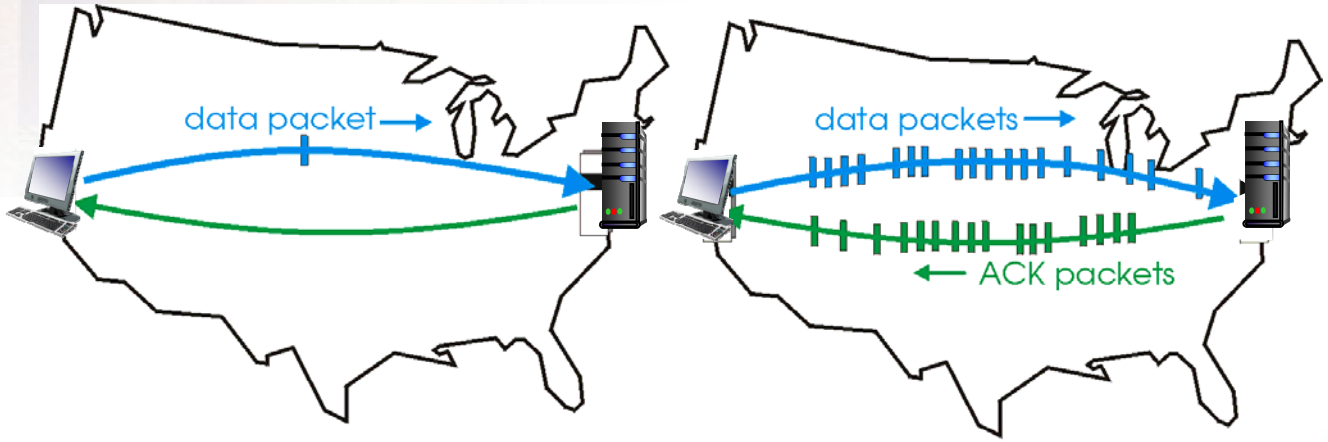


$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

Pipelined protocols

pipelining: sender allows multiple, "in-flight", yet-to-be-acknowledged pkts

- range of sequence numbers must be increased
- buffering at sender and/or receiver



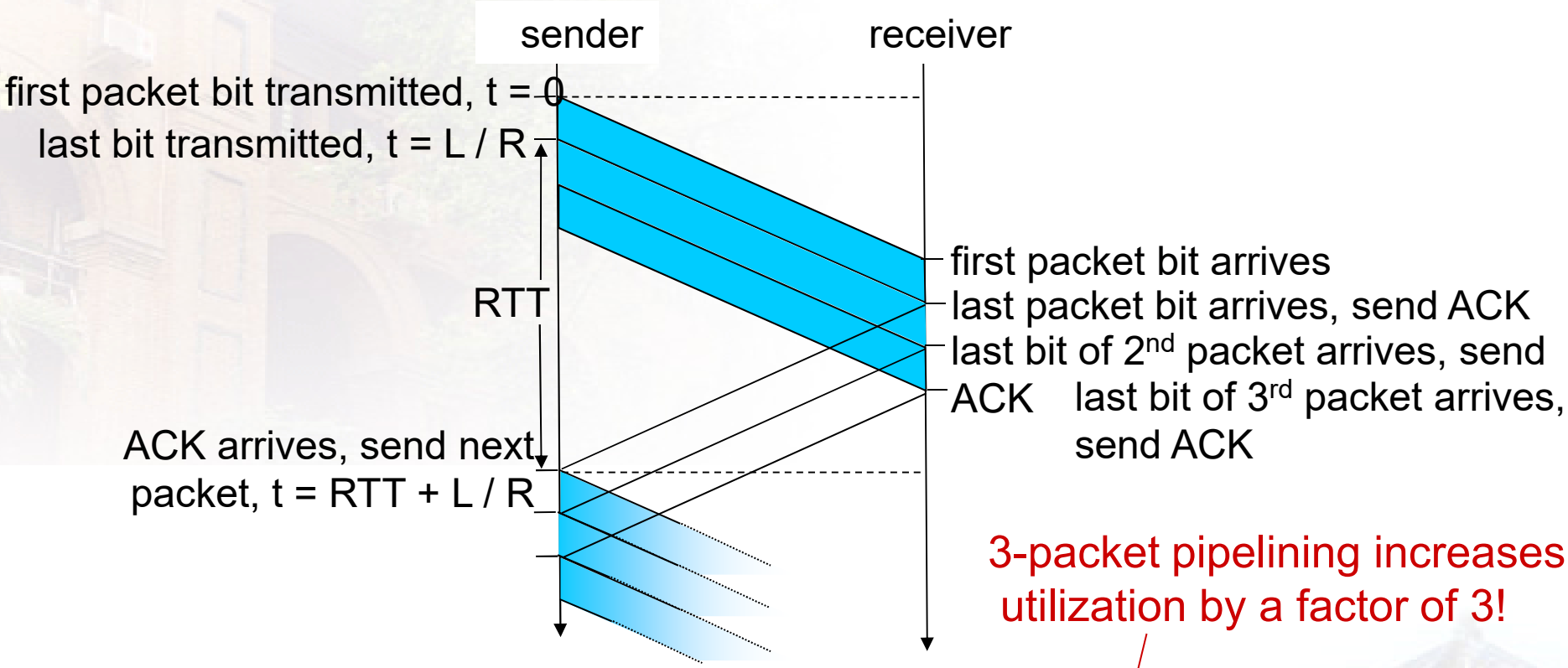
(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- two generic forms of pipelined protocols:

  go-Back-N, selective repeat

Pipelining: increased utilization



3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

Pipelined protocols: overview

Go-back-N:

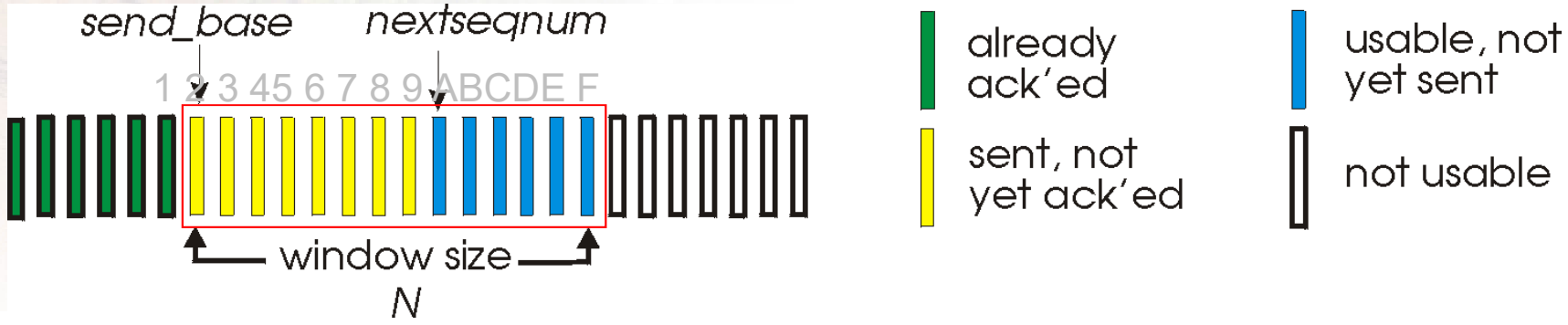
- sender can have up to N unacked packets in pipeline
- receiver only sends *cumulative ack*
  - Doesn't ack packet if there's a gap
- sender has timer for oldest unacked packet
  - when timer expires, retransmit *all* unacked packets

Selective Repeat:

- sender can have up to N unack'ed packets in pipeline
- rcvr sends *individual ack* for each packet
- sender maintains timer for each unacked packet
  - when timer expires, retransmit only that unacked packet

Go-Back-N: sender

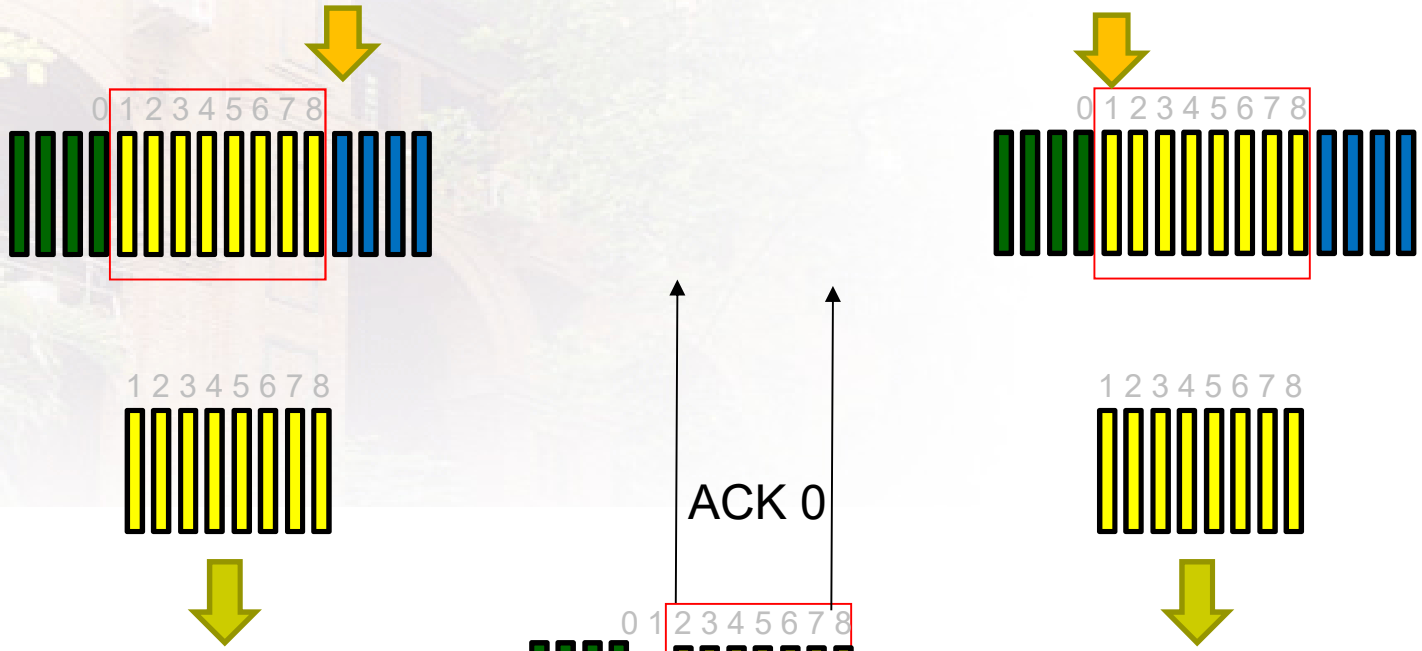
- **k-bit seq # in pkt header**
- **"window" of up to N, consecutive unack'ed pkts allowed**



- ACK(n): ACKs all pkts up to, including seq # n - *"cumulative ACK"*
  - may receive duplicate ACKs (see receiver)
- timer for oldest in-flight pkt
- *timeout(n):* retransmit packet n and all higher seq # pkts in window

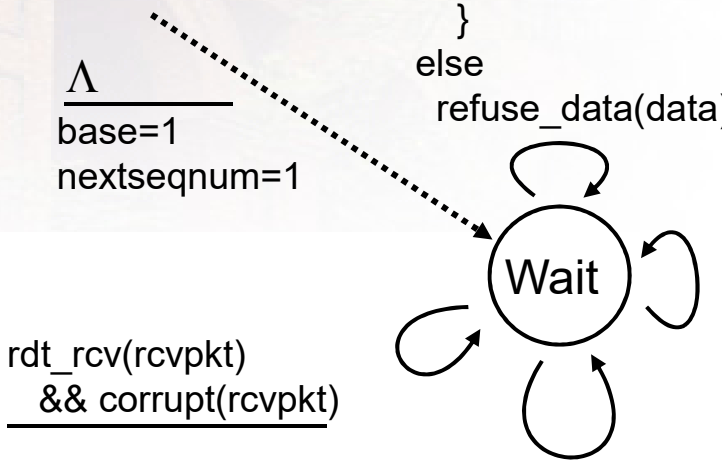
Go-Back-N: sender

Sender:
N =8



receiver:

GBN: sender extended FSM

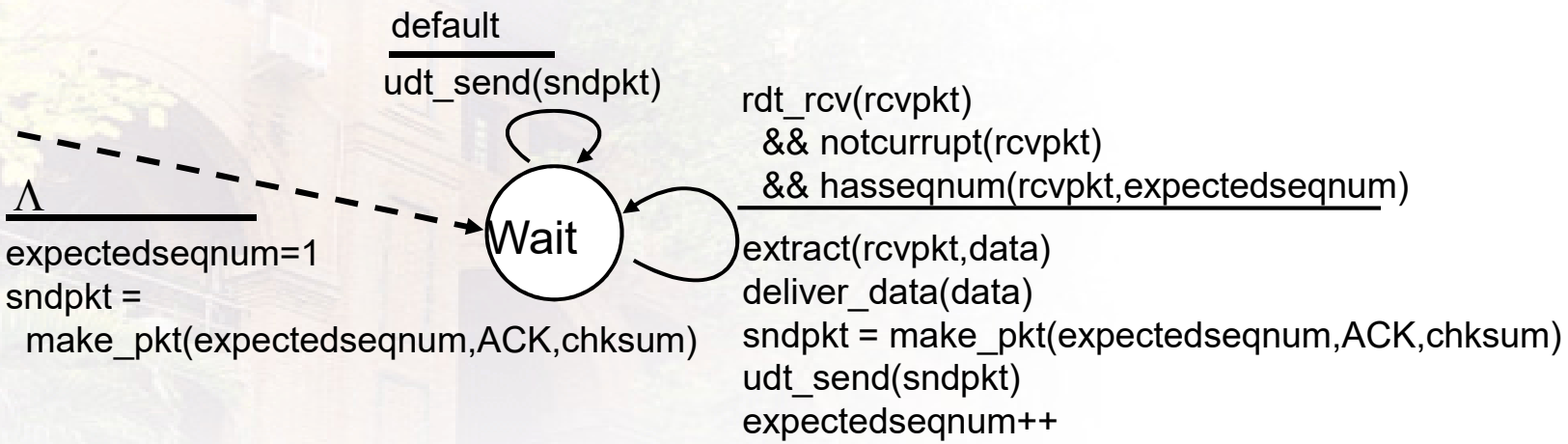



```
rdt_send(data)
if (nextseqnum < base+N) {
   sndpkt[nextseqnum] = make_pkt(nextseqnum,data,chksum)
   udt_send(sndpkt[nextseqnum])
   if (base == nextseqnum)  //窗口中第一个分组启动计时器
     start_timer
   nextseqnum++
   }
else
  refuse_data(data)
```

```
timeout
start_timer
udt_send(sndpkt[base])
udt_send(sndpkt[base+1])
…
udt_send(sndpkt[nextseqnum-1])
```

```
rdt_rcv(rcvpkt) &&
  notcorrupt(rcvpkt)
base = getacknum(rcvpkt)+1
If (base == nextseqnum)
   stop_timer     //窗口底部位于还没发出的数据上,无需计时
  else
   start_timer    //为窗口中下一个已发分组启动计时器
```

GBN: receiver extended FSM



ACK-only: always send ACK for correctly-received pkt with highest *in-order* seq #

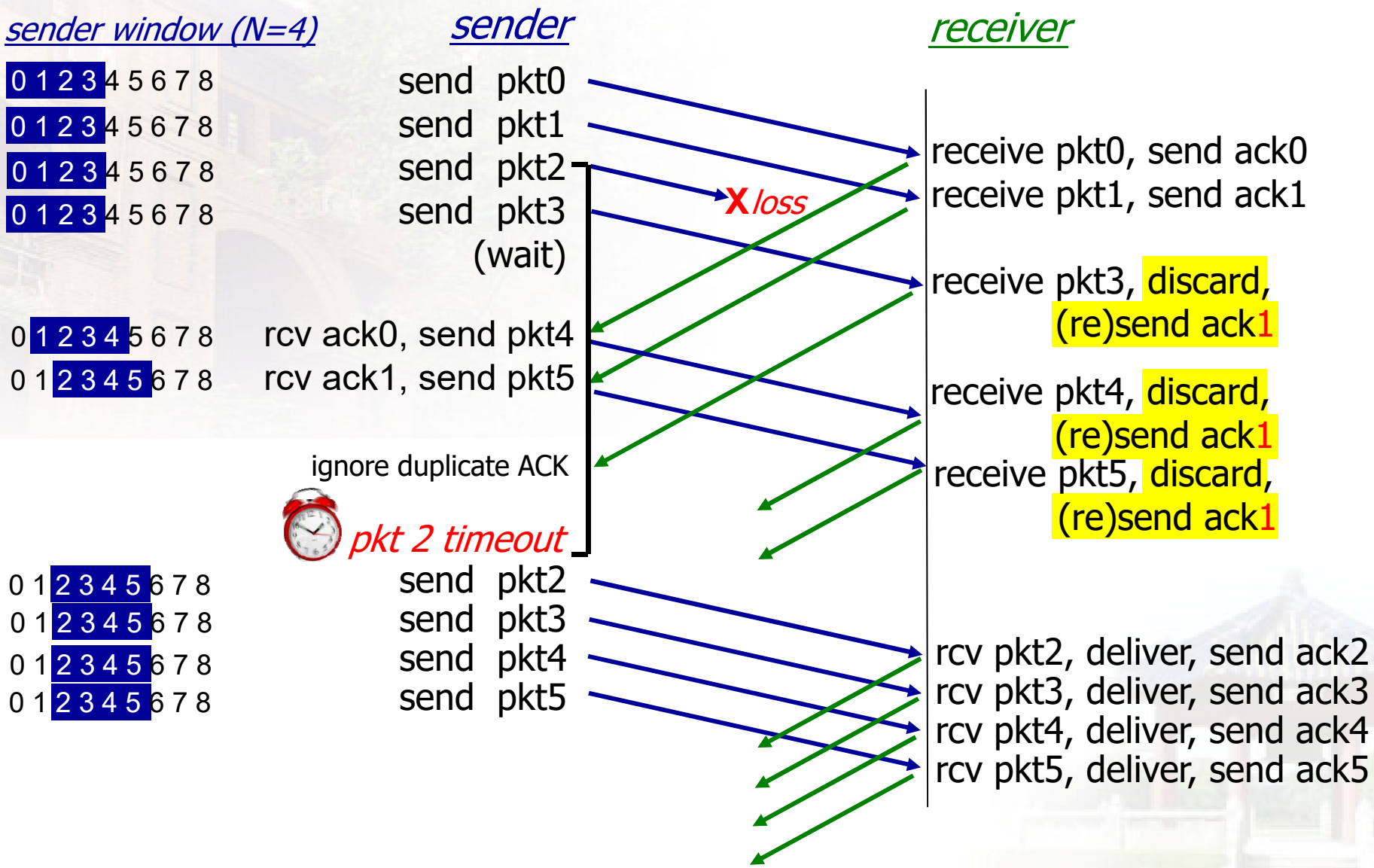
- may generate duplicate ACKs
- need only remember `expectedseqnum`

- **out-of-order pkt:**
  - discard (don't buffer): ***no receiver buffering!***
  - re-ACK pkt with highest in-order seq #

1 2 3 4 5 are received
ACK 5

1 2 4 5 are received
ACK 2; 4-5 are discarded

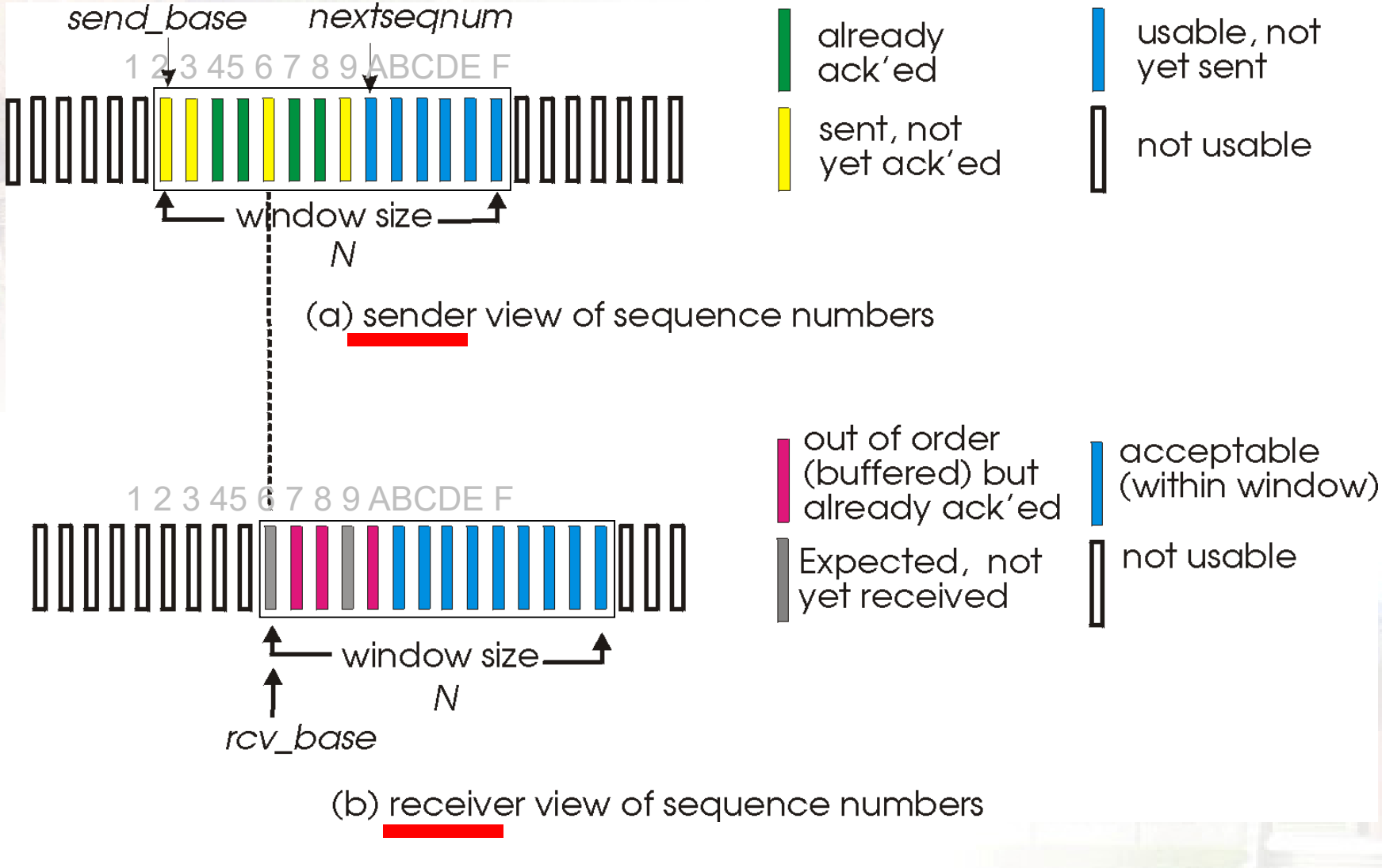
GBN in action



Selective repeat

- receiver *individually* acknowledges all correctly received pkts
  - buffers pkts, as needed, for eventual in-order delivery to upper layer
- sender only resends pkts for which ACK not received
  - sender timer for each unACKed pkt
- sender window
  - *N* consecutive seq #s
  - limits seq #s of sent, unACKed pkts

Selective repeat: sender, receiver windows



Selective repeat

sender

data from above:

- **if next available seq # in window, send pkt**

timeout(n):

- **resend pkt n, restart timer**

ACK(n) in [sendbase,sendbase+N]:

- **mark pkt n as received**
- **if n smallest unACKed pkt, advance window base to next unACKed seq #**

receiver

pkt n in [rcvbase, rcvbase+N-1]

- send ACK(n)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order pkts), advance window to next not-yet-received pkt

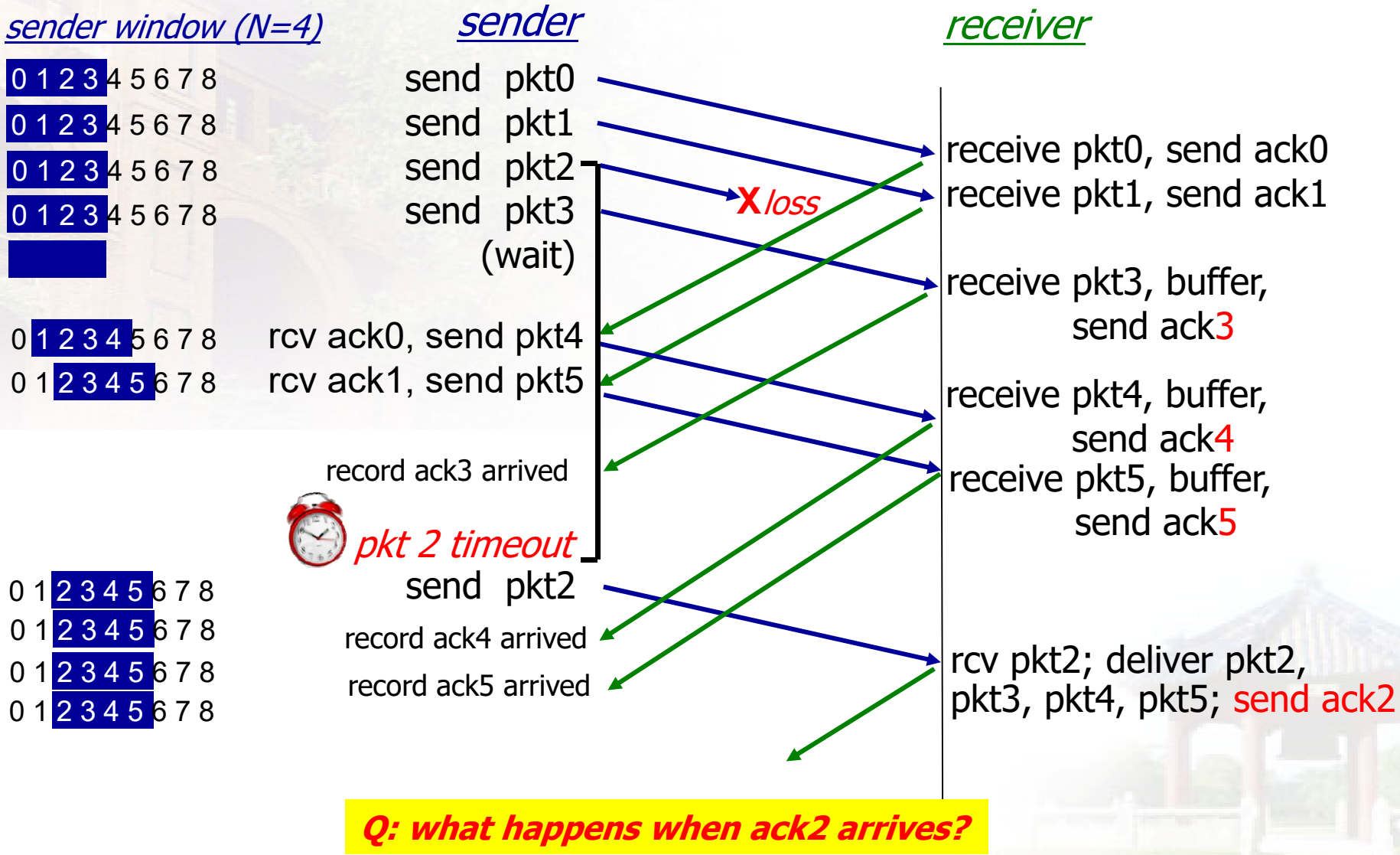
pkt n in [rcvbase-N,rcvbase-1]

- ACK(n)

otherwise:

- ignore

Selective repeat in action



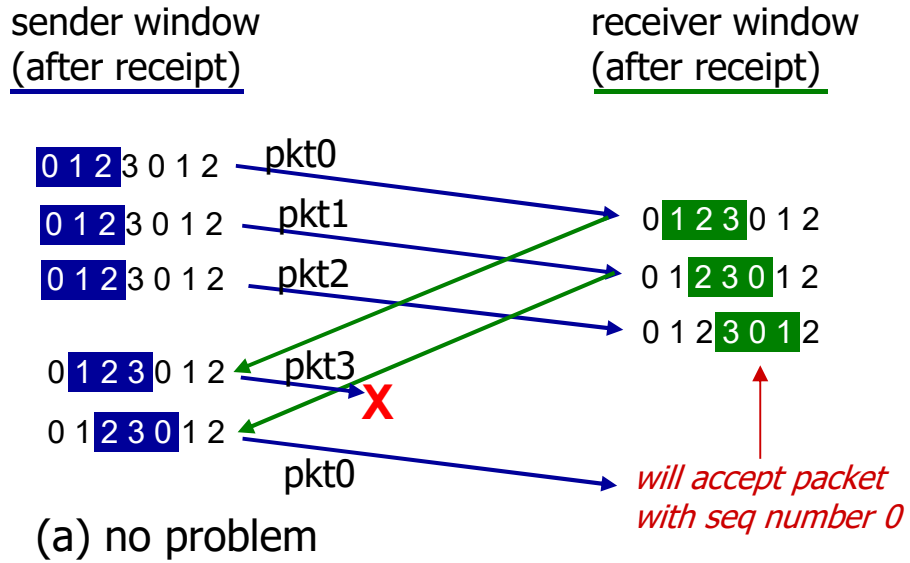
Selective repeat: dilemma

example:

- **seq #'s: 0, 1, 2, 3**
- **window size=3**
  - receiver sees no difference in two scenarios!
  - duplicate data accepted as new in (b)

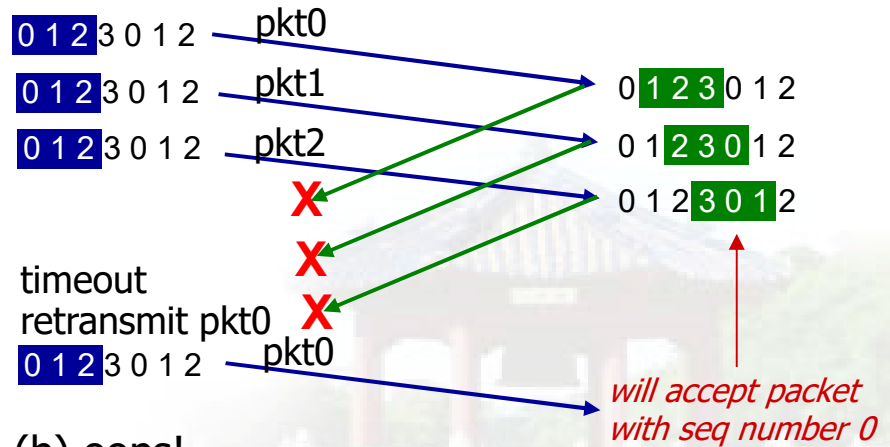
Q: what relationship between seq # size and window size to avoid problem in (b)?

Window size (W) up to N/2
Why?



(a) no problem

receiver can't see sender side.
receiver behavior identical in both cases!
Something is (very) wrong!



(b) oops!

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

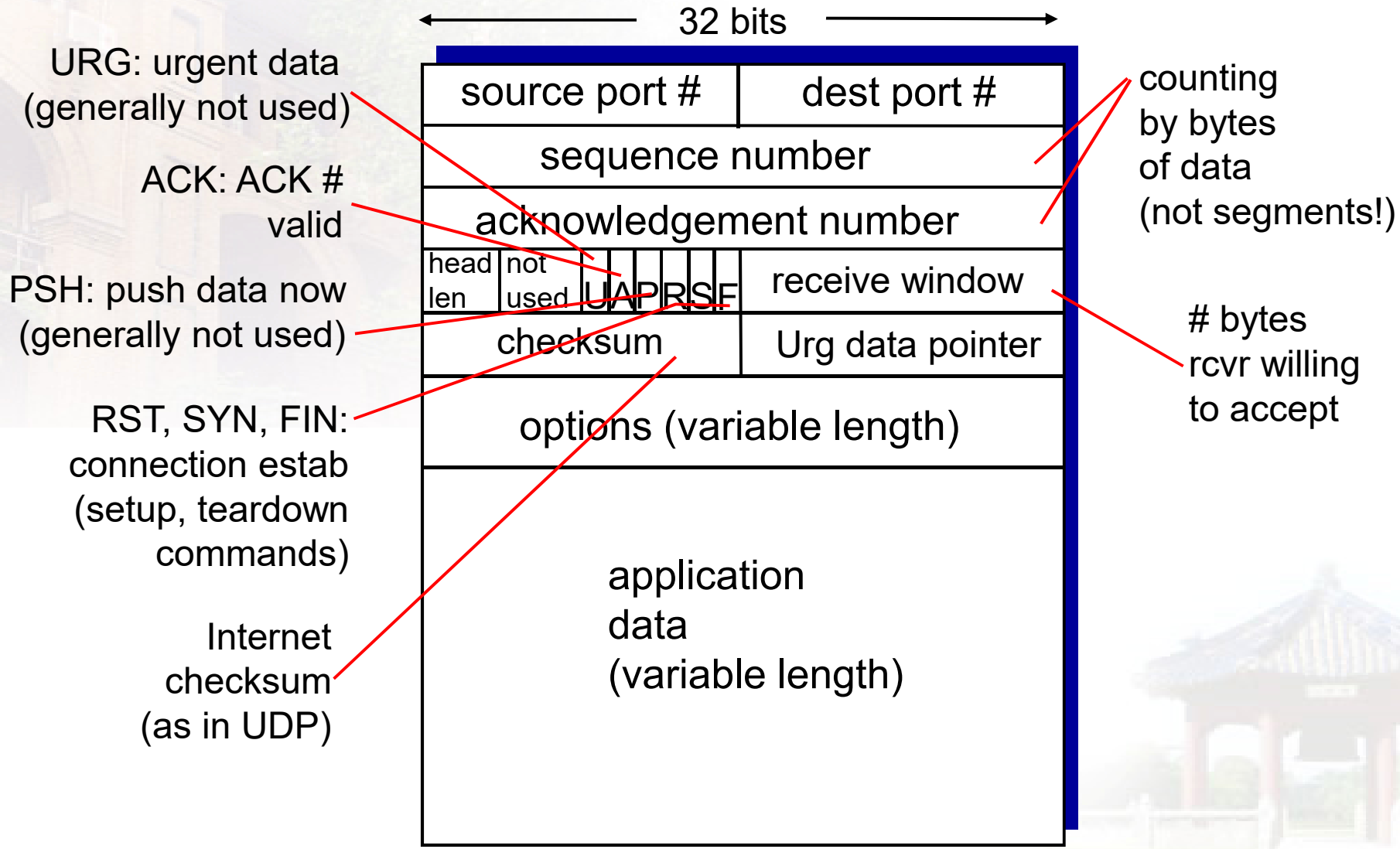
TCP: Overview RFCs: 793,1122, 2018, 5681, 7323

- **point-to-point:**
  - one sender, one receiver
- **reliable, in-order *byte steam:***
  - no "message boundaries"
- **pipelined:**
  - TCP congestion and flow control set window size

What is the purpose of the connection?

- **full duplex data:**
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- **connection-oriented:**
  - handshaking (exchange of control msgs) inits sender, receiver state before data exchange
- **flow controlled:**
  - sender will not overwhelm receiver

TCP segment structure



TCP seq. numbers, ACKs

sequence numbers:

- byte stream "number" of first byte in segment's data

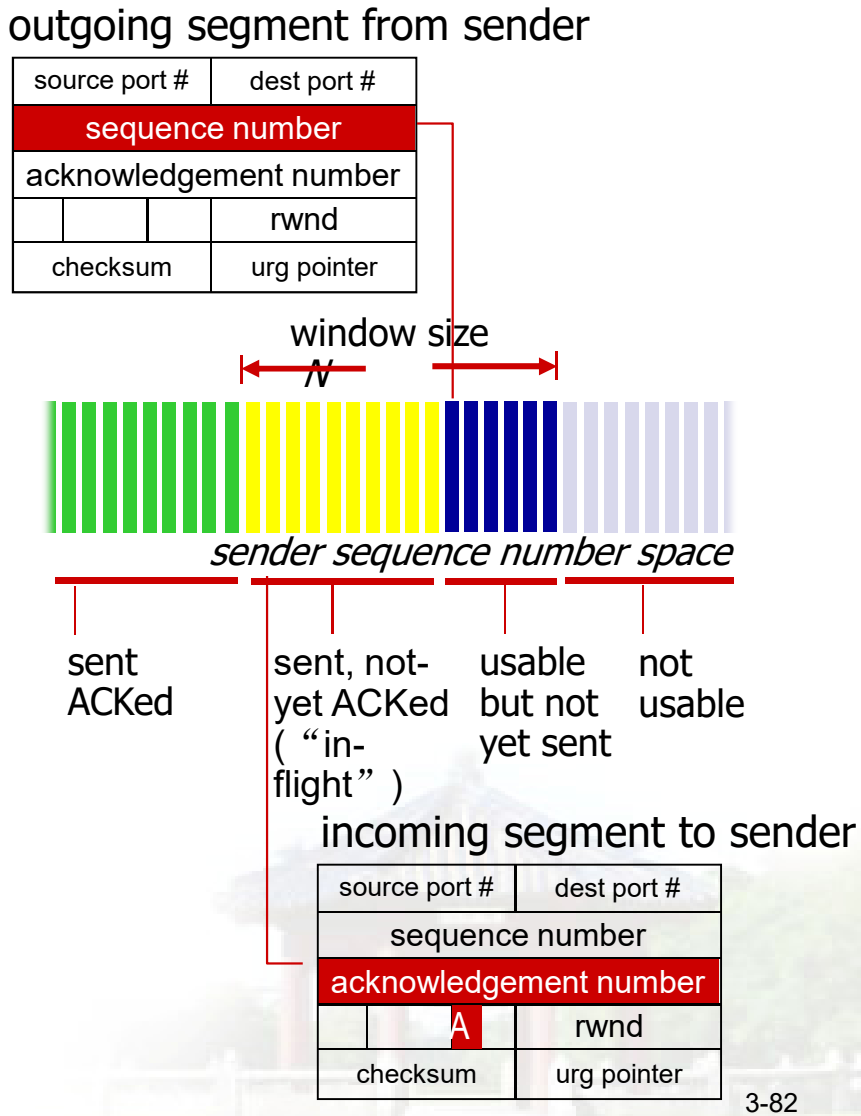
acknowledgements:

- seq # of NEXT byte expected from other side
- cumulative ACK

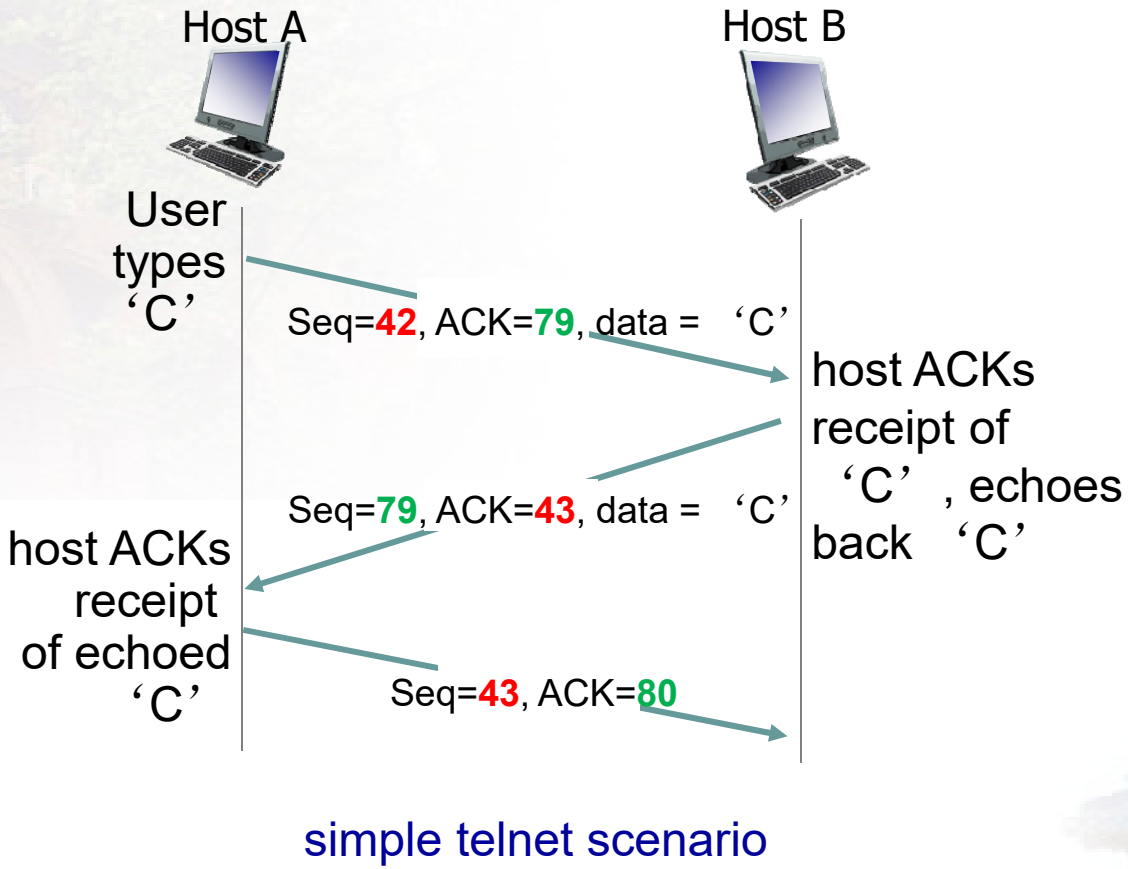
Q: how receiver handles out-of-order segments

- A: TCP spec doesn't say, - up to implementor

Neither GBN nor selective repeat



TCP seq. numbers, ACKs



simple telnet scenario

Seq: 当前序号；

Ack: 待接收序号；

TCP round trip time, timeout

Q: how to set TCP timeout value?

- longer than RTT
  - but RTT varies
- *too short:* premature timeout, unnecessary retransmissions
- *too long:* slow reaction to segment loss

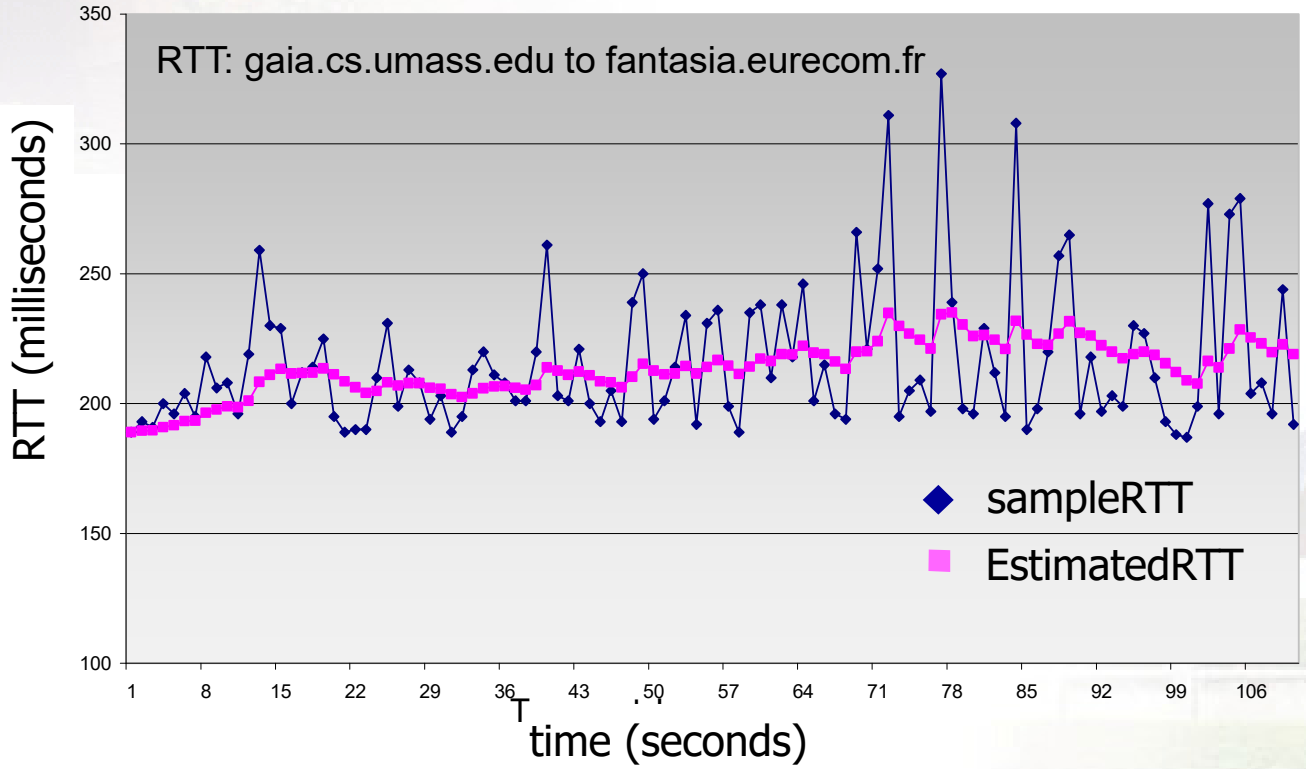
Q: how to estimate RTT?

- `SampleRTT`: measured time from segment transmission until ACK receipt
  - ignore retransmissions
- `SampleRTT` will vary, want estimated RTT "smoother"
  - average several *recent* measurements, not just current `SampleRTT`

TCP round trip time, timeout

EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT

- exponential weighted moving average
- influence of past sample decreases exponentially fast
- typical value: α = 0.125



TCP round trip time, timeout

- **timeout interval:** `EstimatedRTT` plus "safety margin"
  - large variation in `EstimatedRTT` -> larger safety margin
- estimate SampleRTT deviation from EstimatedRTT:

```
DevRTT = (1-β)*DevRTT +
         β*|SampleRTT-EstimatedRTT|
```

(typically, β = 0.25)

```
TimeoutInterval = EstimatedRTT + 4*DevRTT
```



estimated RTT

"safety margin"

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

TCP reliable data transfer

retransmission

- TCP creates rdt service on top of IP's unreliable service
  - pipelined segments
  - cumulative acks
  - single retransmission timer
- retransmissions triggered by:
  - timeout events
  - duplicate acks

Let's initially consider simplified TCP sender:

- ignore duplicate acks
- ignore flow control, congestion control

TCP sender events:

data rcvd from app:

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
  - think of timer as for oldest unacked segment
  - expiration interval: `TimeOutInterval`

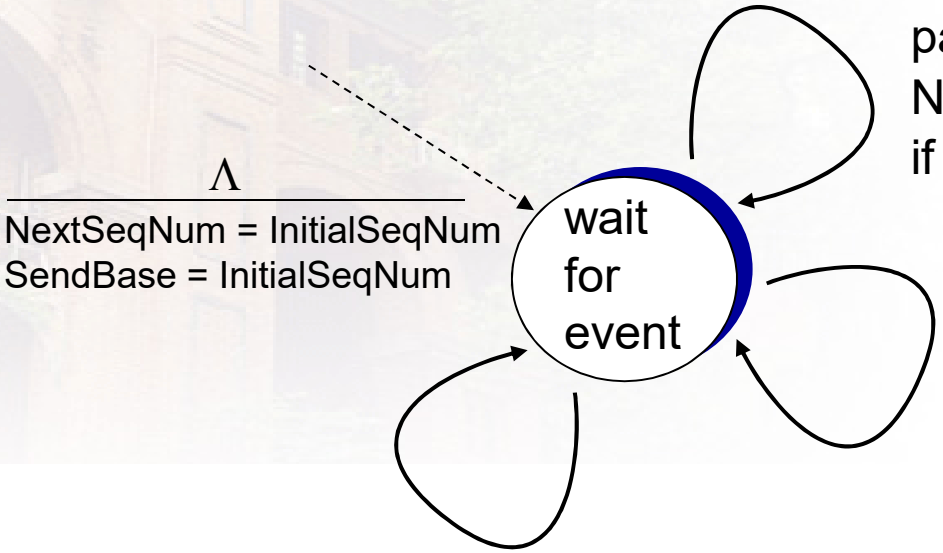
timeout:

- retransmit segment that caused timeout
- restart timer

ack rcvd:

- if ack acknowledges previously unacked segments
  - update what is known to be ACKed
  - start timer if there are still unacked segments

TCP sender (simplified)



data received from application above

create segment, seq. #: NextSeqNum
pass segment to IP (i.e., "send")
NextSeqNum = NextSeqNum + length(data)
if (timer currently not running)
  start timer

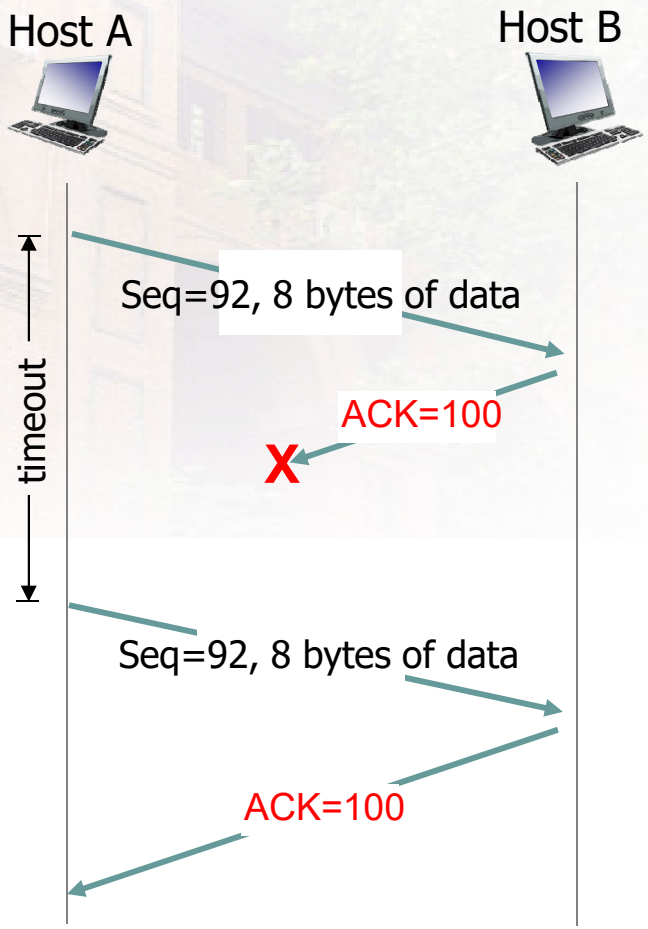
timeout

retransmit not-yet-acked segment
  with smallest seq. #
start timer

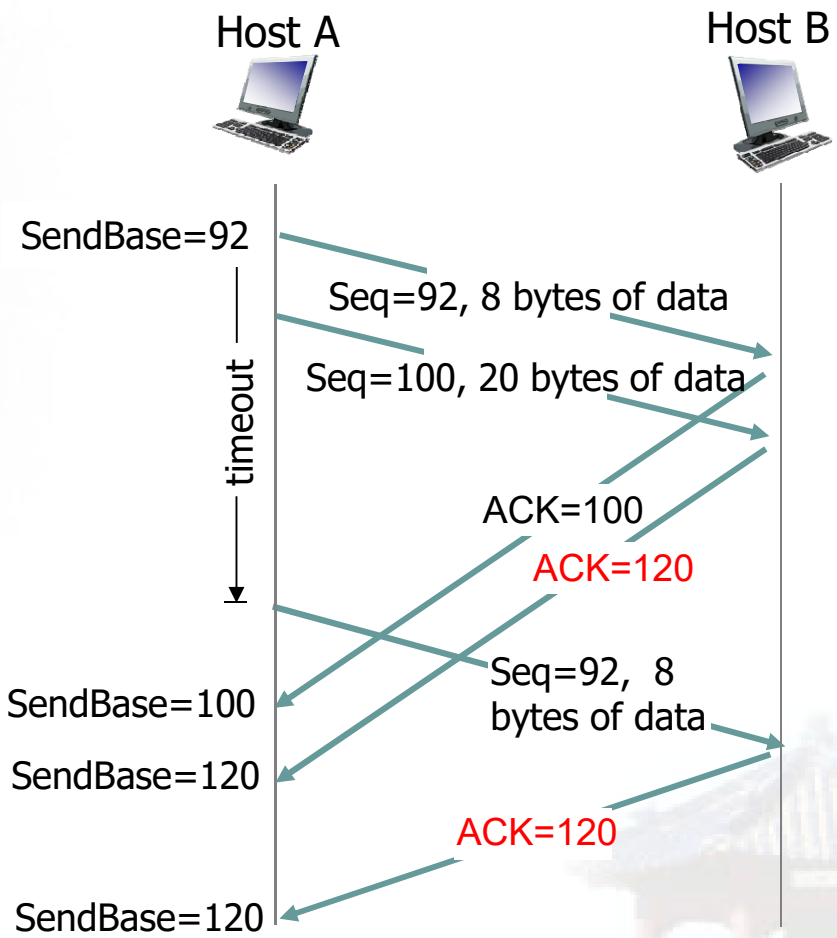
ACK received, with ACK field value y

```
if (y > SendBase) {
   SendBase = y
   /* SendBase–1: last cumulatively ACKed byte */
   if (there are currently not-yet-acked segments)
      start timer
    else stop timer
   }
```

TCP: retransmission scenarios

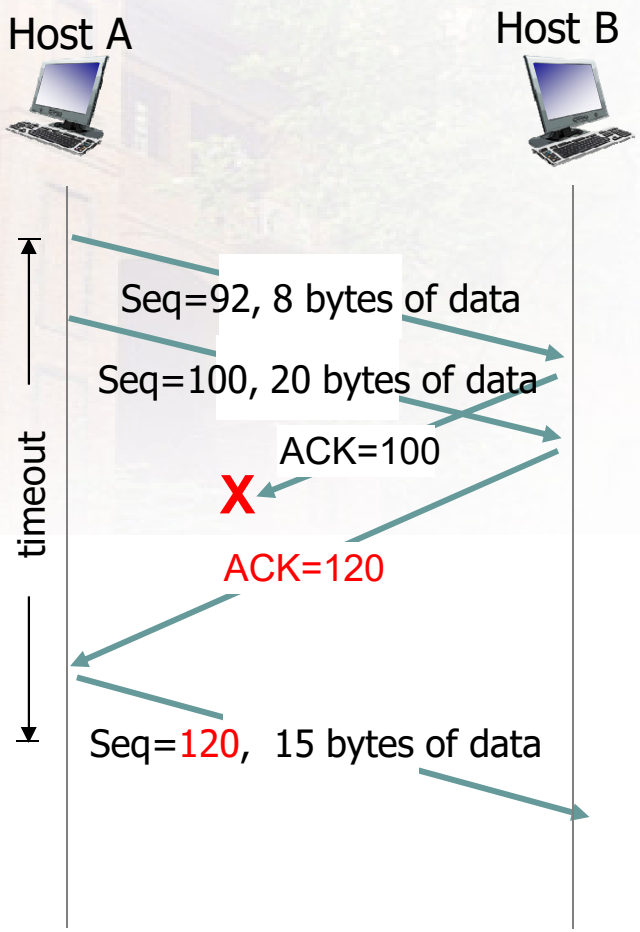


lost ACK scenario



premature timeout

TCP: retransmission scenarios



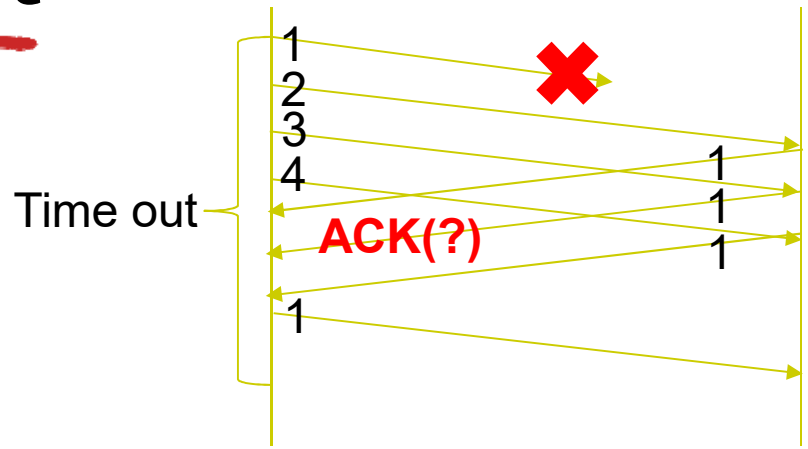
cumulative ACK

TCP ACK generation [RFC 1122, RFC 2581]

event at receiver	*TCP receiver action*
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send *duplicate ACK*, indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

TCP fast retransmit

- **time-out period often relatively long:**
  - **long delay before resending lost packet**
- **detect lost segments via duplicate ACKs.**
  - **sender often sends many segments back-to-back**
  - **if segment is lost, there will likely be many duplicate ACKs.**

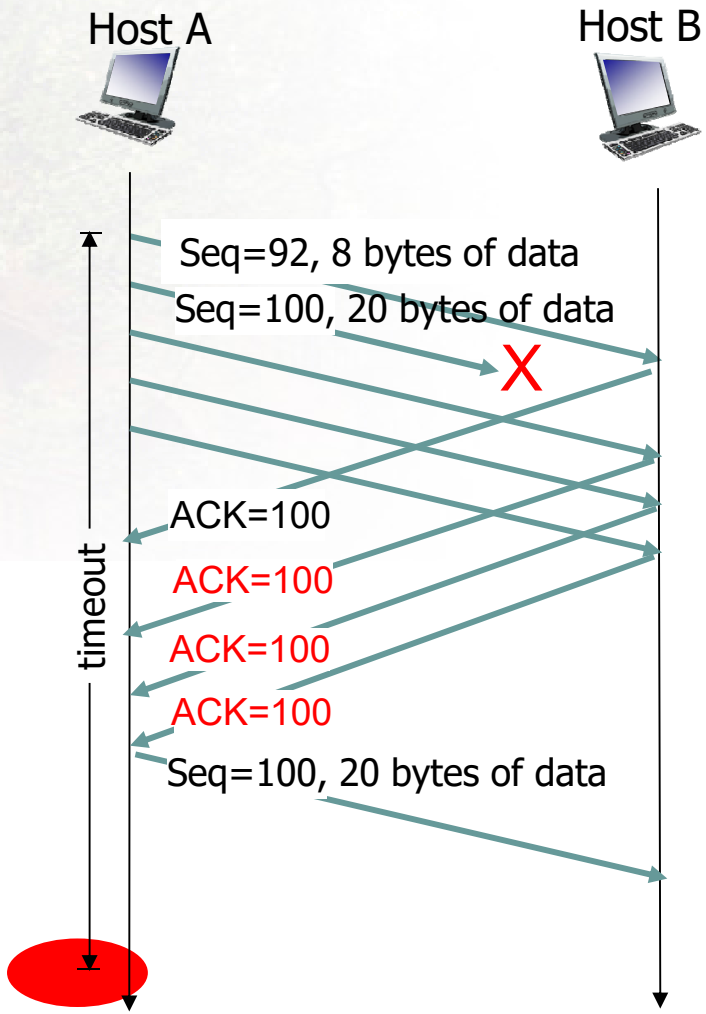


TCP fast retransmit

if sender receives **3** ACKs for same data ("triple duplicate ACKs"), resend unacked segment with smallest seq #

- likely that unacked segment lost, so don't wait for timeout

TCP fast retransmit



fast retransmit after sender receipt of triple duplicate ACK

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

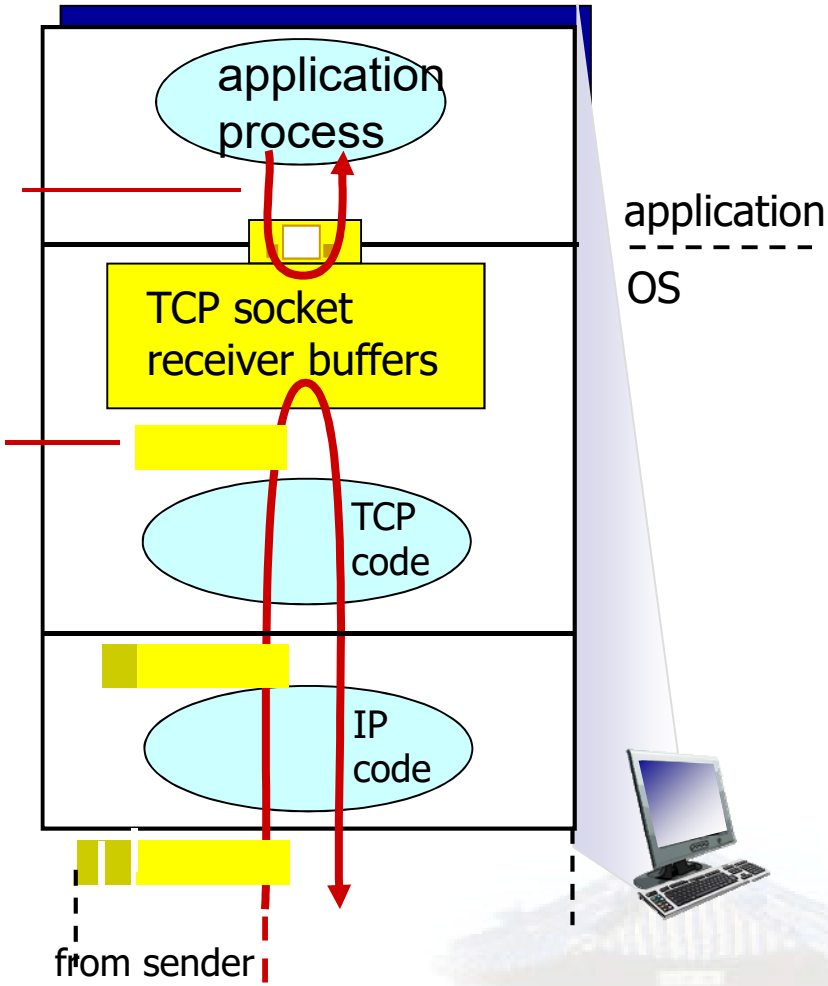
3.6 principles of congestion control

3.7 TCP congestion control

TCP flow control

application may remove data from TCP socket buffers ….

… slower than TCP receiver is delivering (sender is sending)



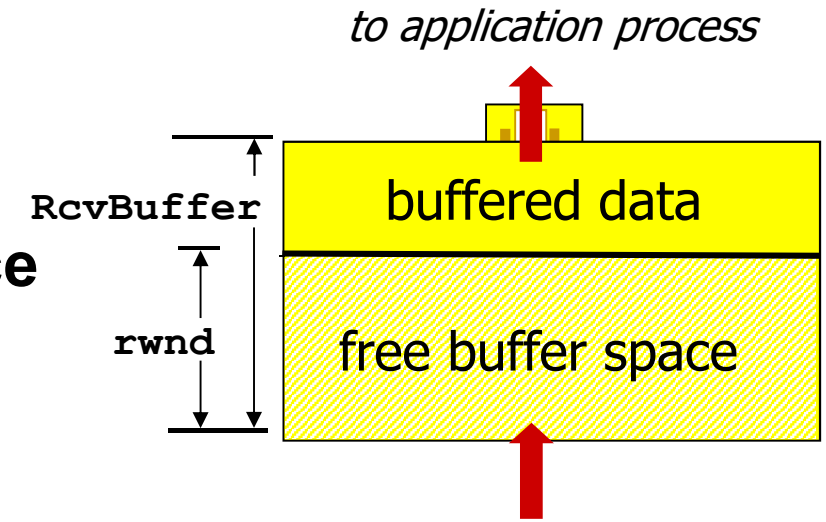
receiver protocol stack

flow control

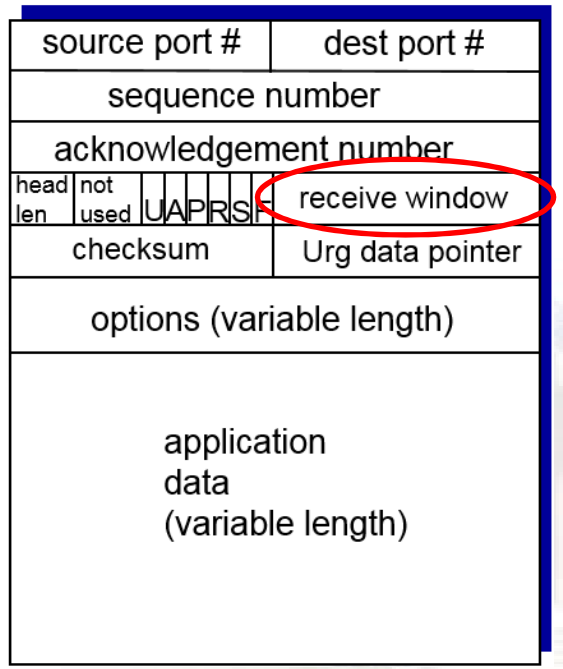
receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast

TCP flow control

- receiver "advertises" free buffer space by including `rwnd` value in TCP header of receiver-to-sender segments
  - `RcvBuffer` size set via socket options (typical default is 4096 bytes)
  - many operating systems autoadjust `RcvBuffer`
- sender limits amount of unacked ("in-flight") data to receiver's `rwnd` value
- guarantees receive buffer will not overflow



receiver-side buffering



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

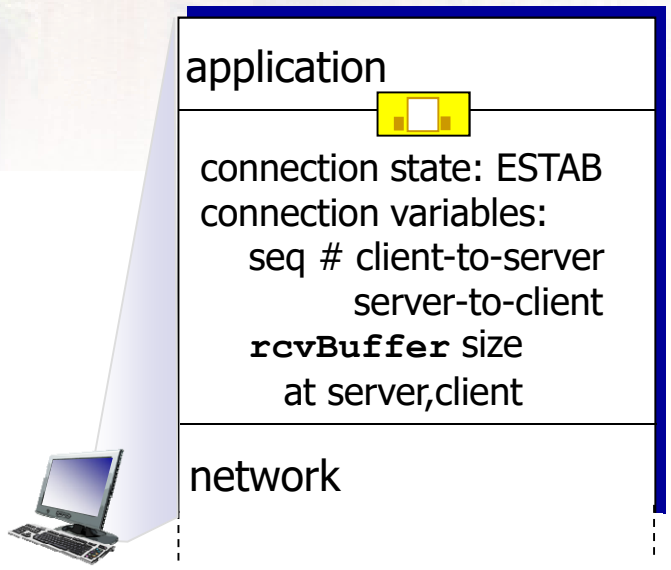
3.6 principles of congestion control

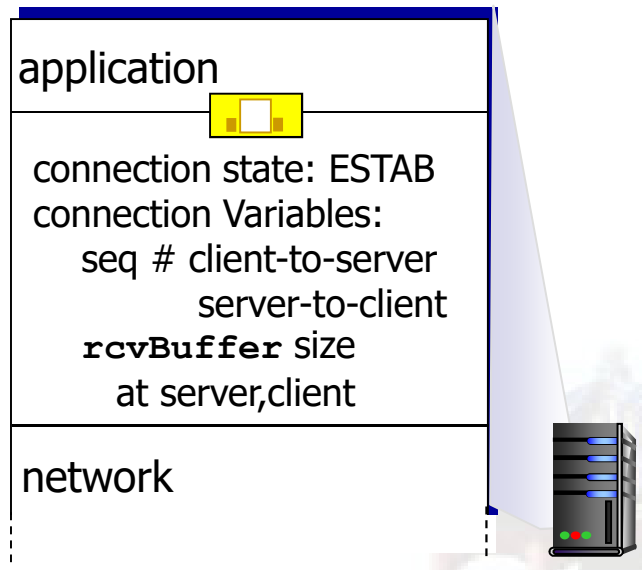
3.7 TCP congestion control

Connection Management

before exchanging data, sender/receiver "handshake":

- **agree to establish connection (each knowing the other willing to establish connection)**
- **agree on connection parameters**



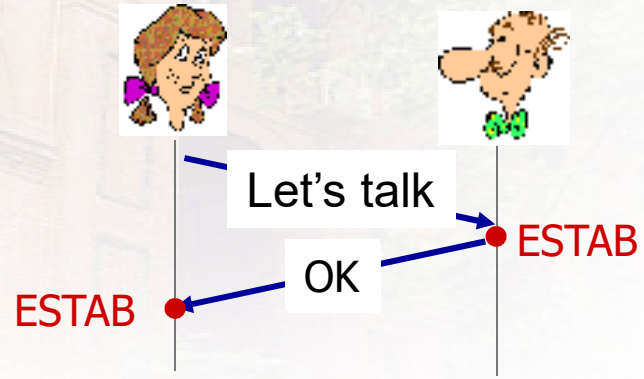


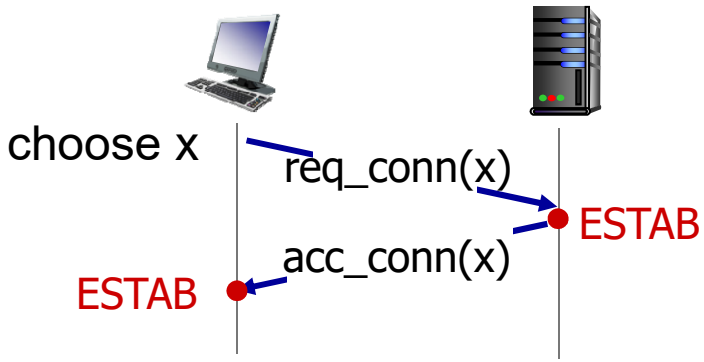

```
Socket clientSocket =
  newSocket("hostname","port
   number");
```

```
Socket connectionSocket =
   welcomeSocket.accept();
```

Agreeing to establish a connection

2-way handshake:





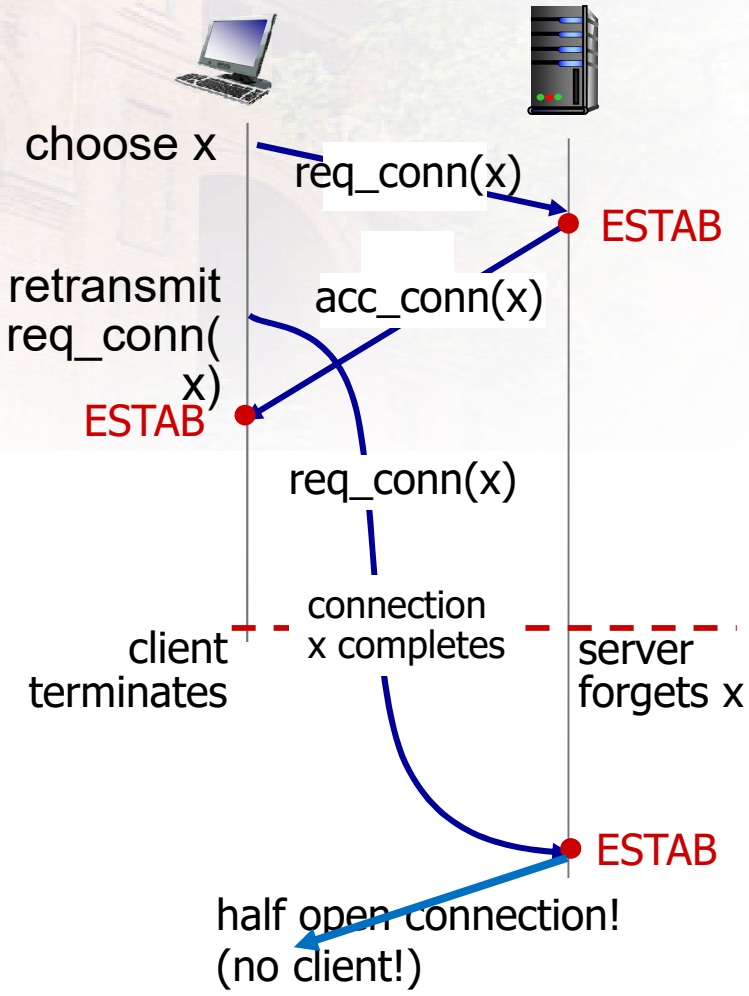
Q: **will 2-way handshake always work in network?**

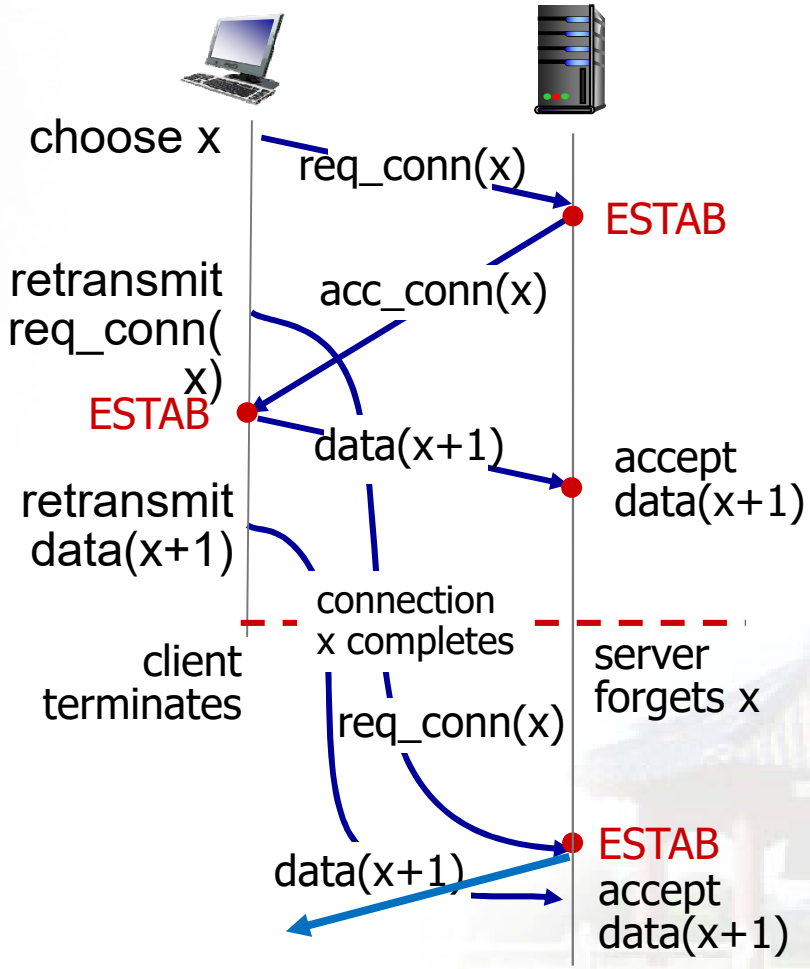
- **variable delays**
- **retransmitted messages (e.g. req_conn(x)) due to message loss**
- **message reordering**
- **Can't "see" other side**

2023.9.27

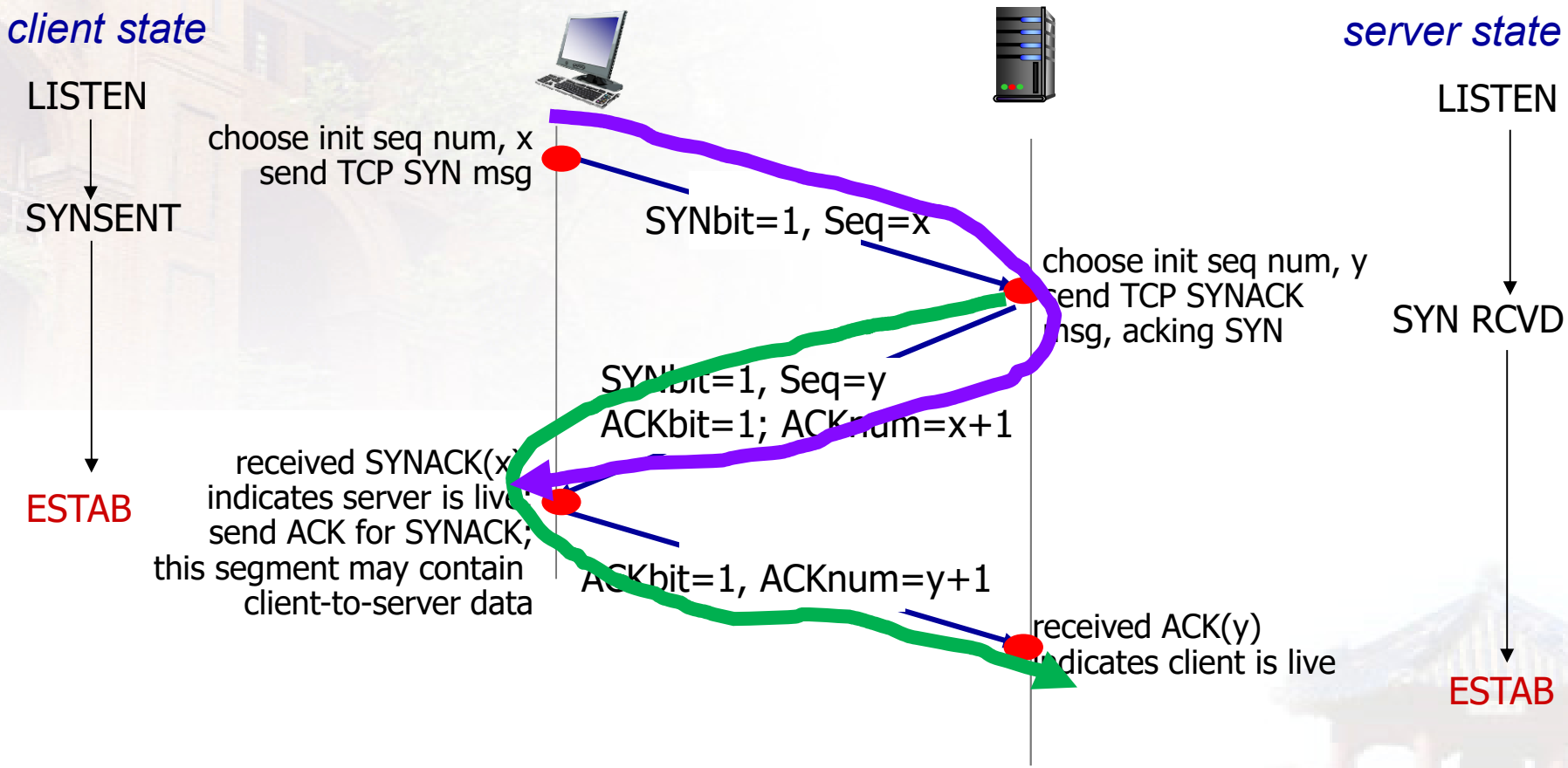
Agreeing to establish a connection

2-way handshake failure scenarios:

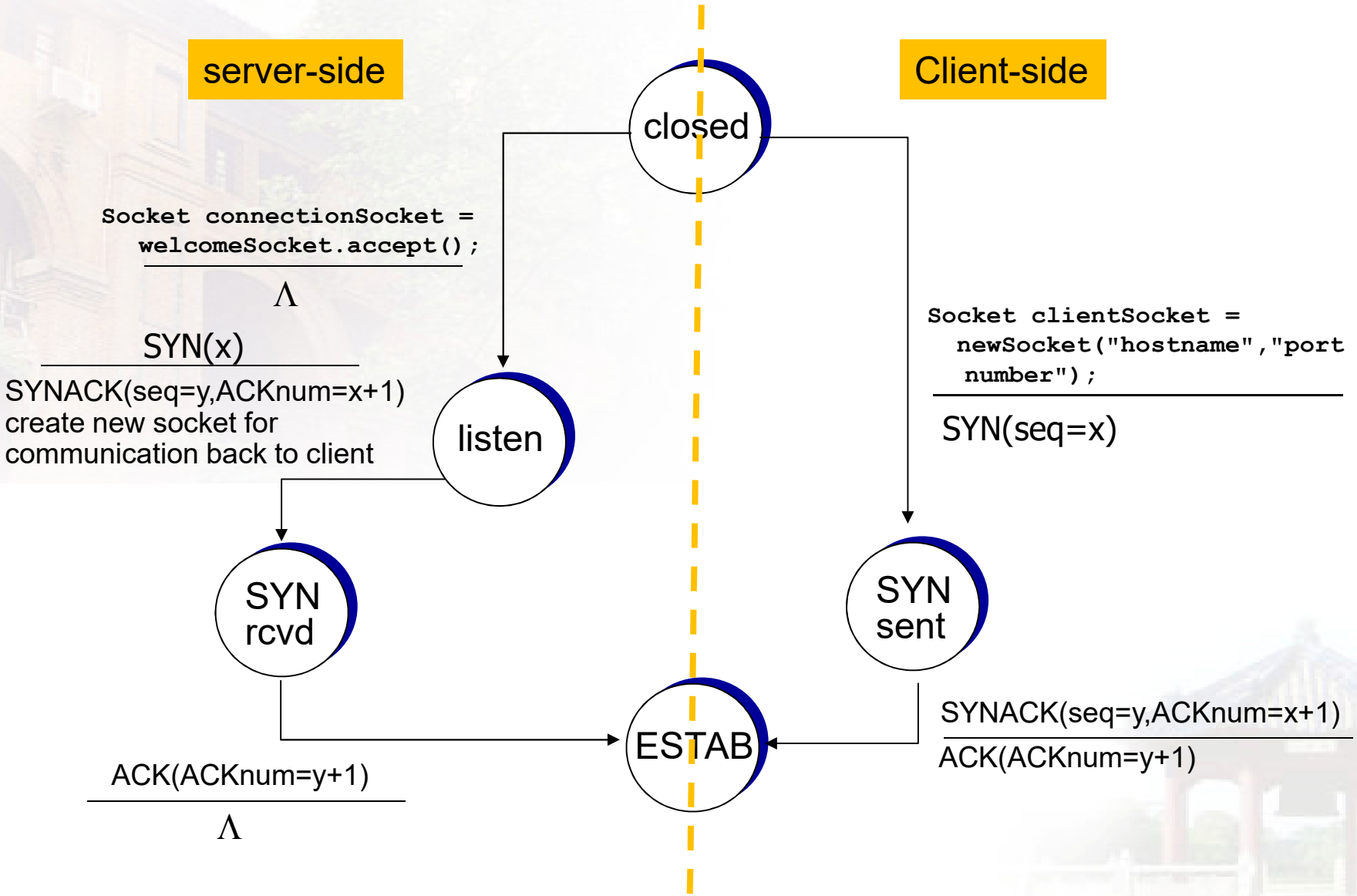




TCP 3-way handshake



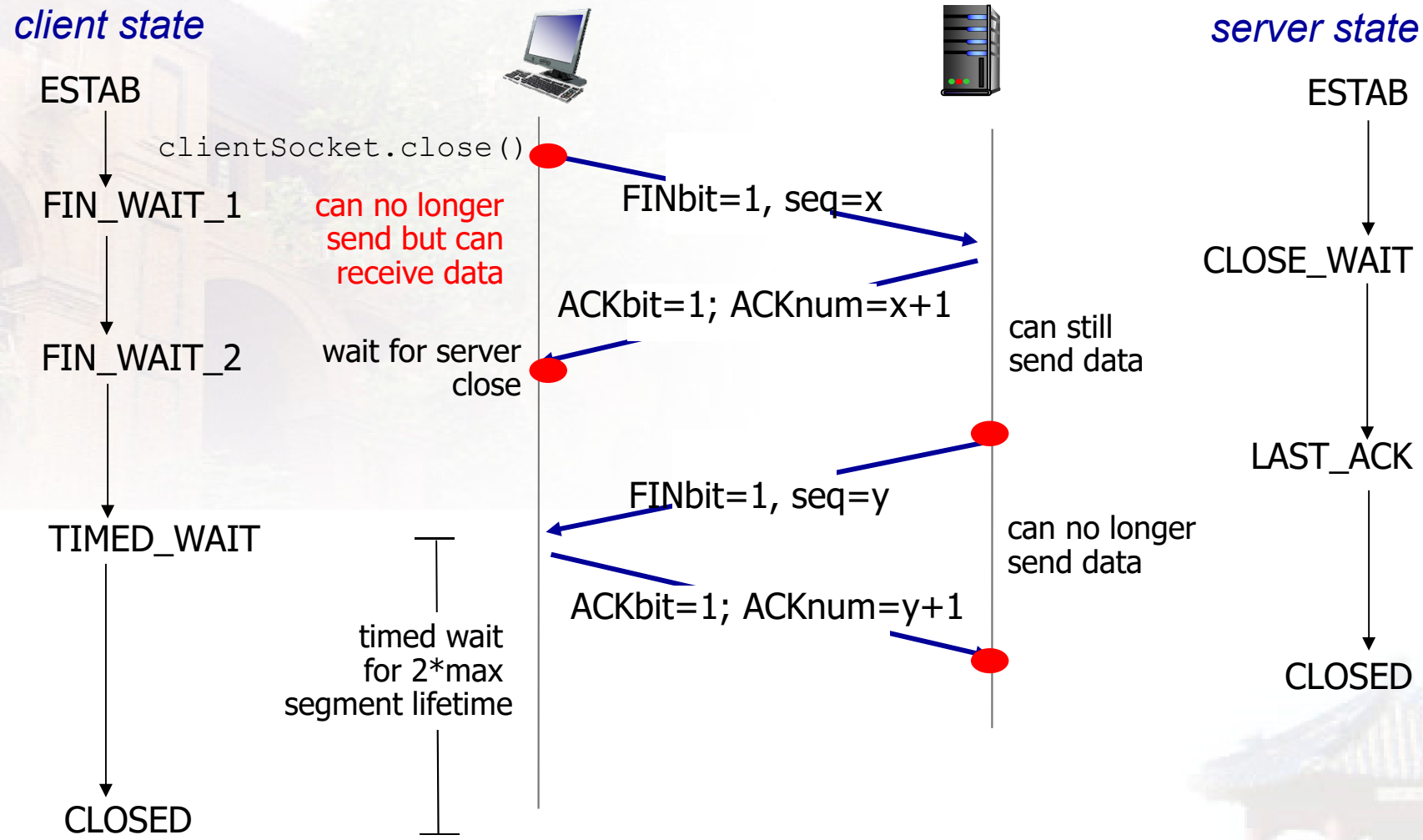
TCP 3-way handshake: FSM



TCP: closing a connection

- client, server each close their side of connection
  - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
  - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

TCP: closing a connection



The key idea of TCP: ACK is necessary for each sent segment.

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

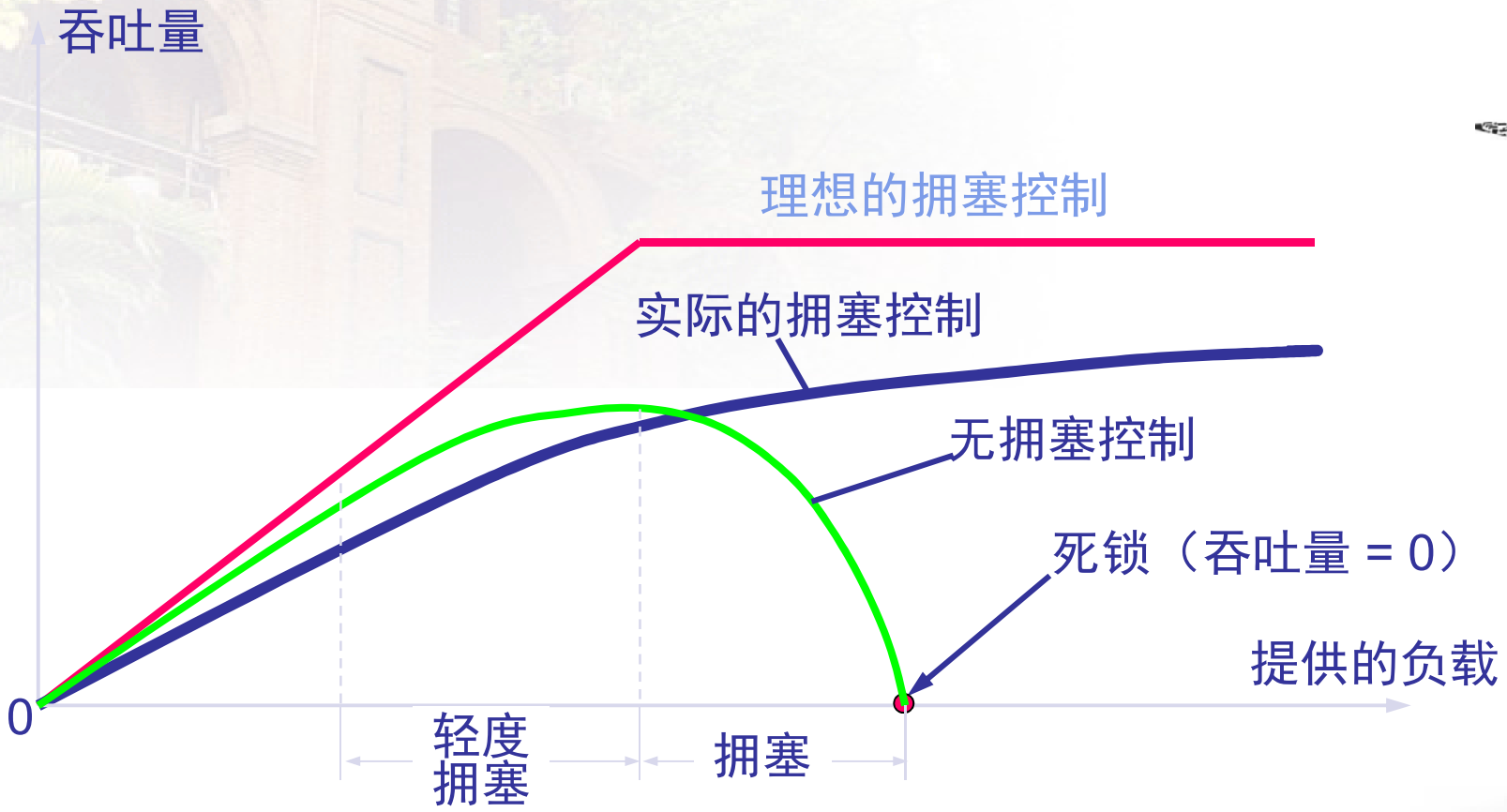
3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

拥塞控制所起的作用





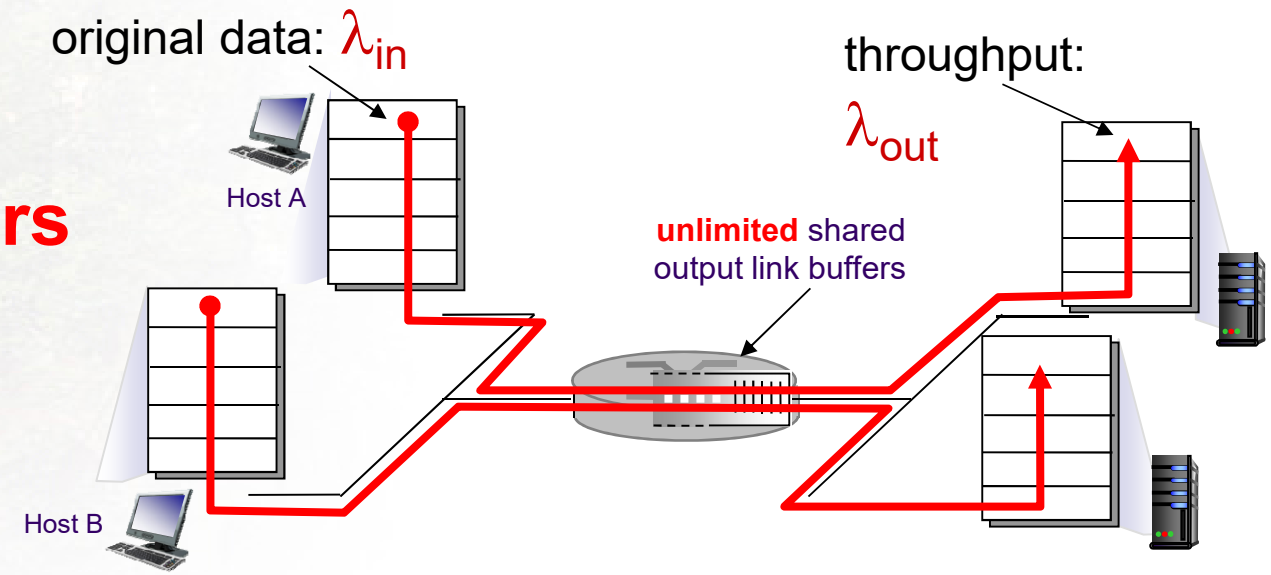
Principles of congestion control

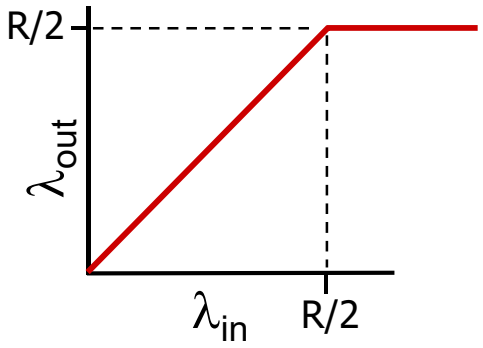
congestion:

- informally: "too many sources sending too much data too fast for ***network*** to handle"
- different from flow control!
- manifestations:
  - lost packets (buffer overflow at routers)
  - long delays (queueing in router buffers)
- a top-10 problem!

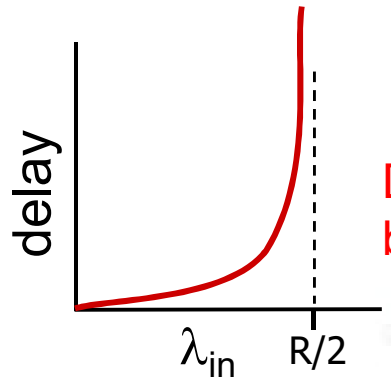
Causes/costs of congestion: scenario 1

- two senders, two receivers
- one router, **infinite buffers**
- output link capacity: R
- **no retransmission**





- **maximum per-connection throughput: R/2**

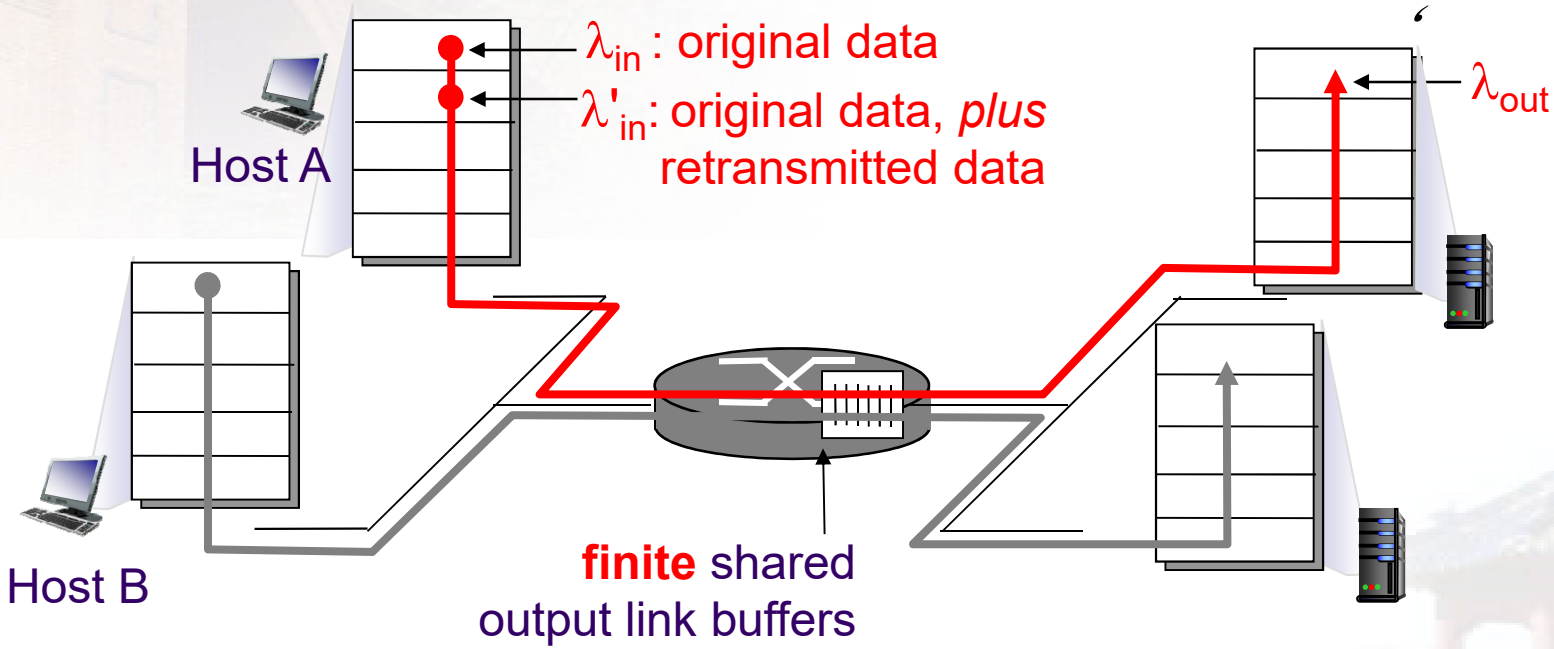


Due to infinite buffers

- large delays as arrival rate, λ_{in}, approaches capacity

Causes/costs of congestion: scenario 2

- one router, ***finite*** buffers
- sender **retransmission** of **timed-out** packet
  - application-layer input = application-layer output: $\lambda_{in} = \lambda_{out}$
  - transport-layer input includes ***retransmissions*** : $\lambda'_{in} \geq \lambda_{in}$



无策略的重传导致网络中数据继续增加， 拥塞更加严重！

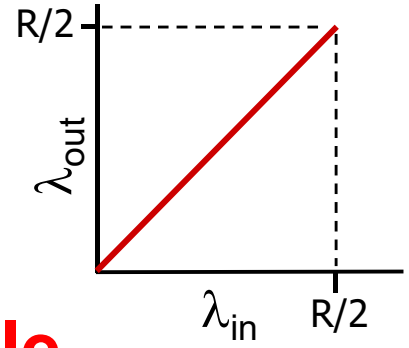
Causes/costs of congestion: scenario 2

Considering sender's **retransmission**

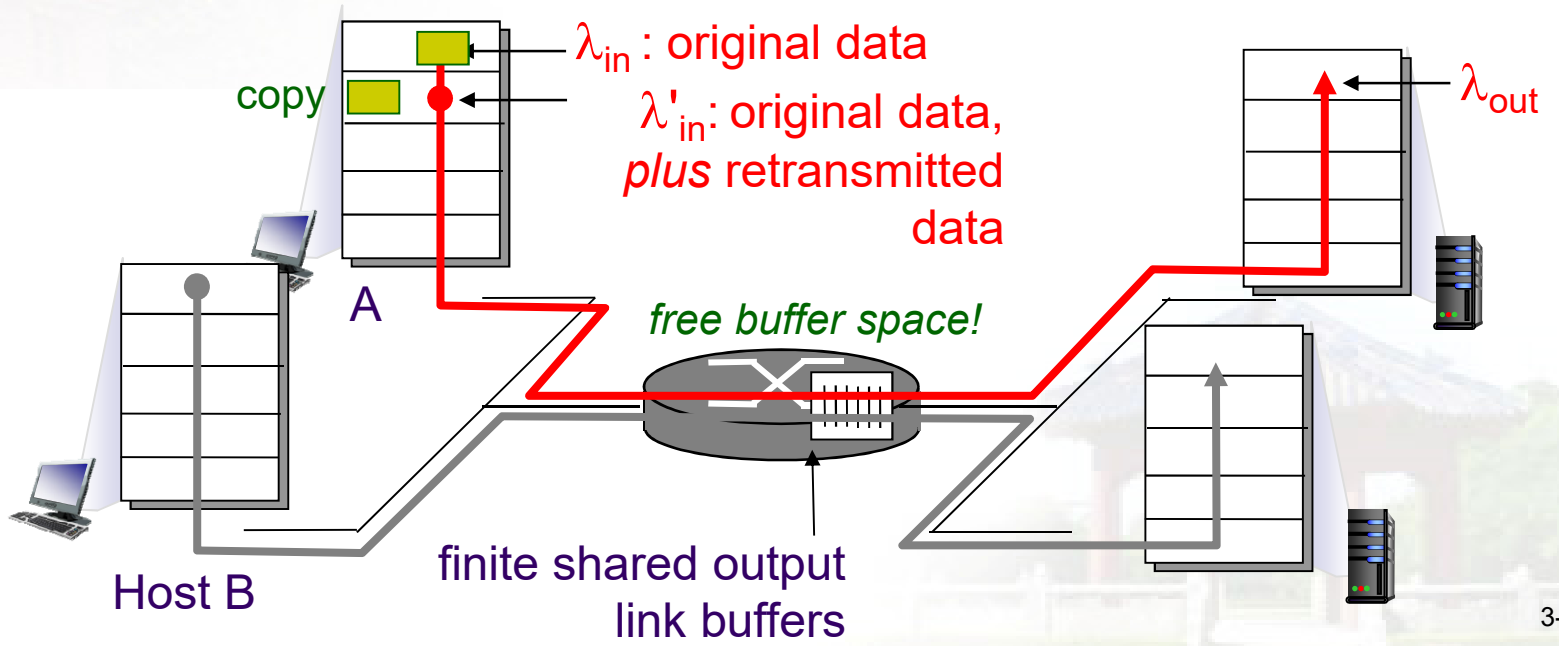
idealization: perfect knowledge

(Sender should retransmits timed-out pkts at the right time!!!)

- **sender sends only when router buffers available**



选择合适的时机进行重传

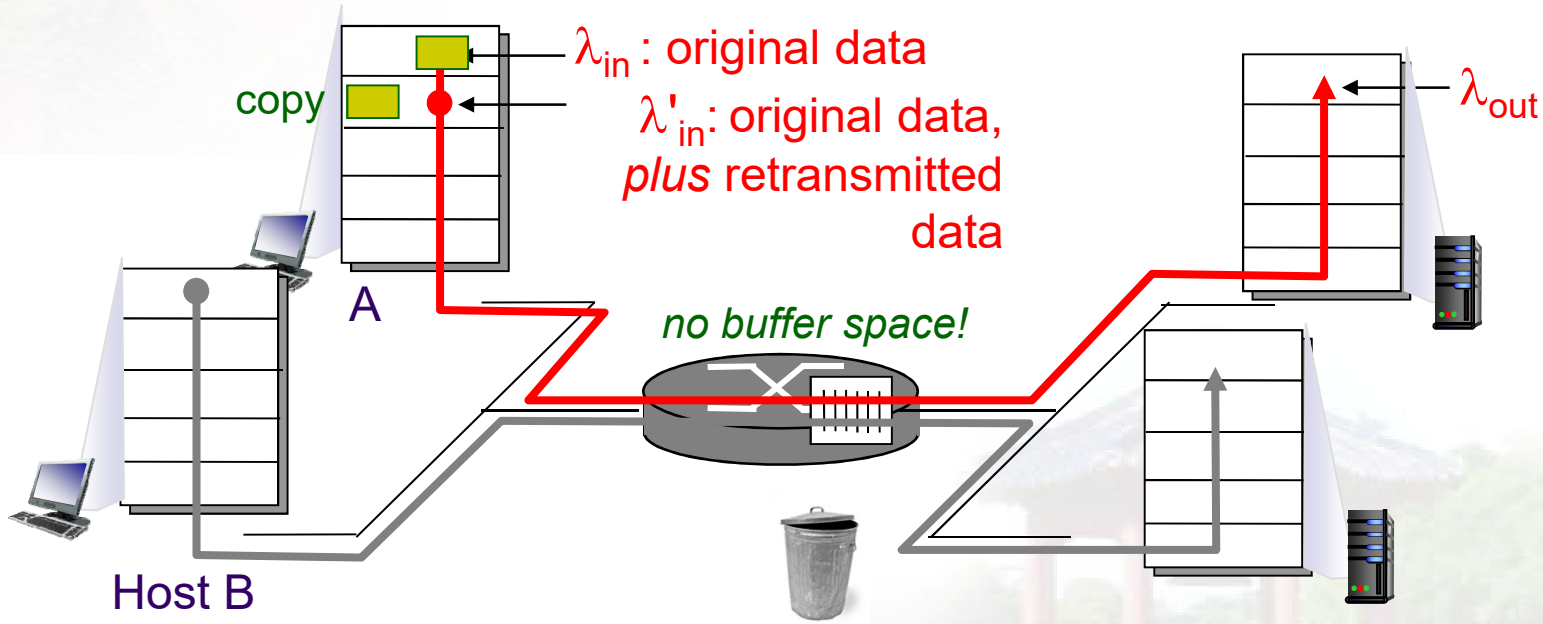


Causes/costs of congestion: scenario 2

Considering sender's retransmission

Idealization: known pkts are indeed lost **Time-out ≠ loss**

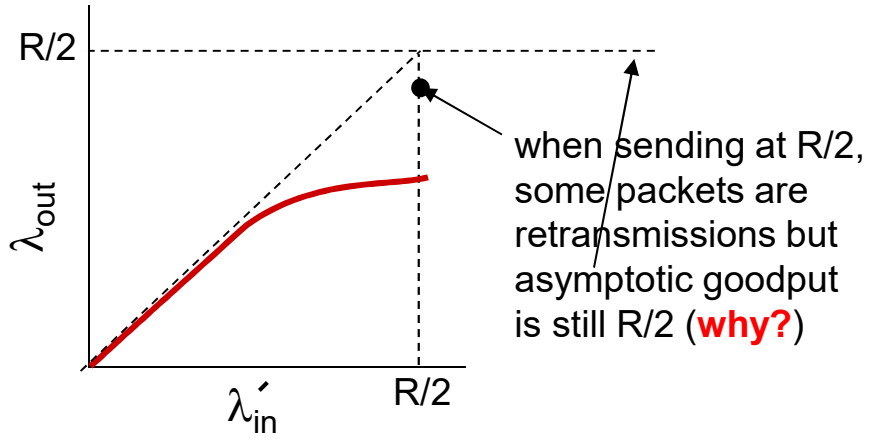
- packets can be lost, dropped at router due to full buffers
- sender only resends if packet *known* to be lost instead of timed-out.



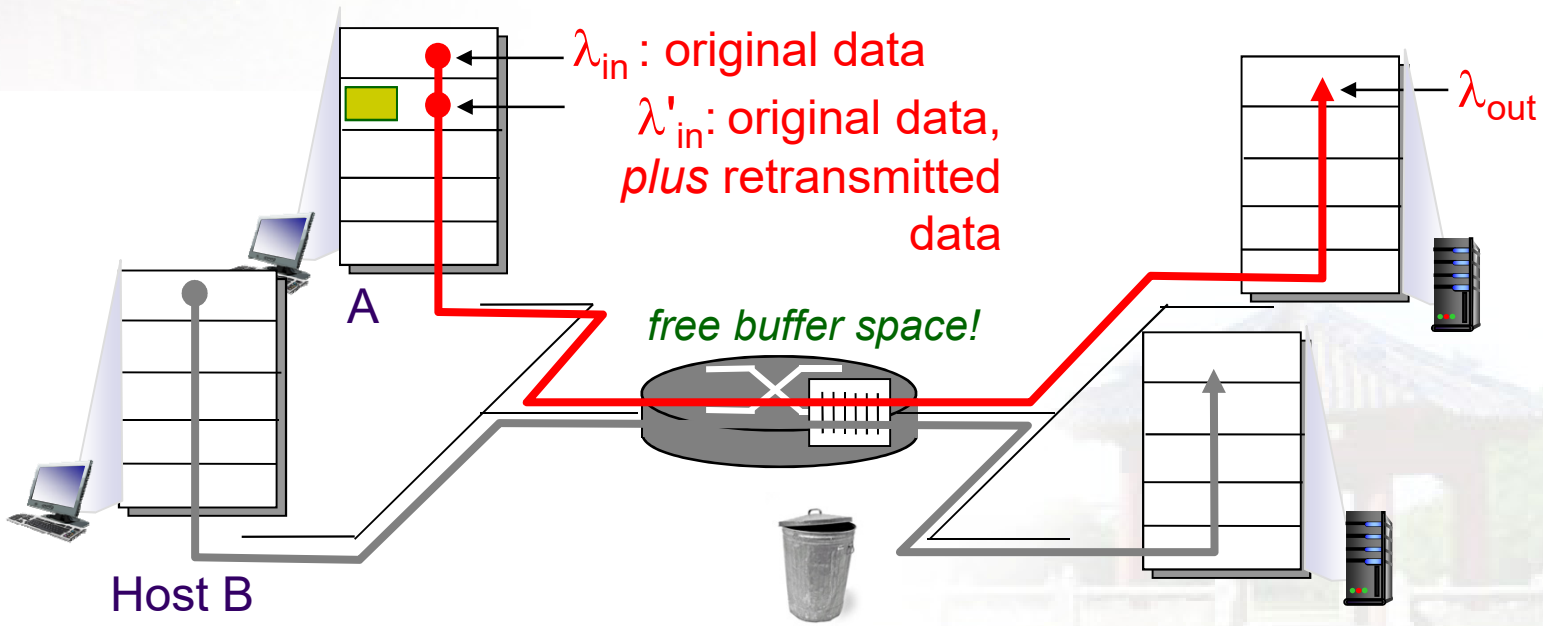
Causes/costs of congestion: scenario 2

Idealization: known loss **packets can be lost, dropped at router due to full buffers**

- **sender only resends if packet *known* to be lost**



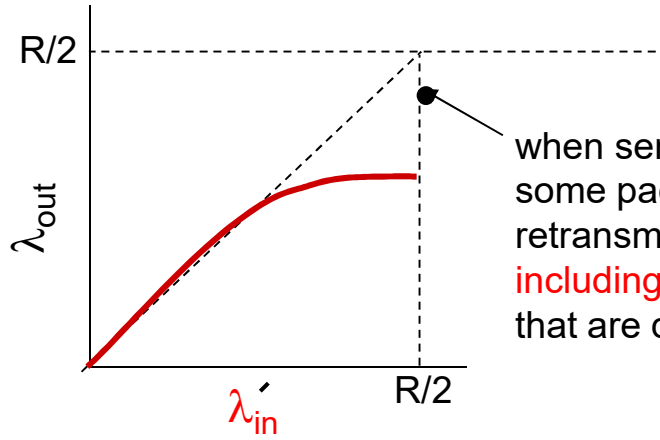
选择丢失分组
进行精准重传



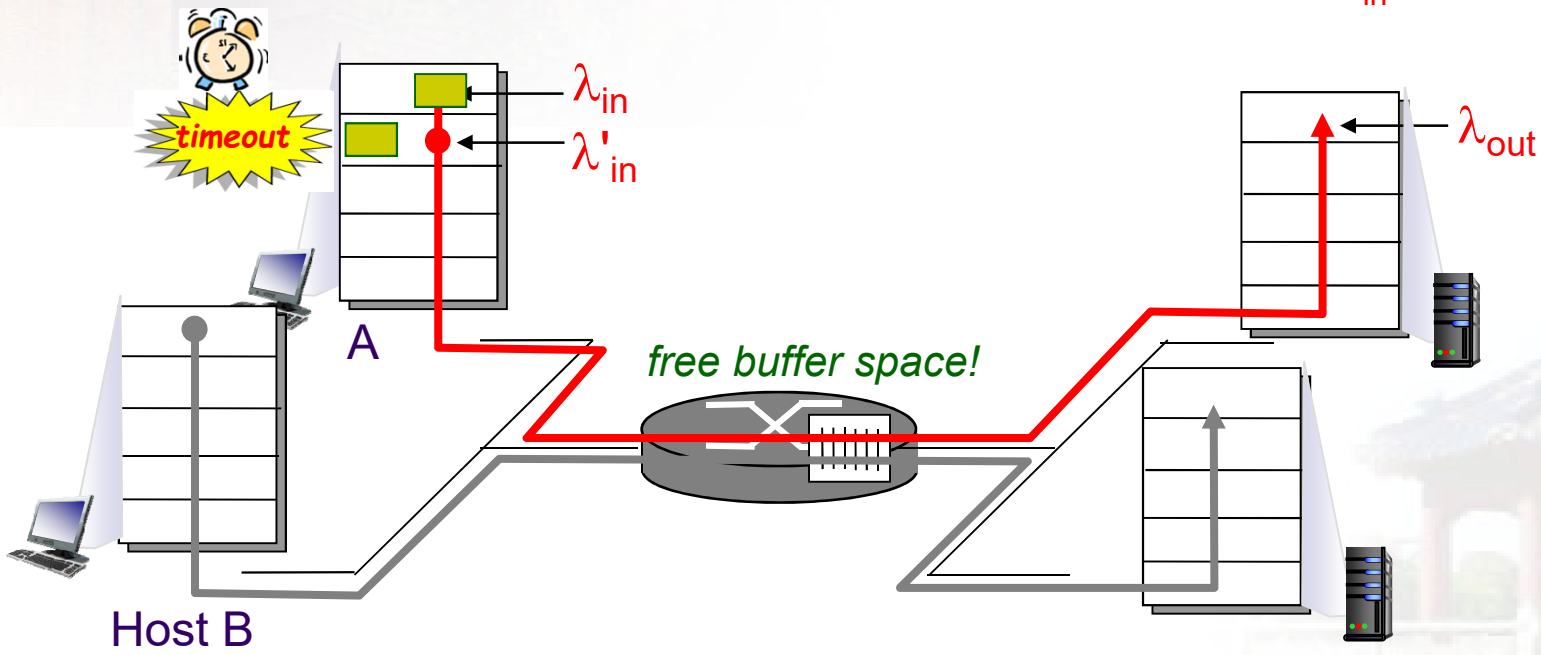
Causes/costs of congestion: scenario 2

Realistic: duplicates

- packets can be lost, dropped at router due to full buffers
- sender times out prematurely, sending ***two copies***, both of which are delivered



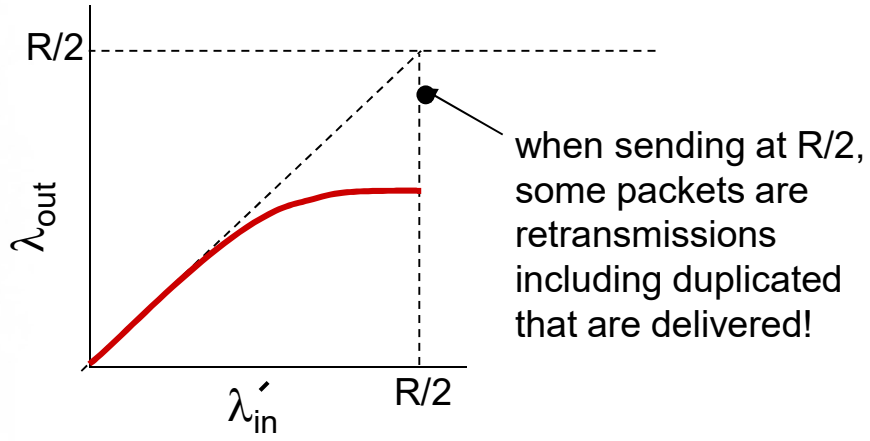
when sending at R/2, some packets are retransmissions including duplicated that are delivered!



Causes/costs of congestion: scenario 2

Realistic: duplicates

- packets can be lost, dropped at router due to full buffers
- sender times out prematurely, sending ***two* copies**, both of which are delivered



"costs" of congestion:

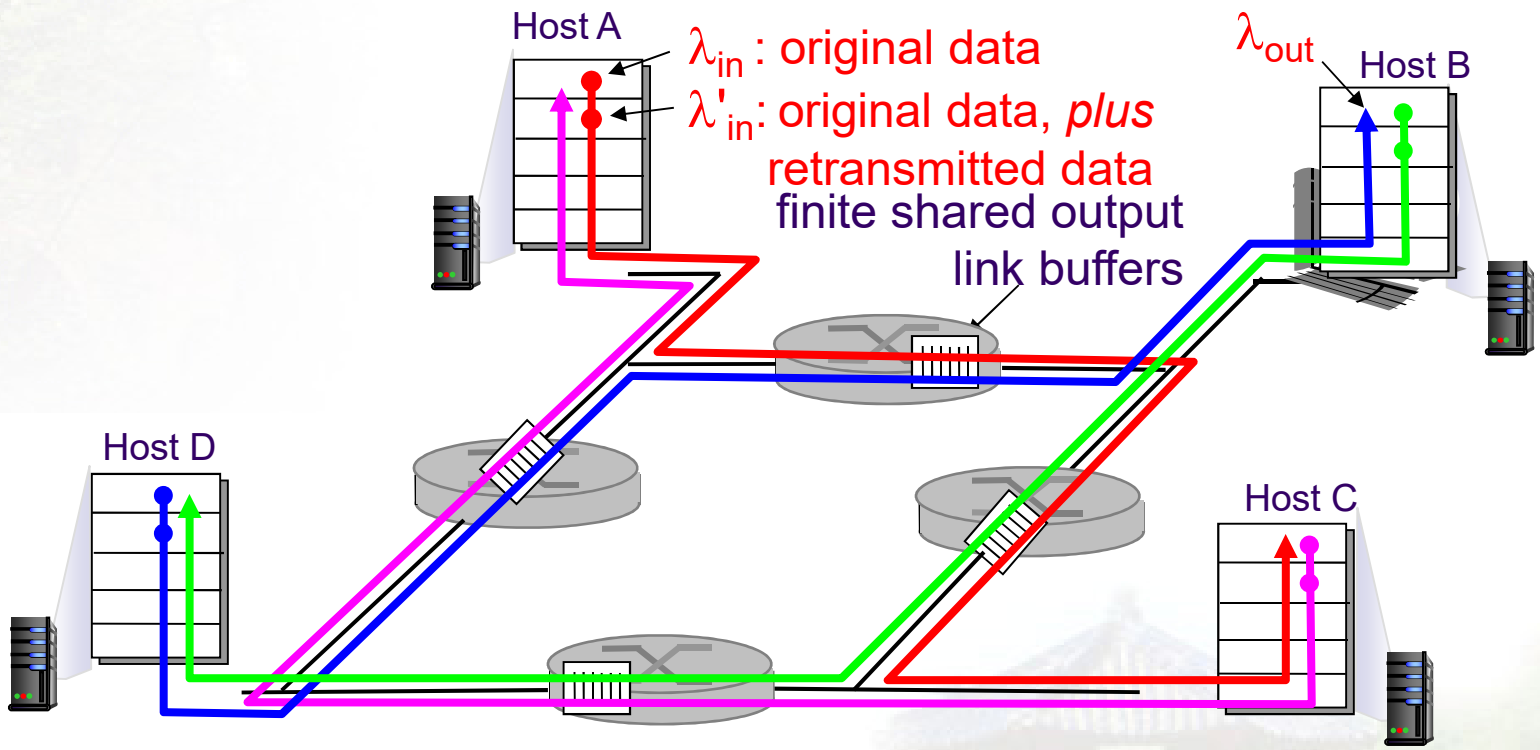
- more work (retrans) for given "goodput"
- unneeded retransmissions: link carries multiple copies of pkt
  - decreasing goodput

Causes/costs of congestion: scenario 3

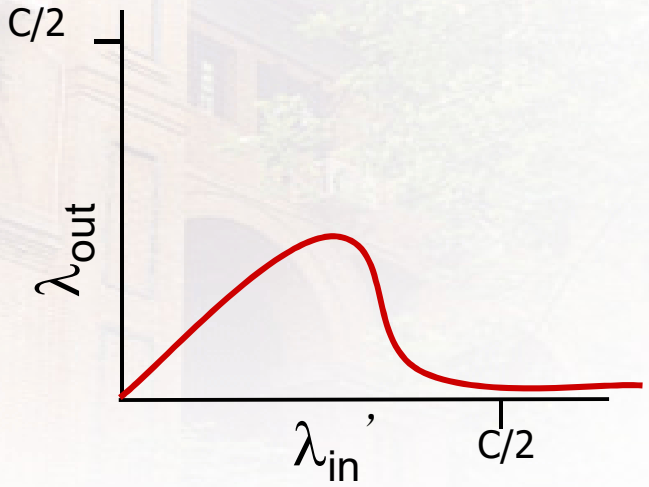
- four senders
- multihop paths
- timeout/retransmit

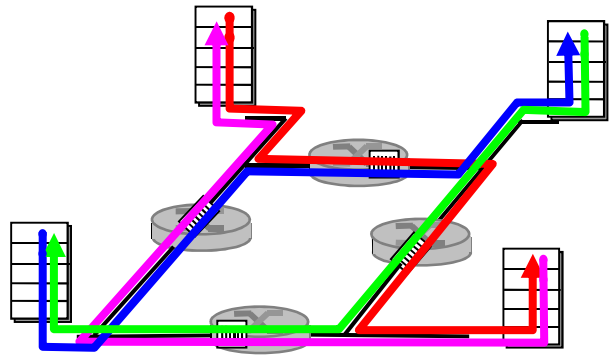
Q: what happens as λ_{in} and λ_{in}' increase ?

A: as red λ_{in}' increases, all arriving **blue** pkts at upper queue are dropped, blue throughput → 0



Causes/costs of congestion: scenario 3





another "cost" of congestion:

- when packet dropped, any upstream transmission capacity used for that packet was wasted!
- Cause upstream congestion!

Approaches towards congestion control

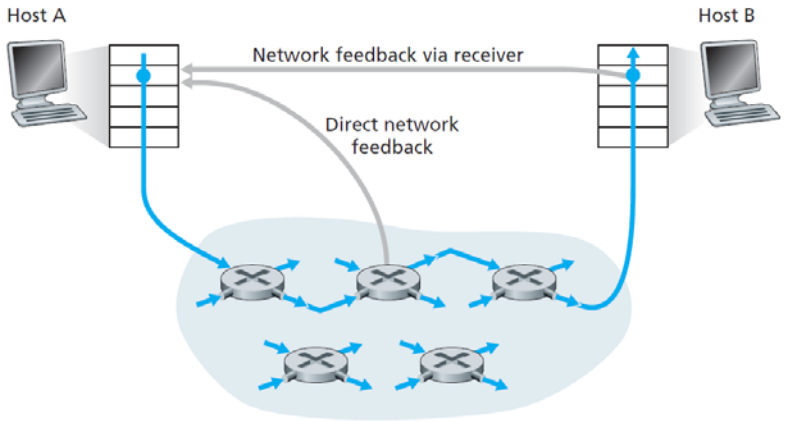
two broad approaches towards congestion control:

end-end congestion control:

- no explicit feedback from network
- congestion inferred from end-system observed loss, delay
- approach taken by TCP

network-assisted congestion control:

- routers provide feedback to end systems
  - single bit indicating congestion (SNA, DECbit, TCP/IP ECN, ATM)
  - explicit rate for sender to send at



Case study: ATM ABR congestion control

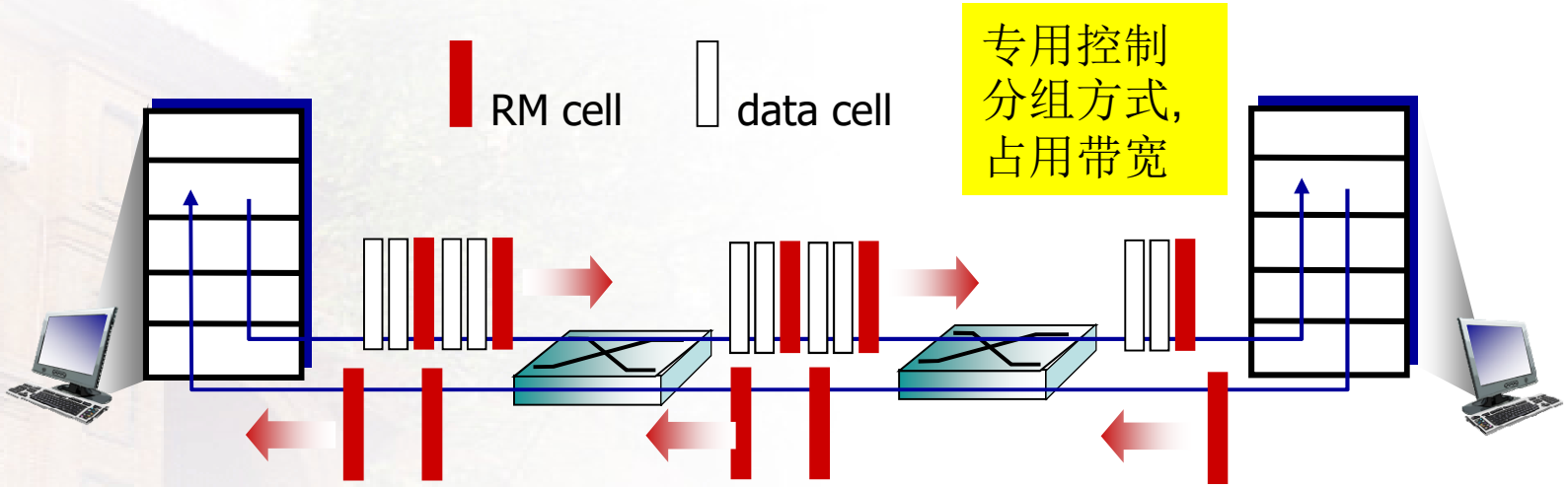
ABR: available bit rate:

- "elastic service"
- if sender's path "underloaded":
  - sender should use available bandwidth
- if sender's path congested:
  - sender throttled to minimum guaranteed rate

RM (resource management) cells:

- sent by sender, interspersed with data cells
- bits in RM cell set by switches ("*network-assisted*")
  - ***NI bit:*** no increase in rate (mild congestion)
  - ***CI bit:*** congestion indication
- RM cells returned to sender by receiver, with bits intact

Case study: ATM ABR congestio control

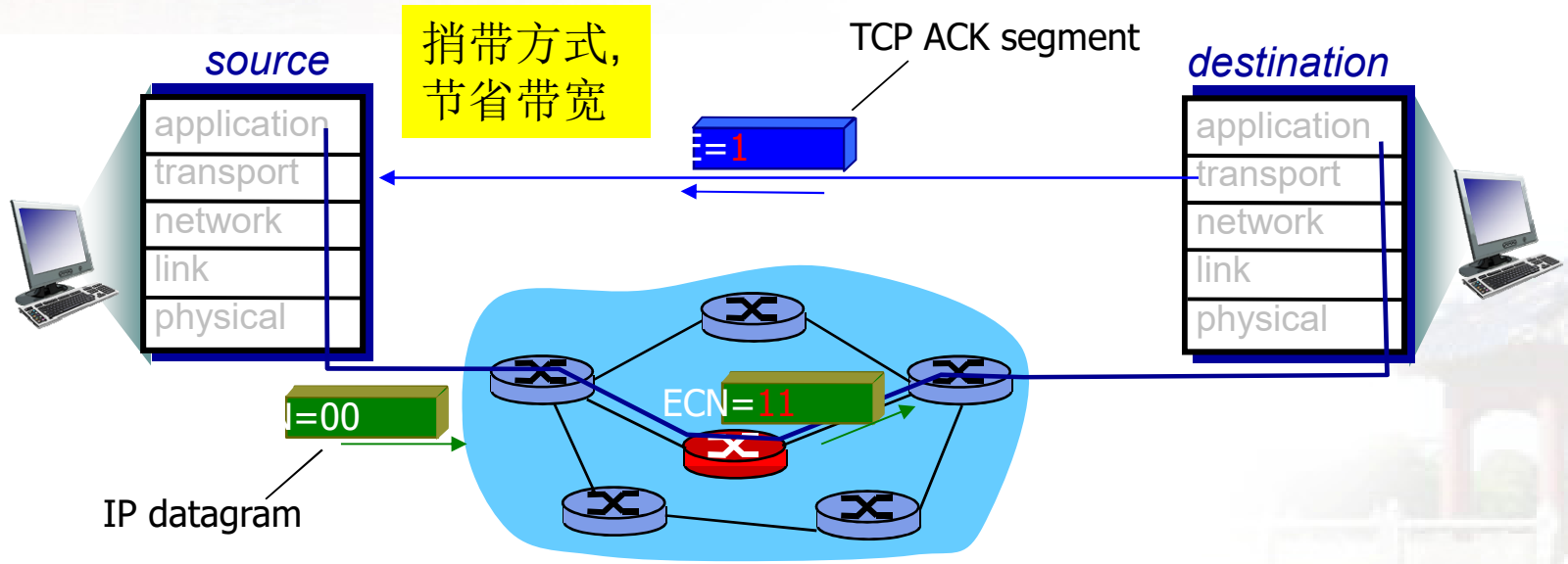


- two-byte ER (explicit rate) field in RM cell
  - congested switch may lower ER value in cell
  - Senders' send rate thus max supportable rate on path
- EFCI bit in data cells: set to 1 in congested switch
  - if data cell preceding RM cell has EFCI set, receiver sets CI bit in returned RM cell

Explicit Congestion Notification (ECN)

network-assisted congestion control:

- two bits in IP header (ToS field) marked *by network router* to indicate congestion
- congestion indication carried to receiving host
- receiver (seeing congestion indication in IP datagram)) sets ECE bit on receiver-to-sender ACK segment to notify sender of congestion



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

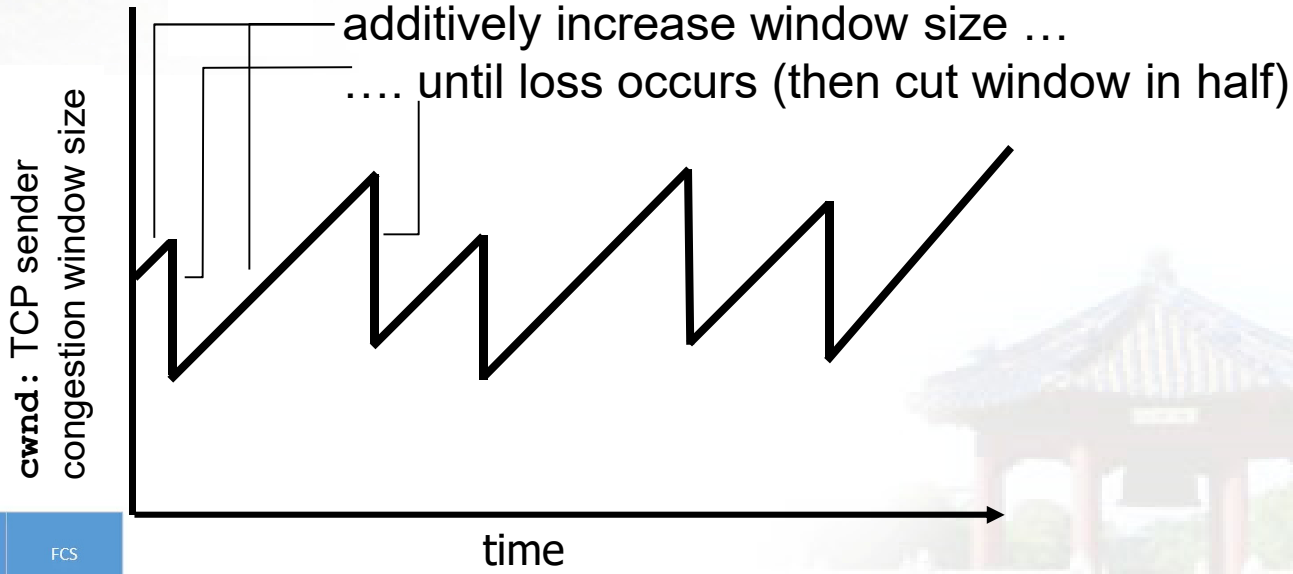
3.6 principles of congestion control

3.7 TCP congestion control

TCP congestion control: additive increase multiplicative decrease

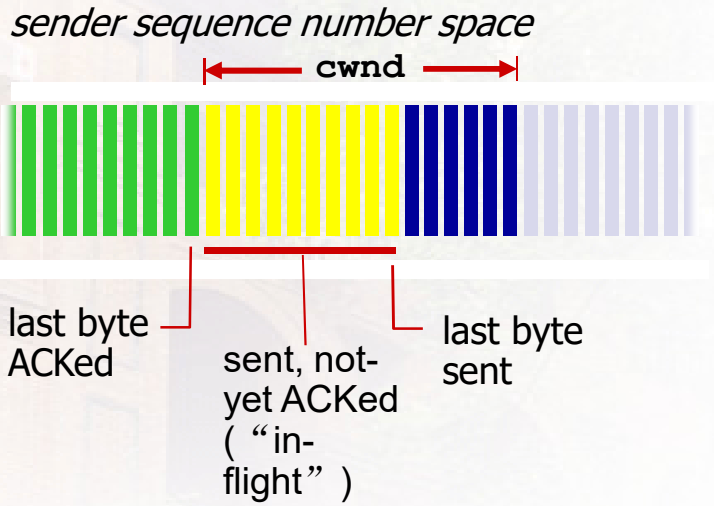
- *approach:* sender increases transmission rate (window size), probing for usable bandwidth, until loss occurs
  - *additive increase:* increase **`cwnd`** by 1 MSS every RTT until loss detected
  - *multiplicative decrease*: cut **`cwnd`** in **half** after loss

AIMD saw tooth behavior: probing for bandwidth





TCP Congestion Control: details



TCP sending rate:

- ***roughly:*** **send cwnd bytes, wait RTT for ACKS, then send more bytes**

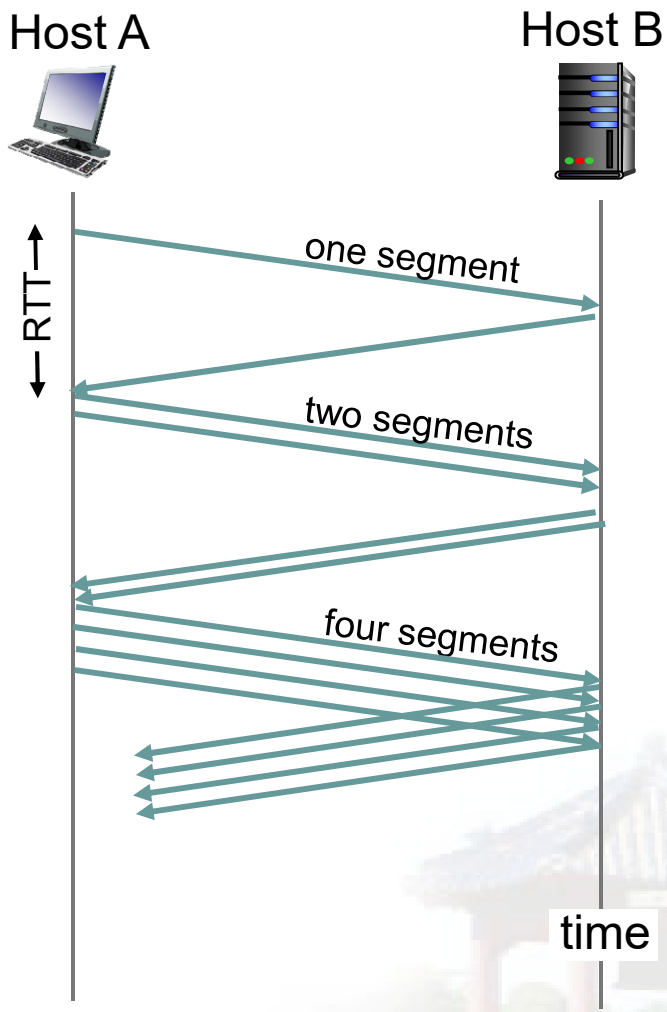
- **sender limits transmission:** $\text{rate} \approx \frac{\text{cwnd}}{\text{RTT}}$ **bytes/sec**

- **`cwnd` is dynamic, function of perceived network congestion**

减小窗口→降低速率→减少网络数据→缓解拥塞

TCP Slow Start

- when connection begins, increase rate exponentially until first loss event:
  - initially `cwnd` = 1 MSS
  - double `cwnd` every RTT
  - done by incrementing `cwnd` for every ACK received
- ***summary:*** initial rate is slow but ramps up exponentially fast



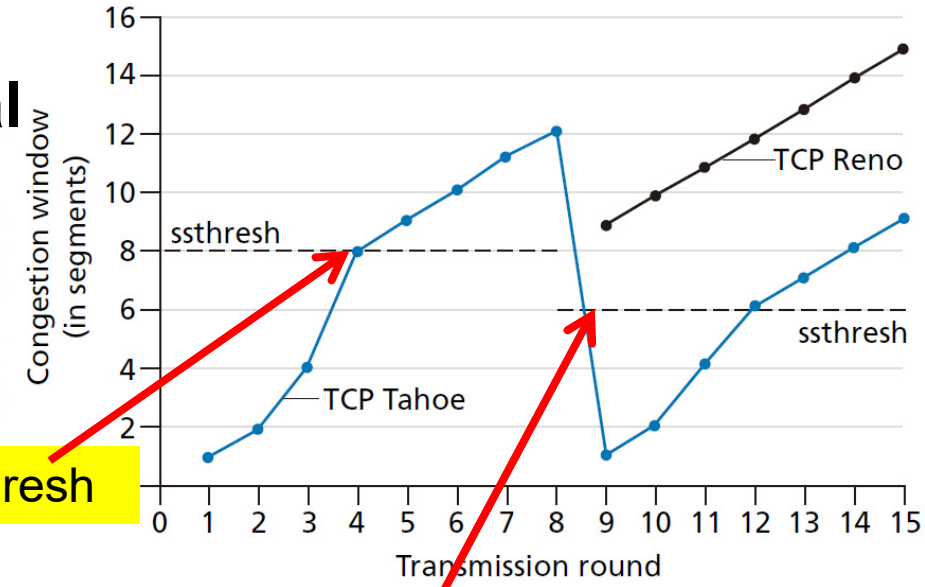
TCP: detecting, reacting to loss

- loss indicated by **timeout**:
  - `cwnd` set to 1 MSS;
  - window then grows **exponentially** (as in slow start) to threshold, then grows linearly
- loss indicated by **3 duplicate ACKs**: TCP RENO
  - dup ACKs indicate network capable of delivering some segments
  - `cwnd` is **cut in half** window then grows **linearly**
- TCP Tahoe always sets `cwnd` to 1 (timeout or 3 duplicate acks)

TCP: switching from slow start to CA

Q: when should the exponential increase switch to linear?

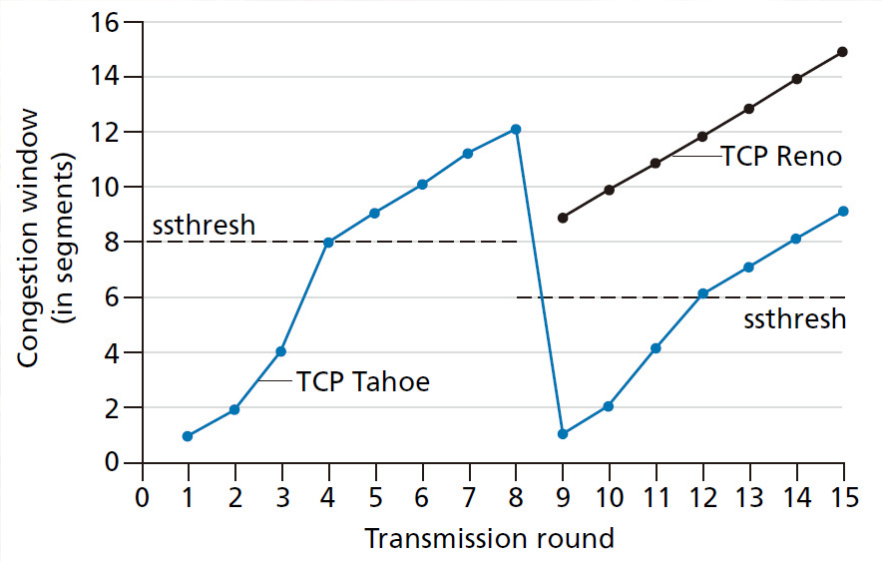
A: when `cwnd` gets to 1/2 of its value before timeout.



Implementation:

- variable `ssthresh`
- on loss event, `ssthresh` is set to 1/2 of `cwnd` just before loss event

Summary: TCP Congestion Control



Increase one MSS for each ack

Increase one MSS when ALL acks of a cwnd are received

New ACK!

duplicate ACK
dupACKcount++

new ACK
cwnd = cwnd+MSS
dupACKcount = 0
transmit new segment(s), as allowed

new ACK
cwnd = cwnd + MSS •(MSS/cwnd)
dupACKcount = 0
transmit new segment(s), as allowed

Λ
cwnd = 1 MSS
ssthresh = 64 KB
dupACKcount = 0

slow start

cwnd ≥ ssthresh
Λ

congestion avoidance

timeout
ssthresh = cwnd/2
cwnd = 1 MSS
dupACKcount = 0
retransmit missing segment

duplicate ACK
dupACKcount++

timeout
ssthresh = cwnd/2
cwnd = 1 MSS
dupACKcount = 0
retransmit missing segment

timeout
ssthresh = cwnd/2
cwnd = 1
dupACKcount = 0
retransmit missing segment

New ACK!

New ACK
cwnd = ssthresh
dupACKcount = 0

dupACKcount == 3
ssthresh= cwnd/2
cwnd = ssthresh + 3
retransmit missing segment

dupACKcount == 3
ssthresh= cwnd/2
cwnd = ssthresh + 3
retransmit missing segment

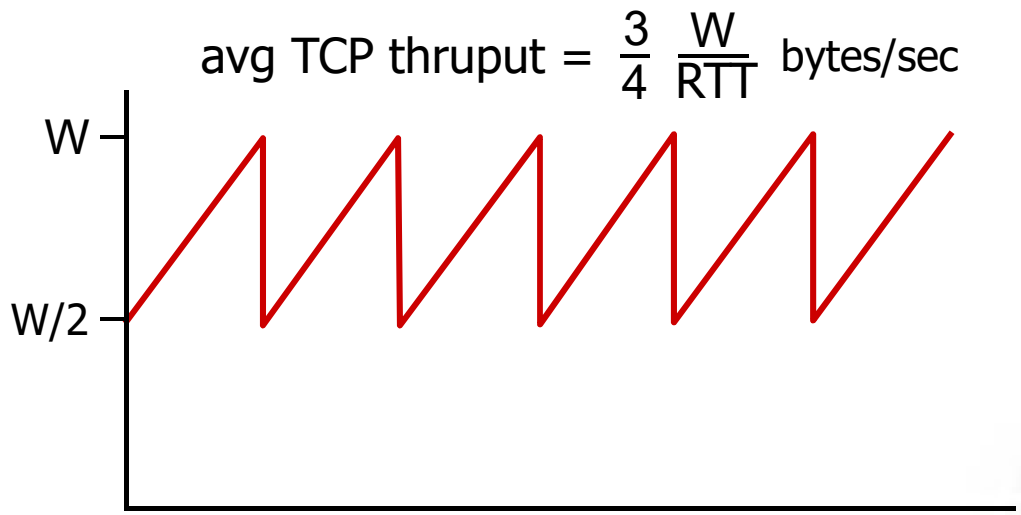
fast recovery

duplicate ACK
cwnd = cwnd + MSS
transmit new segment(s), as allowed

Very Important!!!

TCP throughput

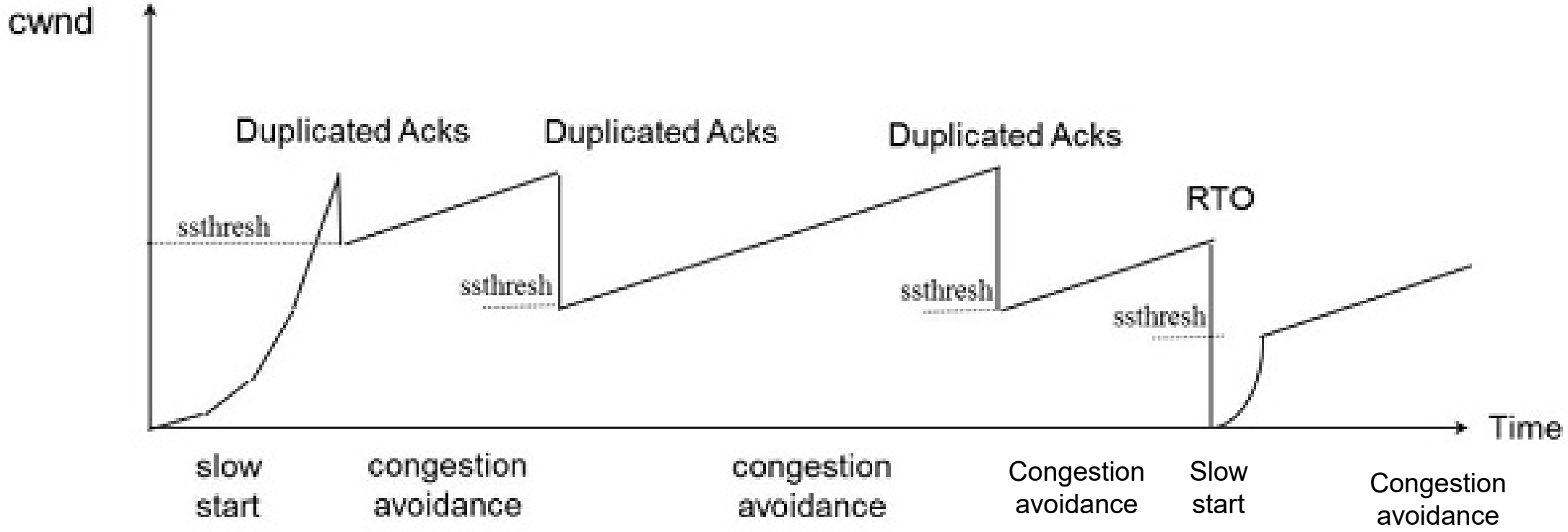
- avg. TCP thruput as function of window size, RTT?
  - ignore slow start, assume always data to send
- W: window size (measured in bytes) where loss occurs
  - avg. window size (# in-flight bytes) is ¾ W
  - avg. thruput is 3/4W per RTT



2023.10.9

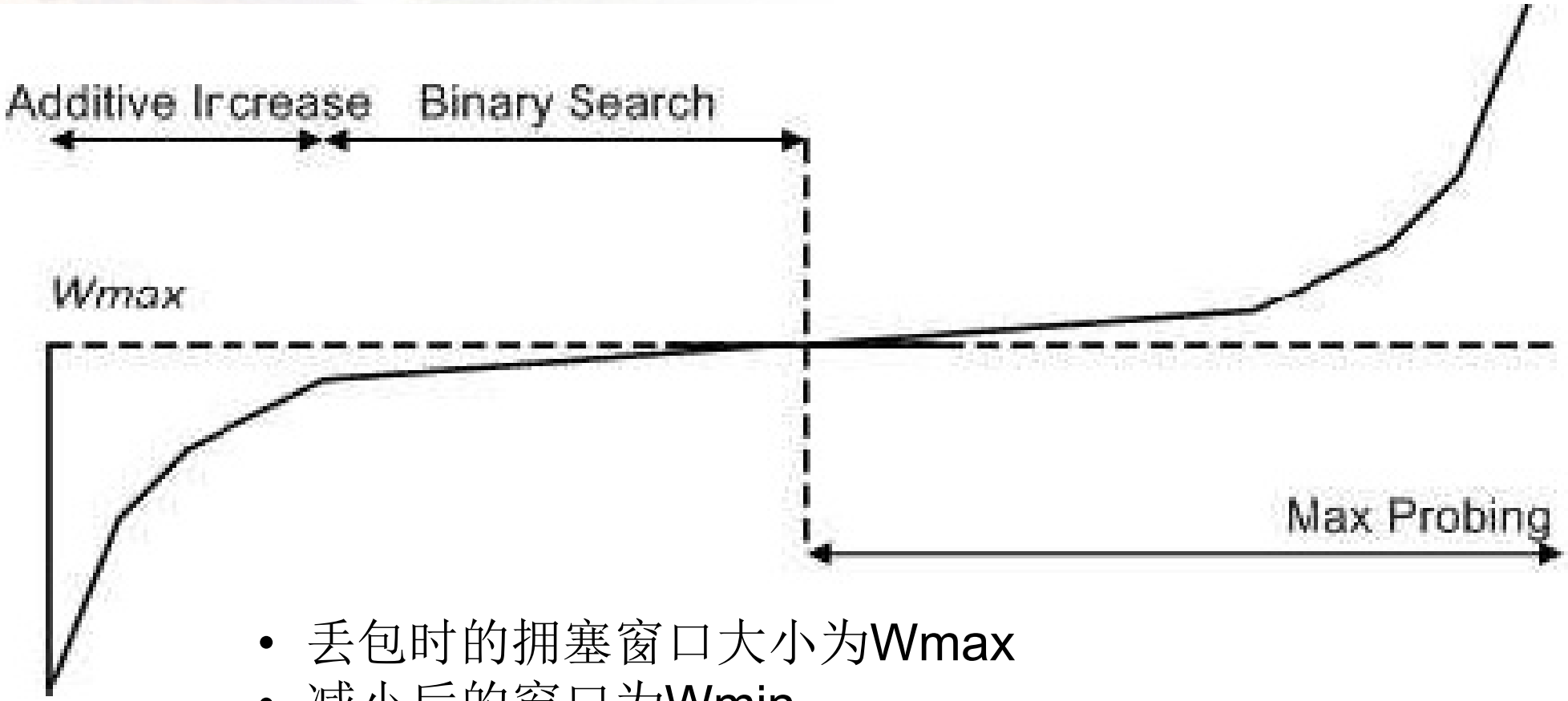
TCP BIC

- **TCP RENO 的问题： 窗口恢复慢**



在reno版本的拥塞控制下，进入拥塞避免状态或快速恢复状态后，每经过一个RTT才会将窗口大小加1，如果链路状况是好的，但RTT很长，reno需要很长时间才能达到最佳拥塞窗口

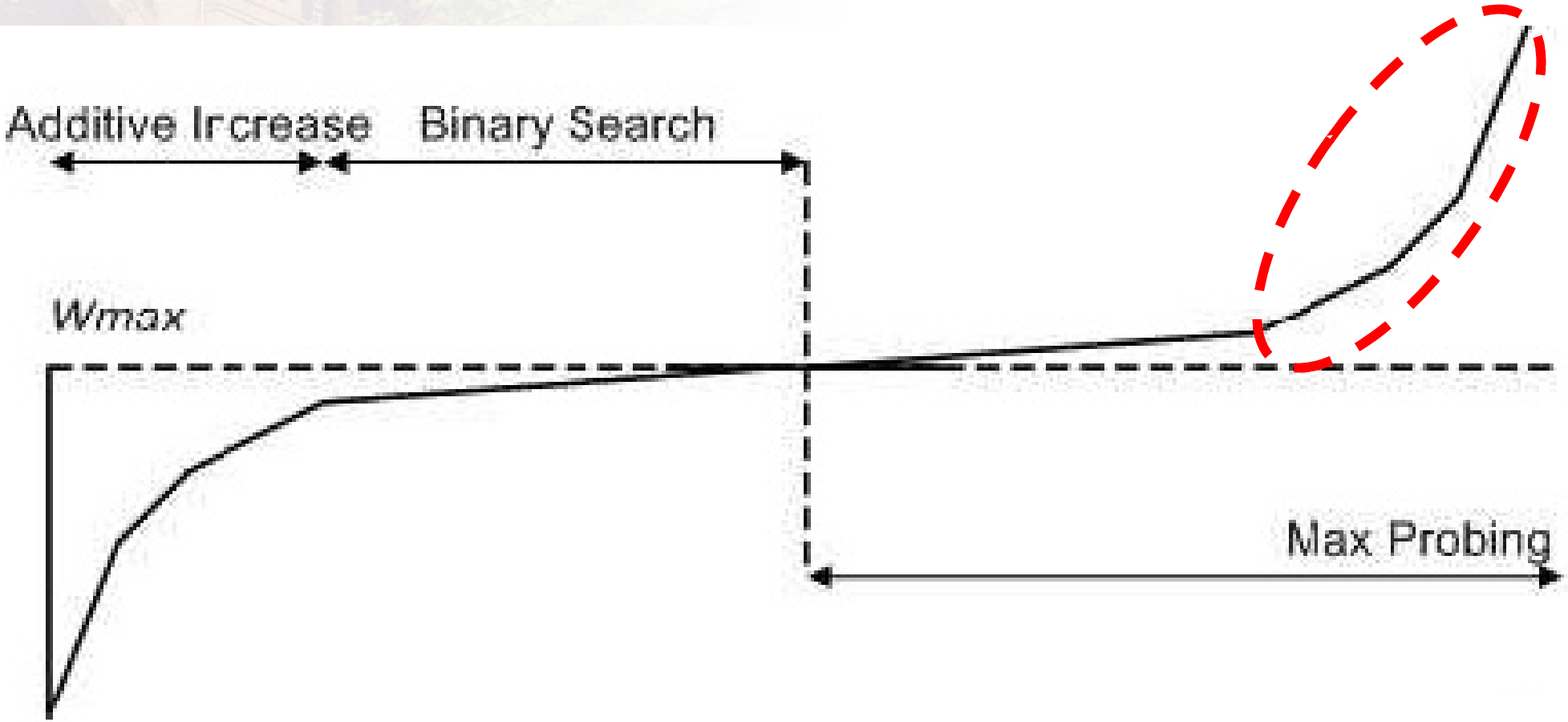
TCP BIC



- 丢包时的拥塞窗口大小为Wmax
- 减小后的窗口为Wmin
- Wmin<W<Wmax
- reno采用加法去搜索W的方式实在是太慢了，因此BIC采用了二分搜索的方式去找到这个W

TCP CUBIC

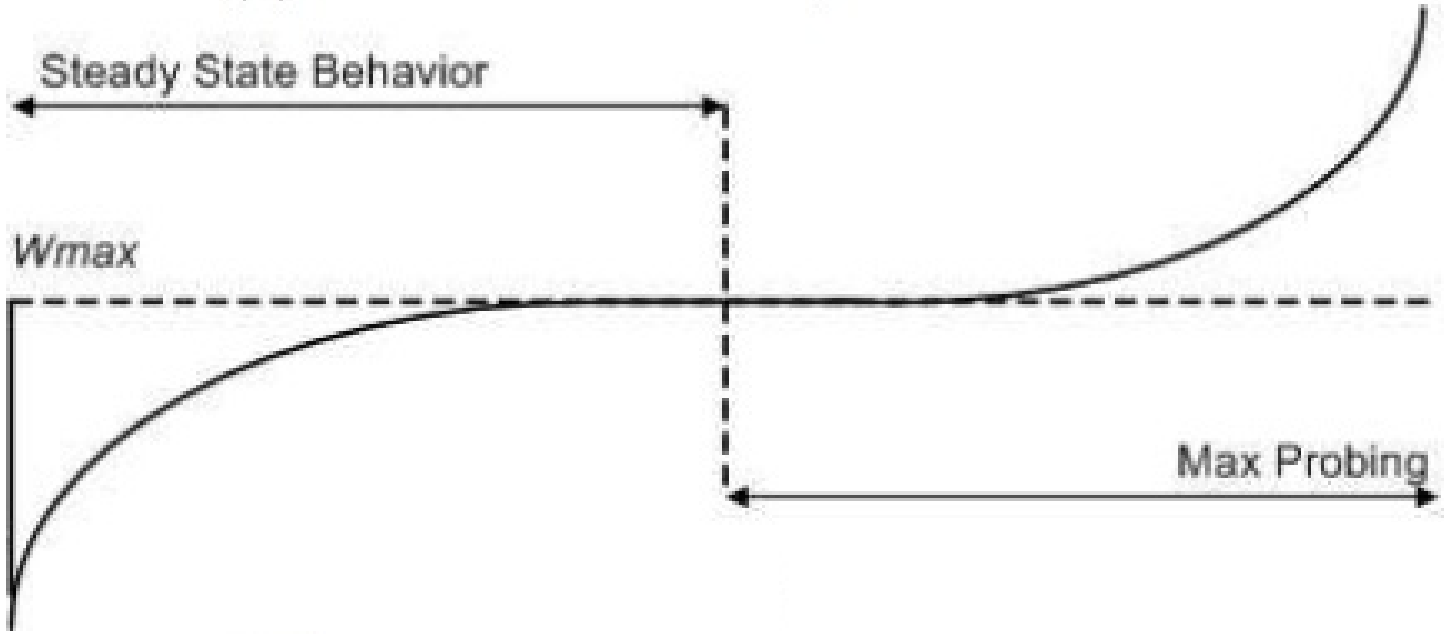
- TCP BIC存在的问题：**BIC不能用到RTT较小，丢包较高的环境中**



TCP CUBIC

- 思路：把窗口调整与RTT脱钩，不再以RTT为单位调整窗口

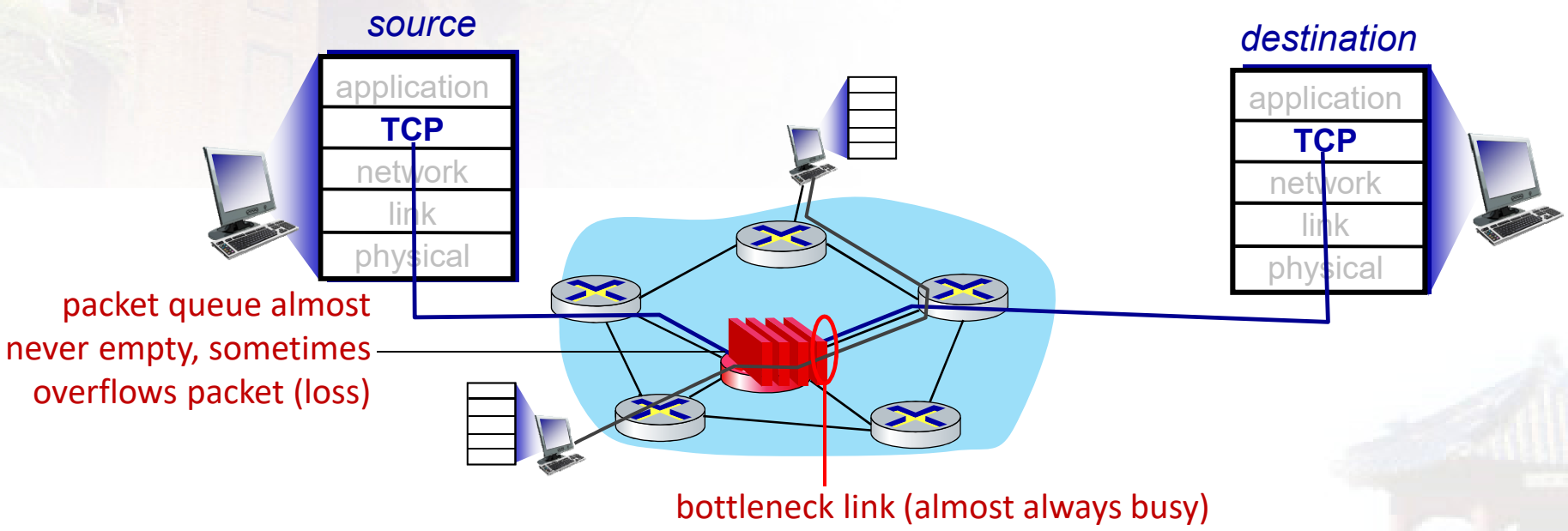
$$W(t) = C(t-K)^3 + W_{\max}$$



t是从窗口上次降低开始到现在的时间,K是从窗口降低到再次到达Wmax的时间.
t=0,W(0)=βWmax

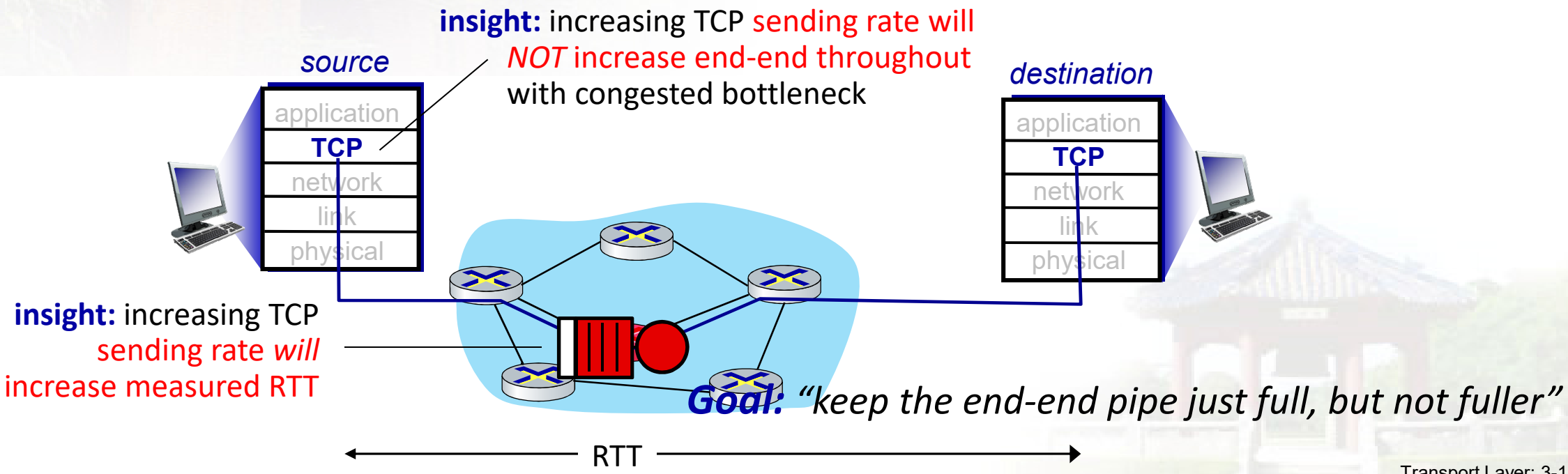
TCP and the congested "bottleneck link"

- TCP (classic, CUBIC) increase TCP's sending rate until packet loss occurs at some router's output: the *bottleneck link*



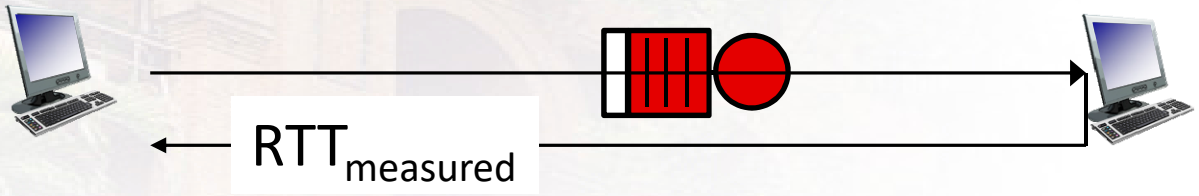
TCP and the congested "bottleneck link"

- TCP (classic, CUBIC) increase TCP's sending rate until packet loss occurs at some router's output: the *bottleneck link*
- understanding congestion: useful to focus on congested bottleneck link



Delay-based TCP congestion control

Keeping sender-to-receiver pipe "just full enough, but no fuller": keep bottleneck link busy transmitting, but avoid high delays/buffering



$$\text{measured throughput} = \frac{\text{\# bytes sent in last RTT interval}}{RTT_{measured}}$$

Delay-based approach:

- RTT_{min} - minimum observed RTT (uncongested path)
- uncongested throughput with congestion window `cwnd` is cwnd/RTT_{min}

if measured throughput "very close" to uncongested throughput
    increase `cwnd` linearly /* since path not congested */
else if measured throughput "far below" uncongested throughout
    decrease `cwnd` linearly /* since path is congested */

Delay-based TCP congestion control

- congestion control without inducing/forcing loss
- maximizing throughout ("keeping the just pipe full… ") while keeping delay low ("…but not fuller")
- a number of deployed TCPs take a delay-based approach
  - BBR deployed on Google's (internal) backbone network

TCP Futures: TCP over "long, fat pipes"

- example: 1500 byte segments, 100ms RTT, want 10 Gbps throughput
- requires W = 83,333 in-flight segments
- throughput in terms of segment loss probability, L [Mathis 1997]:

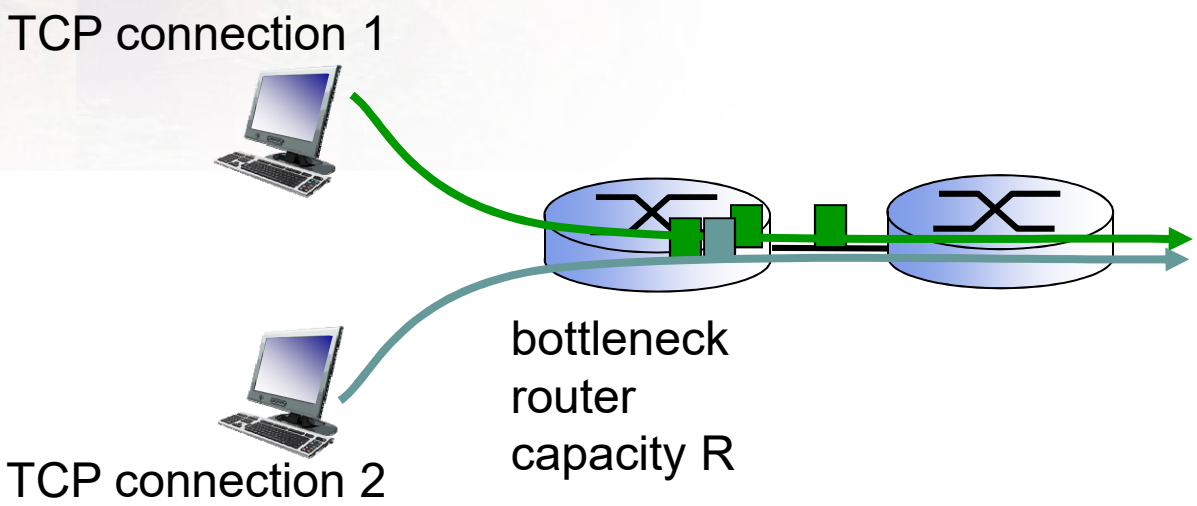
$$\text{TCP throughput} = \frac{1.22 \cdot \text{MSS}}{\text{RTT}\sqrt{L}}$$

➜ to achieve 10 Gbps throughput, need a loss rate of $L = 2 \cdot 10^{-10}$ – *a very small loss rate!*

- new versions of TCP for high-speed

TCP Fairness

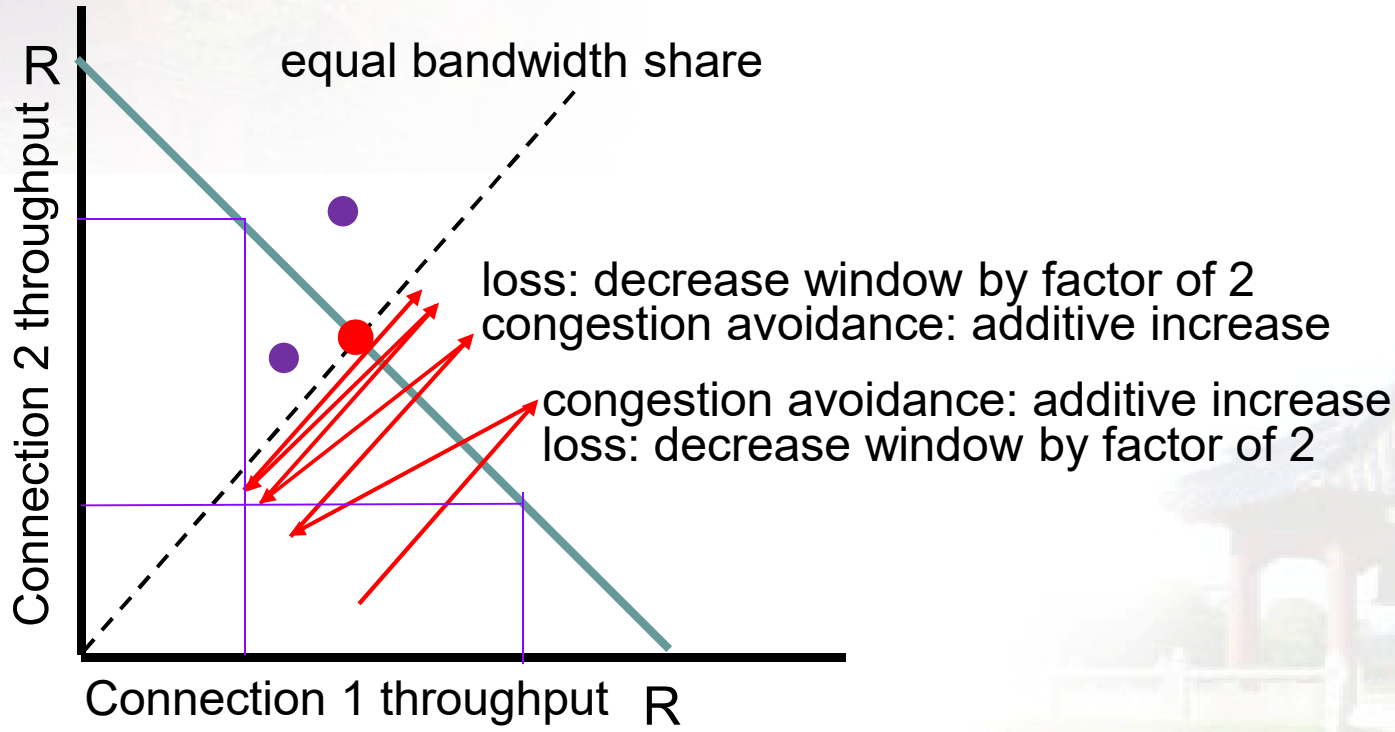
fairness goal: **if K TCP sessions share same bottleneck link of bandwidth R, each should have average rate of R/K**



Why is TCP fair?

two competing sessions:

- **additive increase gives slope of 1, as throughout increases**
- **multiplicative decrease decreases throughput proportionally**



Fairness (more)

Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss

Fairness, parallel TCP connections

- application can open multiple parallel connections between two hosts
- web browsers do this
- e.g., link of rate R with 9 existing connections:
  - new app asks for 1 TCP, gets rate R/10
  - new app asks for 11 TCPs, gets R/2

Basic Control Model

- Let's assume window-based operation
- Reduce window when congestion is perceived
  - How is congestion signaled?
    - Either mark or drop packets
  - When is a router congested?
    - Drop tail queues – when queue is full
    - Average queue length – at some threshold
- Increase window otherwise
  - Probe for available bandwidth – how?

Simple linear control

- Many different possibilities for reaction to congestion and methods for probing
  - Examine simple linear controls
  - Window(t + 1) = a + b Window(t)
  - Different a_i/b_i for increase and a_d/b_d for decrease

Simple linear control

$$x_i(t+1) = \begin{cases} a_I + b_I x_i(t) & increase \\ a_D + b_D x_i(t) & decrease \end{cases}$$

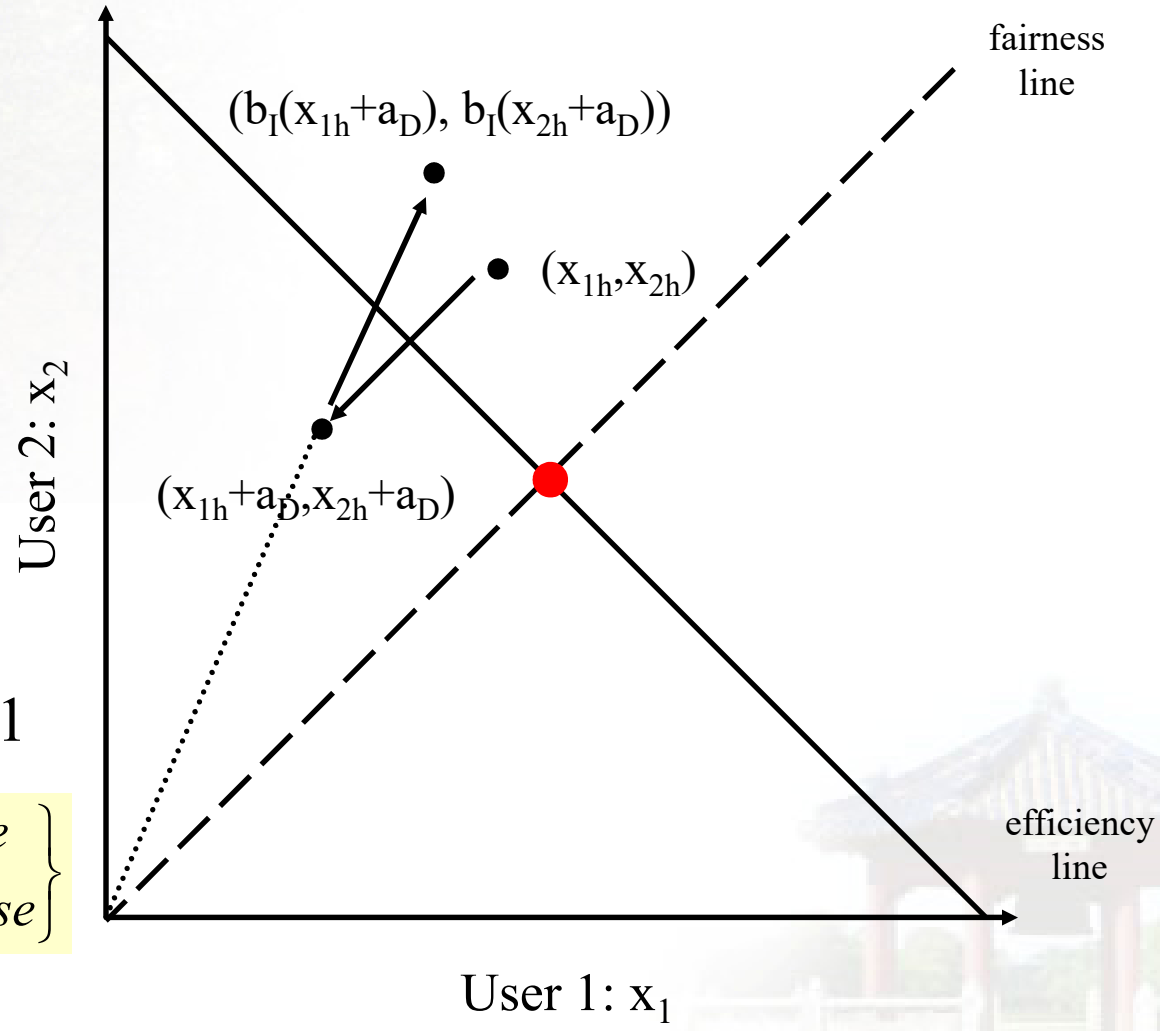
- Multiplicative increase, additive decrease
  - $a_I=0, b_I>1;$ $a_D<0, b_D=1$
- Additive increase, additive decrease
  - $a_I>0, b_I=1;$ $a_D<0, b_D=1$
- Multiplicative increase, multiplicative decrease
  - $a_I=0, b_I>1;$ $a_D=0, 0<b_D<1$
- Additive increase, multiplicative decrease
  - $a_I>0, b_I=1;$ $a_D=0, 0<b_D<1$
- Which one?

Multiplicative Increase, Additive Decrease

- Does not converge to fairness
  - Not stable at all
- Does not converges to efficiency

$a_I=0,\ b_I>1,\ a_D<0,\ b_D=1$

$$x_i(t+1) = \begin{cases} b_I x_i(t) & increase \\ a_D + x_i(t) & decrease \end{cases}$$

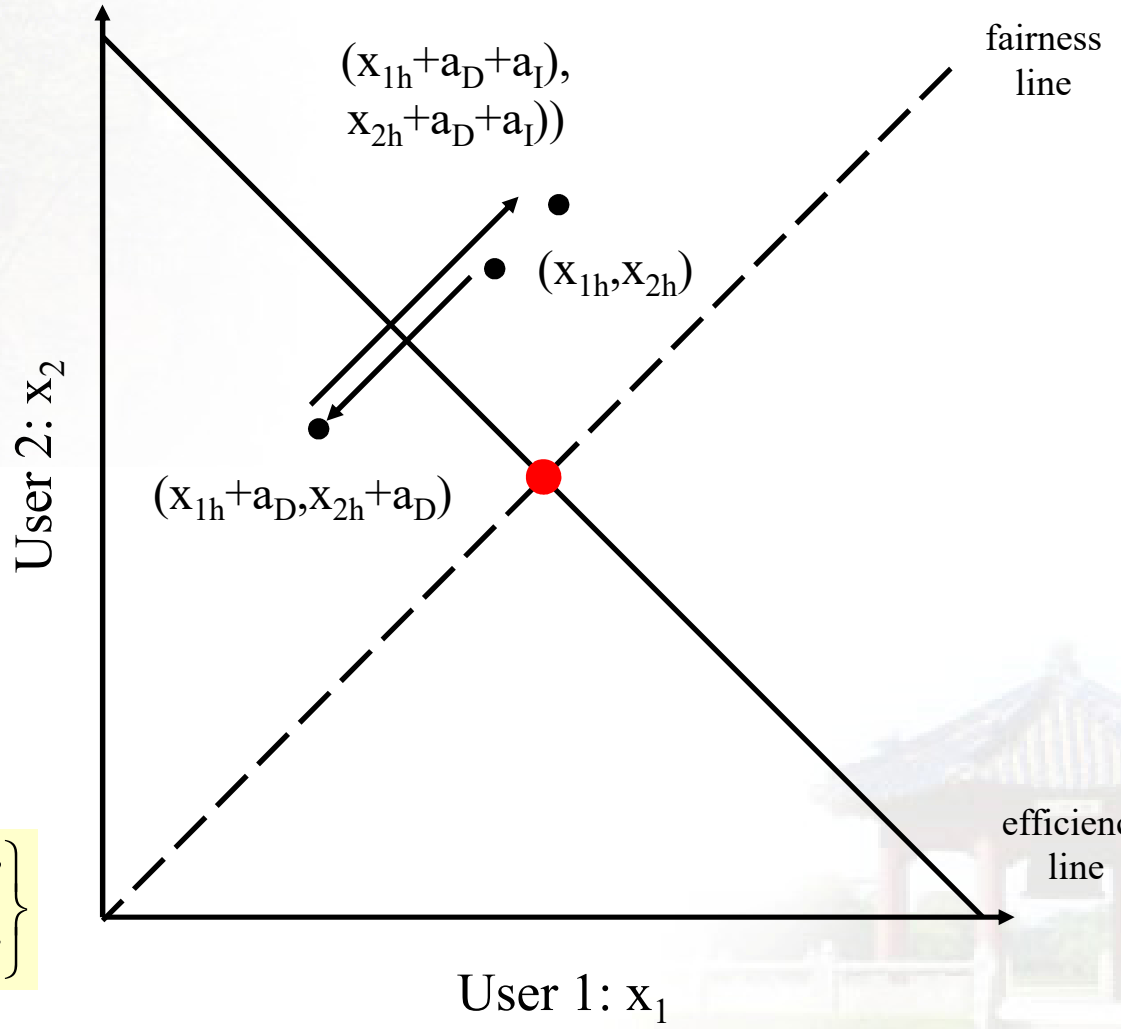


Additive Increase, Additive Decrease

- **Does not converge to fairness**
- **Does not converge to efficiency**

$a_I>0$, $b_I=1$, $a_D<0$, $b_D=1$

$$x_i(t+1) = \begin{cases} a_I + x_i(t) & increase \\ a_D + x_i(t) & decrease \end{cases}$$



Multiplicative Increase, Multiplicative Decrease

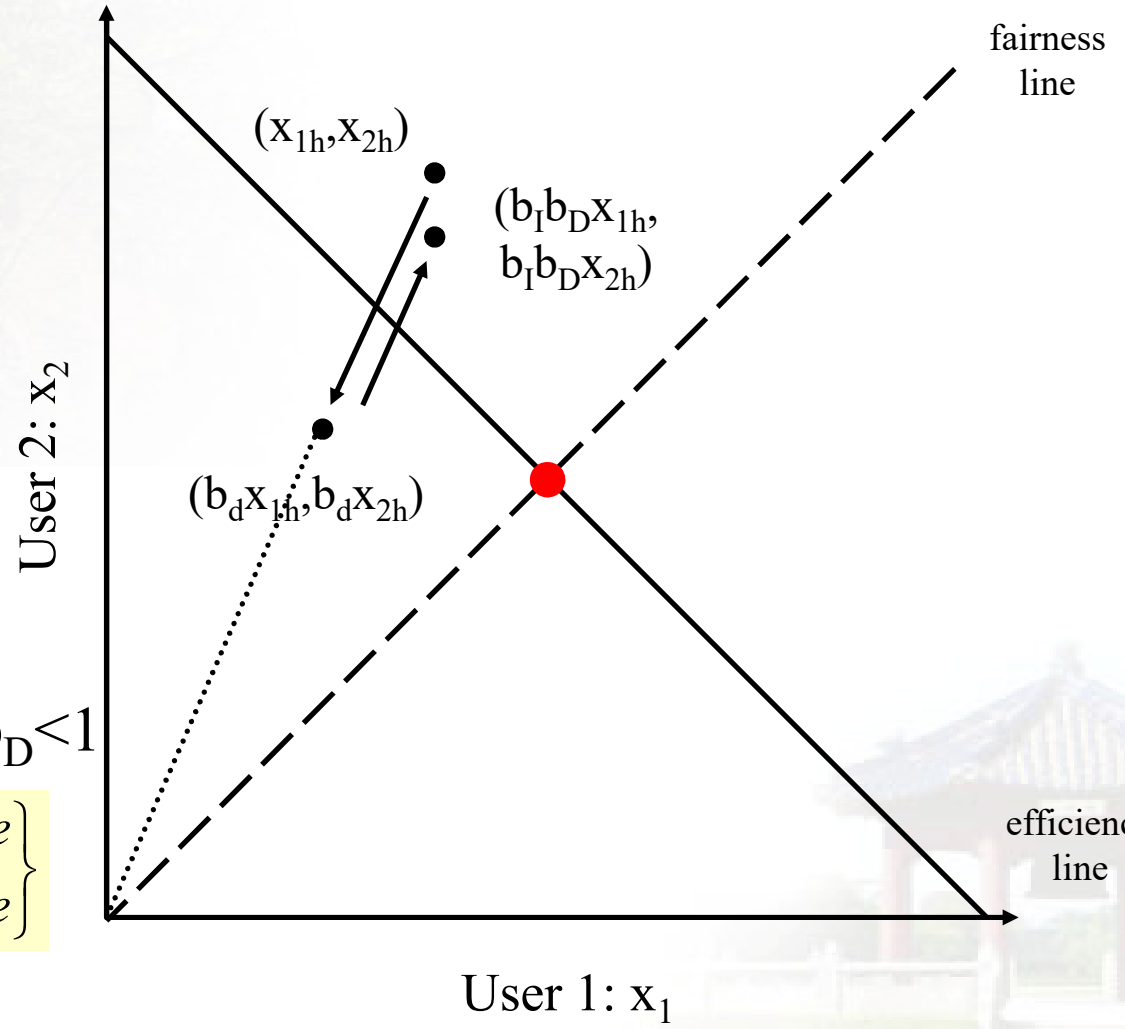
- **Does not converge to fairness**
- **Converges to efficiency iff**

$$b_I \geq 1$$

$$0 \leq b_D < 1$$

$a_I=0$, $b_I>1$, $a_D=0$, $0<b_D<1$

$$x_i(t+1) = \begin{cases} b_I x_i(t) & increase \\ b_D x_i(t) & decrease \end{cases}$$



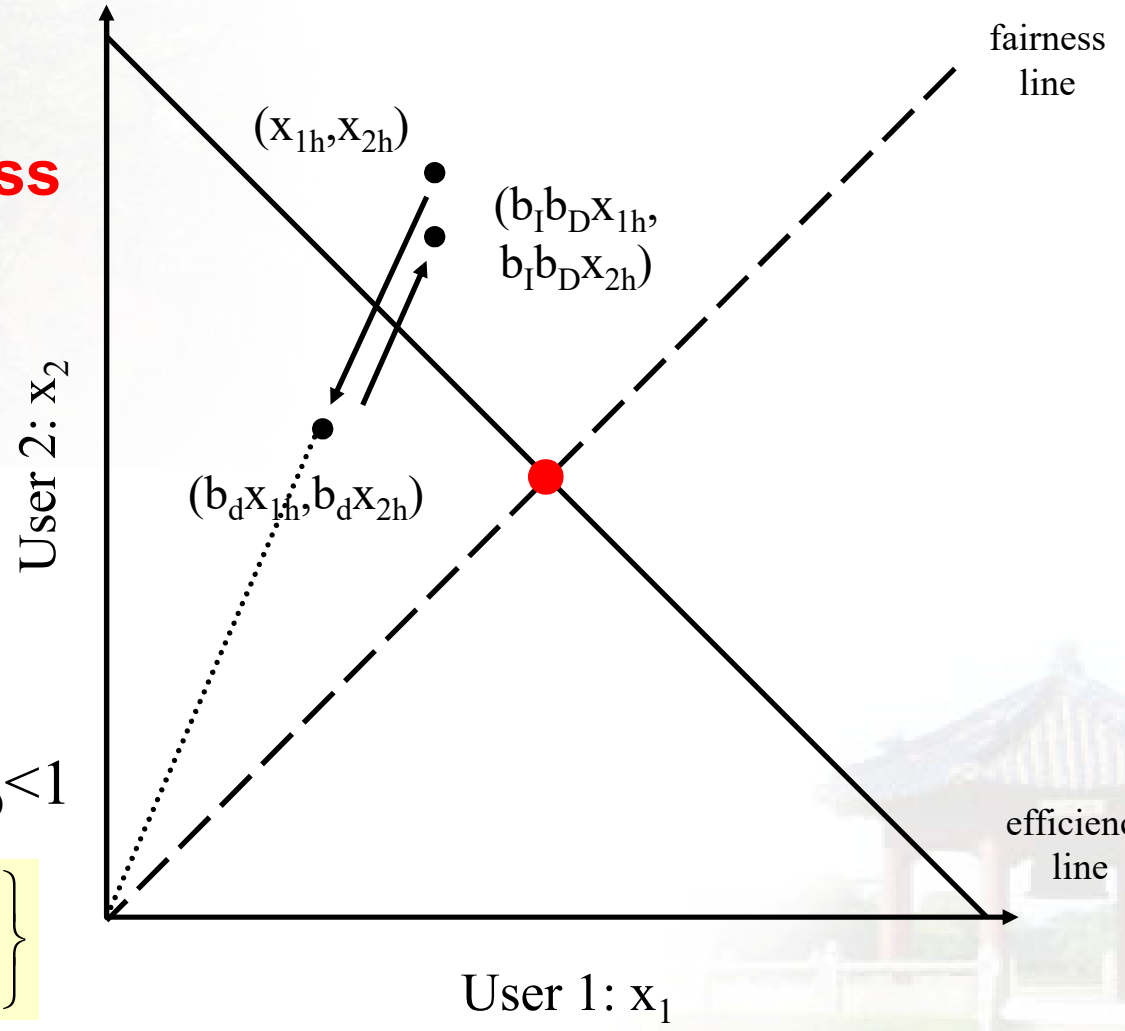
Multiplicative Increase, Multiplicative Decrease

- **Does not converge to fairness**
- **Converges to efficiency iff**

$$b_I \geq 1$$
$$0 \leq b_D < 1$$

$a_I=0$, $b_I>1$, $a_D=0$, $0<b_D<1$

$$x_i(t+1) = \begin{cases} b_I x_i(t) & increase \\ b_D x_i(t) & decrease \end{cases}$$

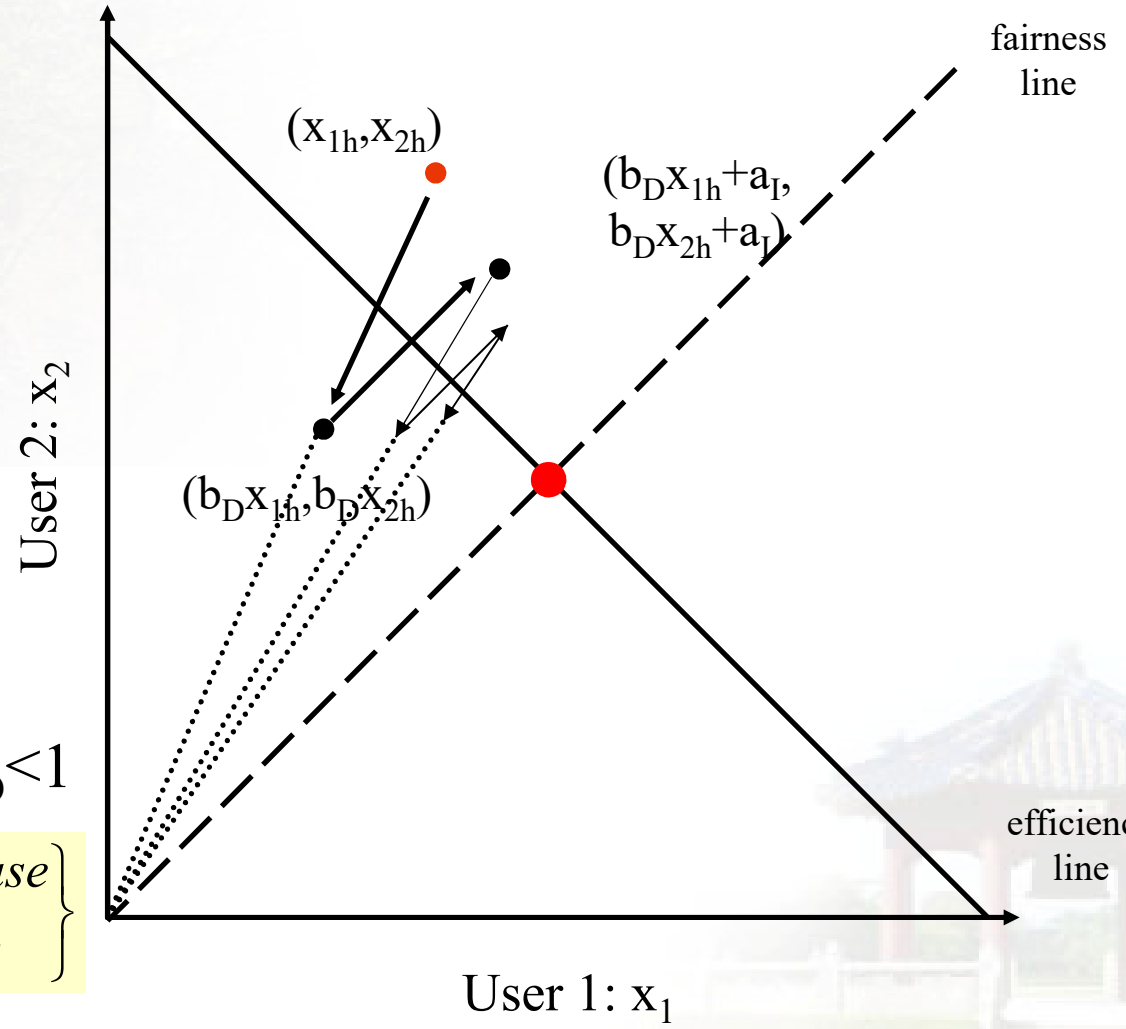


Additive Increase, Multiplicative Decrease

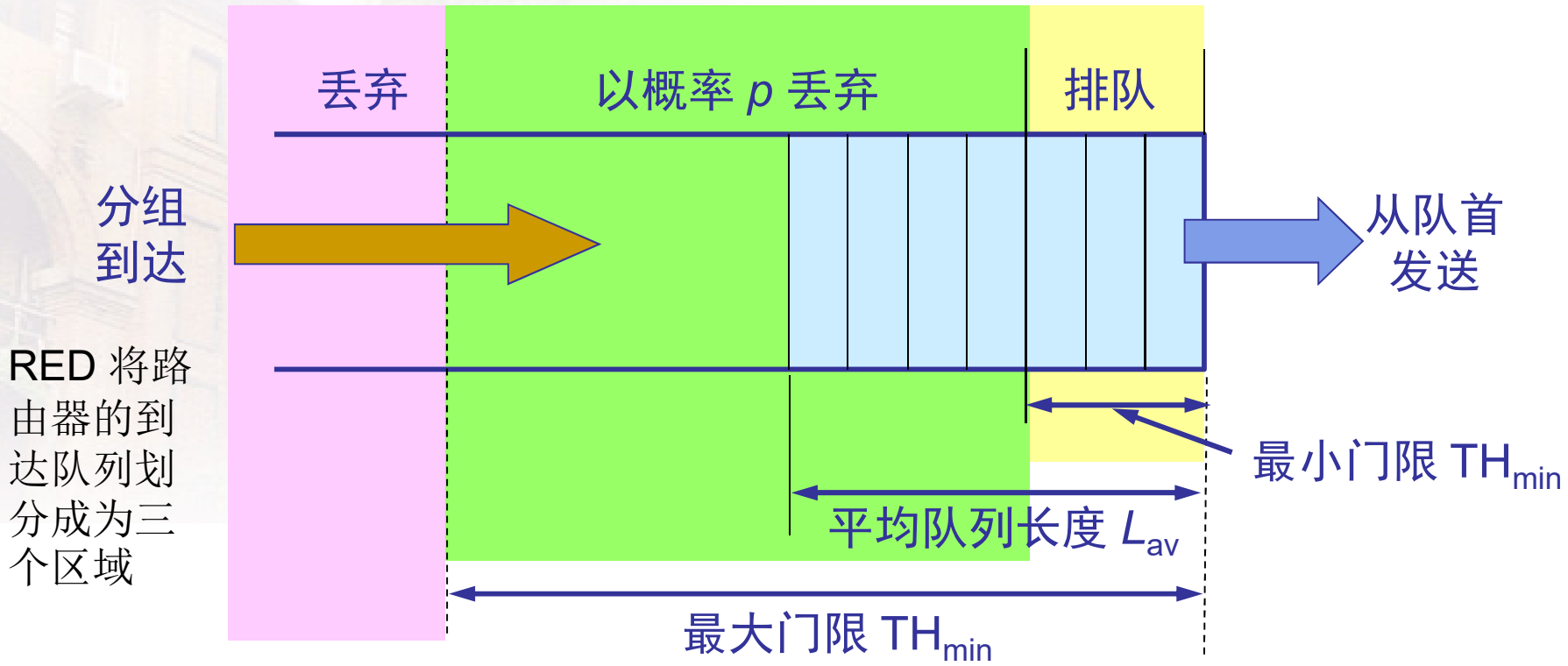
- Converges to fairness
- Converges to efficiency
- Increments smaller as fairness increases

$a_I>0$, $b_I=1$, $a_D=0$, $0<b_D<1$

$$x_i(t+1) = \begin{cases} a_I + b_I x_i(t) & increase \\ b_D x_i(t) & decrease \end{cases}$$



随机早期检测 RED (Random Early Detection)

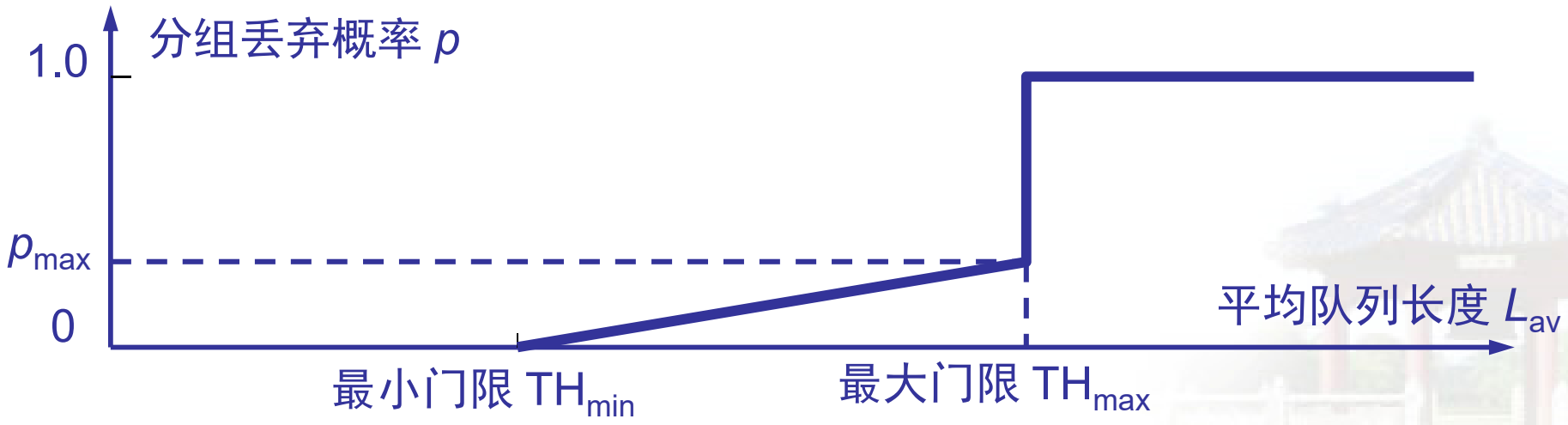


- 使路由器的队列维持两个参数，即队列长度最小门限 TH_{min} 和最大门限 TH_{max}。
- **RED** 对每一个到达的数据报都先计算平均队列长度 L_{AV}。
- 若平均队列长度小于最小门限 TH_{min}，则将新到达的数据报放入队列进行排队。
- 若平均队列长度超过最大门限 TH_{max}，则将新到达的数据报丢弃。
- 若平均队列长度在最小门限 TH_{min} 和最大门限TH_{max} 之间，则按照某一概率 ***p*** 将新到达的数据报丢弃。

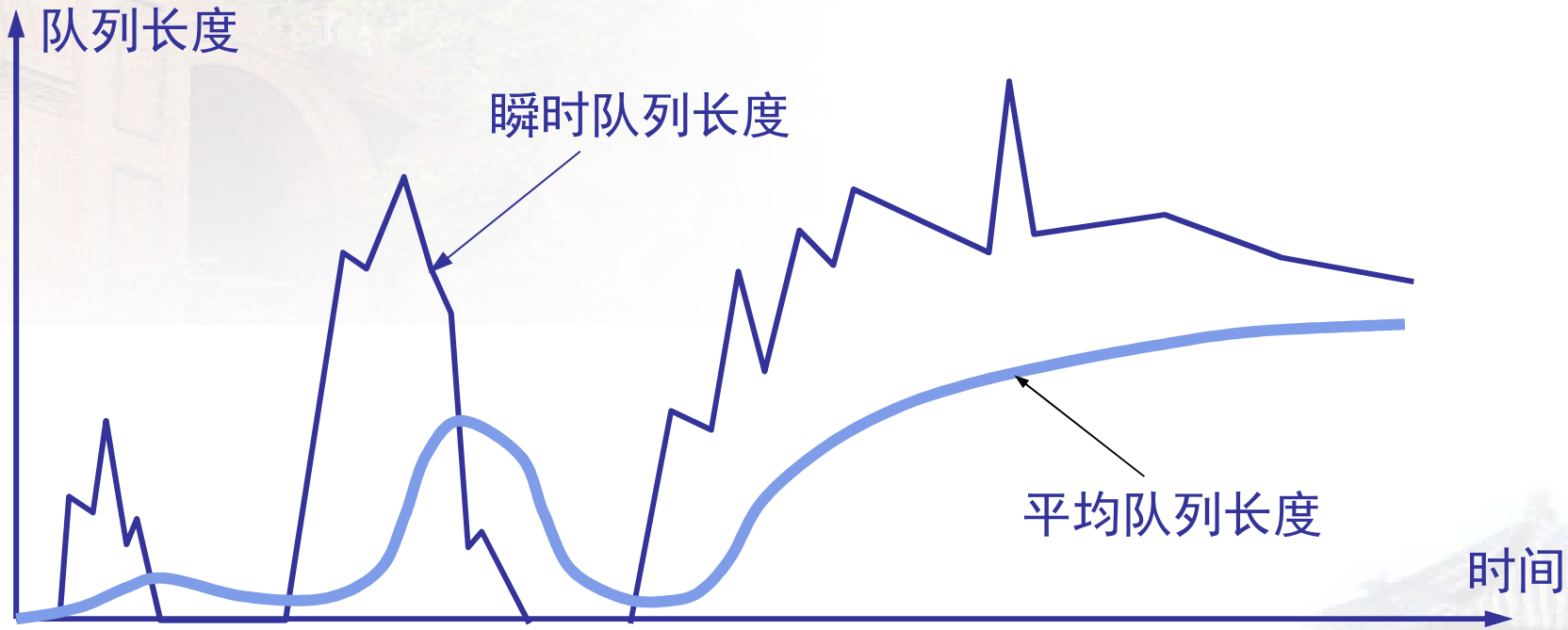
丢弃概率 p 与 TH_{min} 和 Th_{max} 的关系

- 当 $L_{AV} < Th_{min}$ 时，丢弃概率 $p = 0$。
- 当 $L_{AV} > Th_{max}$ 时，丢弃概率 $p = 1$。
- 当 $TH_{min} < L_{AV} < TH_{max}$ 时，$0 < p < 1$。

  例如，按线性规律变化，从 0 变到 p_{max}。



瞬时队列长度和平均队列长度的区别



Evolving transport-layer functionality

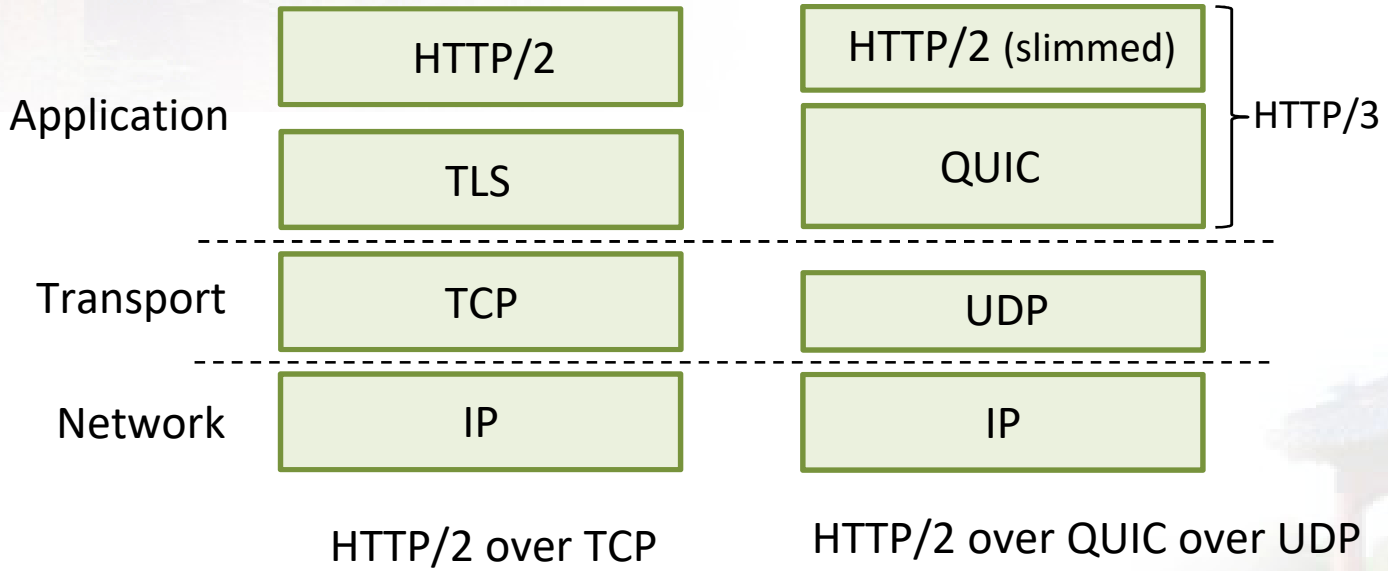
- **TCP, UDP: principal transport protocols for 40 years**
- **different "flavors" of TCP developed, for specific scenarios:**

Scenario	Challenges
Long, fat pipes (large data transfers)	Many packets "in flight"; loss shuts down pipeline
Wireless networks	Loss due to noisy wireless links, mobility; TCP treat this as congestion loss
Long-delay links	Extremely long RTTs
Data center networks	Latency sensitive
Background traffic flows	Low priority, "background" TCP flows

- moving transport–layer functions to application layer, on top of UDP
  - HTTP/3: QUIC

QUIC: Quick UDP Internet Connections

- application-layer protocol, on top of UDP
  - increase performance of HTTP
  - deployed on many Google servers, apps (Chrome, mobile YouTube app)



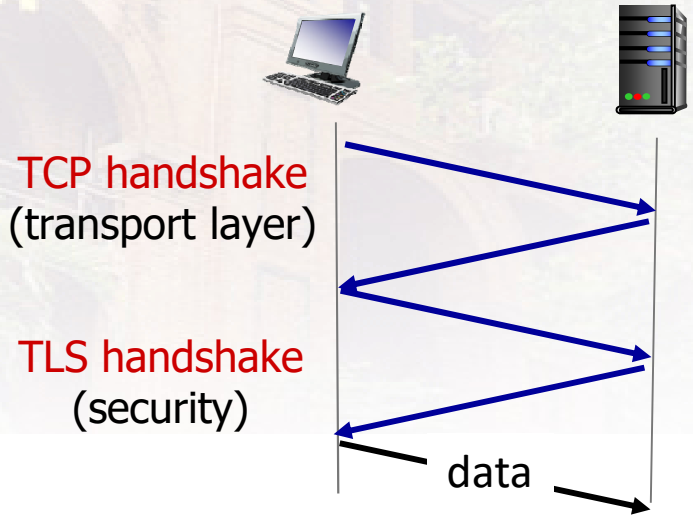
QUIC: Quick UDP Internet Connections

adopts approaches we've studied in this chapter for connection establishment, error control, congestion control

- **error and congestion control:** "Readers familiar with TCP's loss detection and congestion control will find algorithms here that parallel well-known TCP ones." [from QUIC specification]
- **connection establishment:** reliability, congestion control, authentication, encryption, state established in one RTT

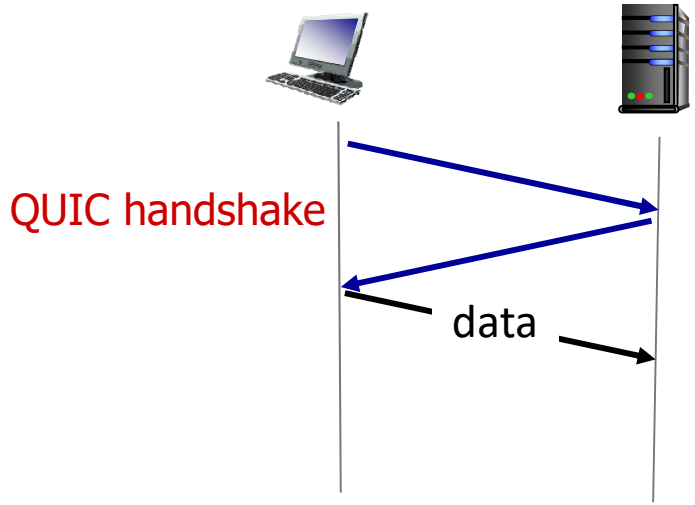
- **multiple application-level "streams" multiplexed over single QUIC connection**
  - **separate reliable data transfer, security**
  - **common congestion control**

QUIC: Connection establishment



TCP (reliability, congestion control state) + TLS (authentication, crypto state)

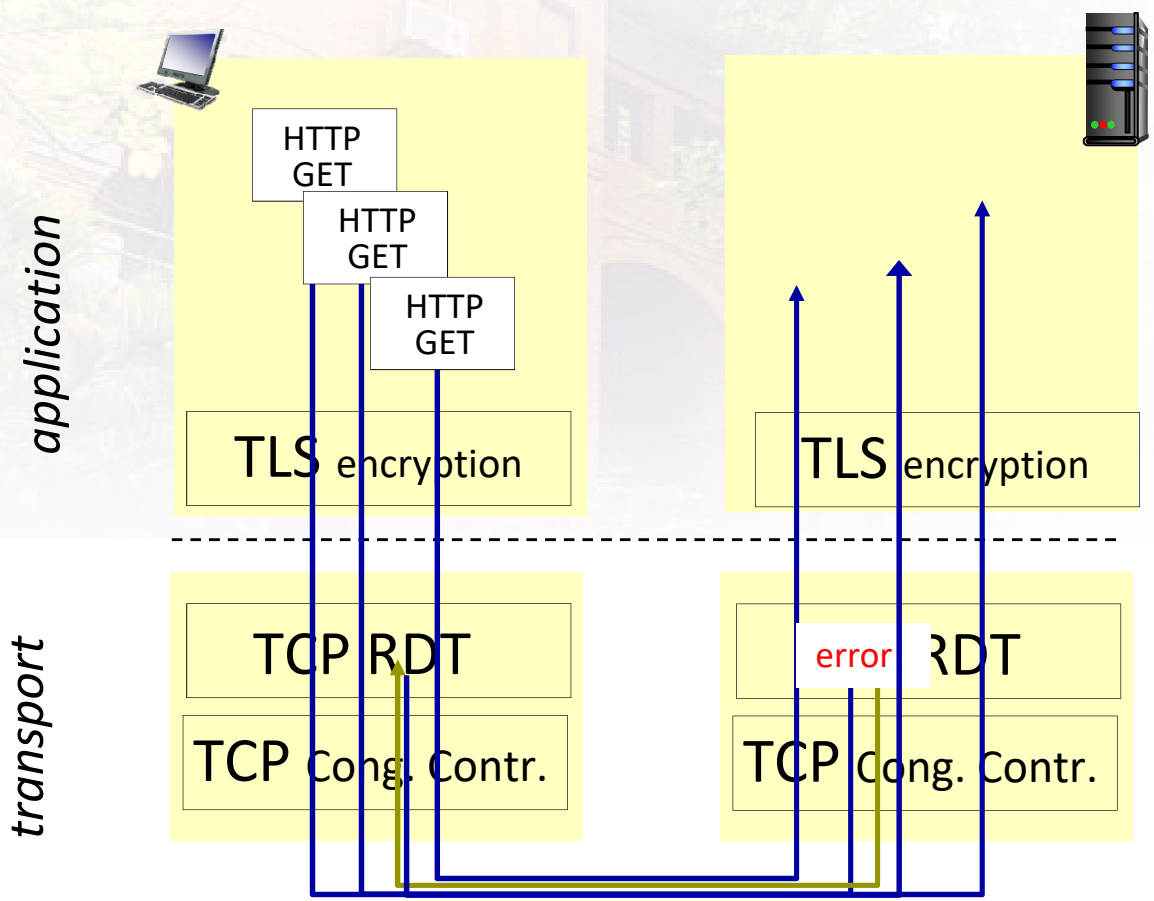
- 2 serial handshakes



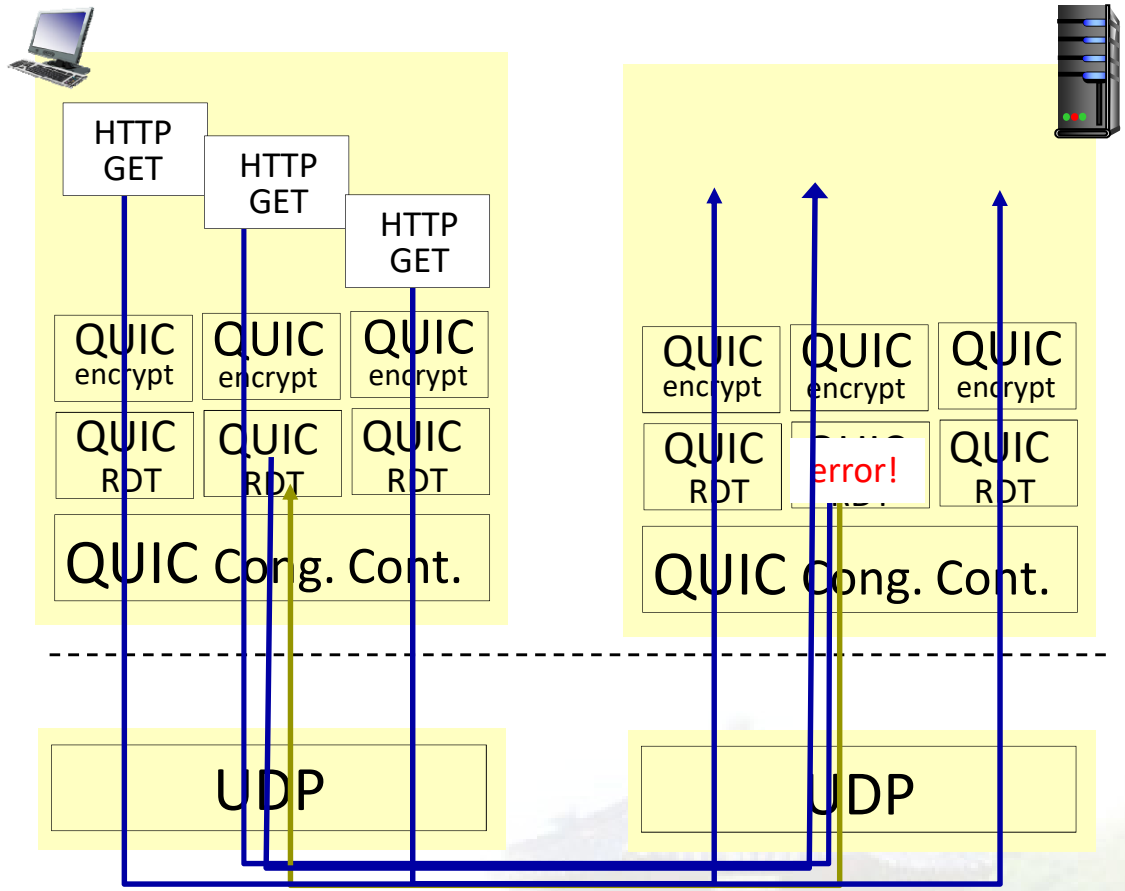
QUIC: reliability, congestion control, authentication, crypto state

- 1 handshake

QUIC: streams: parallelism, no HOL blocking



(a) HTTP 1.1



(b) HTTP/2 with QUIC: **no HOL blocking**

Thanks

Q & A

Email: xieyi5@mail.sysu.edu.cn
https://cse.sysu.edu.cn/content/2462